

System V Application Binary Interface
AMD64 Architecture Processor Supplement
(With LP64 and ILP32 Programming Models)
Version 1.0

Edited by
H.J. Lu¹, Michael Matz², Milind Girkar³, Jan Hubička⁴,
Andreas Jaeger⁵, Mark Mitchell⁶

March 12, 2025

¹hongjiu.lu@intel.com

²matz@suse.de

³milind.girkar@intel.com

⁴jh@suse.cz

⁵aj@suse.de

⁶mark@codesourcery.com

Contents

1	Introduction	12
2	Software Installation	13
3	Low Level System Information	14
3.1	Machine Interface	14
3.1.1	Processor Architecture	14
3.1.2	Data Representation	16
3.2	Function Calling Sequence	20
3.2.1	Registers	21
3.2.2	The Stack Frame	22
3.2.3	Parameter Passing	23
3.3	Operating System Interface	30
3.3.1	Exception Interface	30
3.3.2	Virtual Address Space	30
3.3.3	Page Size	32
3.3.4	Virtual Address Assignments	32
3.4	Process Initialization	33
3.4.1	Initial Stack and Register State	33
3.4.2	Thread State	36
3.4.3	Auxiliary Vector	37
3.5	Coding Examples	40
3.5.1	Architectural Constraints	40
3.5.2	Conventions	43
3.5.3	Position-Independent Function Prologue	43
3.5.4	Data Objects	44
3.5.5	Function Calls	50
3.5.6	Branching	53

3.5.7	Variable Argument Lists	56
3.6	DWARF Definition	60
3.6.1	DWARF Release Number	61
3.6.2	DWARF Register Number Mapping	61
3.7	Stack Unwind Algorithm	61
4	Object Files	64
4.1	ELF Header	64
4.1.1	Machine Information	64
4.1.2	Number of Program Headers	65
4.2	Sections	65
4.2.1	Section Flags	65
4.2.2	Section types	66
4.2.3	Special Sections	66
4.2.4	EH_FRAME sections	67
4.3	Symbol Table	71
4.4	Relocation	72
4.4.1	Relocation Types	72
4.4.2	Large Models	78
5	Program Loading and Dynamic Linking	79
5.1	Program Loading	79
5.1.1	Program header	80
5.2	Dynamic Linking	80
5.2.1	Program Interpreter	86
5.2.2	Initialization and Termination Functions	87
5.3	Program Property	87
6	Libraries	92
6.1	C Library	92
6.1.1	Global Data Symbols	92
6.1.2	Floating Point Environment Functions	92
6.2	Unwind Library Interface	92
6.2.1	Exception Handler Framework	93
6.2.2	Data Structures	95
6.2.3	Throwing an Exception	97
6.2.4	Exception Object Management	100
6.2.5	Context Management	100
6.2.6	Personality Routine	103

6.3	Unwinding Through Assembler Code	106
7	Development Environment	109
8	Execution Environment	110
9	Conventions	111
9.1	C++	112
9.2	Fortran	113
9.2.1	Names	113
9.2.2	Representation of Fortran Types	114
9.2.3	Argument Passing	115
9.2.4	Functions	116
9.2.5	COMMON blocks	117
9.2.6	Intrinsics	118
10	ILP32 Programming Model	128
10.1	Parameter Passing	128
10.2	Address Space	128
10.3	Thread-Local Storage Support	128
10.3.1	Global Thread-Local Variable	128
10.3.2	Static Thread-Local Variable	130
10.3.3	TLS Linker Optimization	132
10.4	Kernel Support	135
10.5	Coding Examples	135
10.5.1	Indirect Branch	135
11	Alternate Code Sequences For Security	136
11.1	Code Sequences without PLT	136
11.1.1	Indirect Call via the GOT Slot	136
11.1.2	Thread-Local Storage without PLT	137
12	Intel CET Extension	142
12.1	Program Loading	142
12.1.1	Process GNU_PROPERTY_X86_FEATURE_1_IBT	142
12.1.2	Process GNU_PROPERTY_X86_FEATURE_1_SHSTK	143
12.2	Dynamic Linking	143

A	Linux Conventions	145
A.1	Execution of 32-bit Programs	145
A.2	AMD64 Linux Kernel Conventions	145
A.2.1	Calling Conventions	145
A.2.2	Stack Layout	146
A.2.3	Miscellaneous Remarks	146
B	Linker Optimization	147
B.1	Combine GOTPLT and GOT Slots	147
B.2	Optimize GOTPCRELX Relocations	148

List of Tables

3.1	Micro-Architecture Levels	15
3.2	Hardware Exceptions and Signals	31
3.3	Floating-Point Exceptions	31
3.4	x87 Floating-Point Control Word	34
3.5	MXCSR Status Bits	34
3.6	rFLAGS Bits	35
4.1	AMD64 Specific Section Header Flag, <code>sh_flags</code>	65
4.2	Section Header Types	66
4.3	Special sections	66
4.4	Additional Special Sections for the Medium and Large Code Models . . .	67
4.5	Common Information Entry (CIE)	68
4.6	CIE Augmentation Section Content	69
4.7	Frame Descriptor Entry (FDE)	70
4.8	FDE Augmentation Section Content	71
4.9	Relocation Types	74
4.10	Large Model Relocation Types	78
5.1	Program Header Types	80
5.2	Optional Dynamic Array Tags, <code>d_tag</code>	81
5.3	Program Property Type Ranges	87
5.4	Program Property Types	88
5.5	GNU_PROPERTY_X86_FEATURE_1_AND Bit Flags	89
5.6	Bit Flags For X86 Processor Features	90
5.7	Bit Flags For X86 Instruction Sets	91
7.1	Predefined Pre-Processor Symbols	109
9.1	Mil intrinsics	118
9.2	F77 intrinsics	120

9.3	F90 intrinsics	121
9.4	Math intrinsics	121
9.5	Unix intrinsics	123
10.1	General Dynamic Model Code Sequence	129
10.2	General Dynamic Model Code Sequence with TLSDESC	129
10.3	Initial Exec Model Code Sequence	129
10.4	Initial Exec Model Code Sequence, II	130
10.5	Local Dynamic Model Code Sequence With Lea	130
10.6	Local Dynamic Model Code Sequence With Add	130
10.7	General Dynamic Model Code Sequence with TLSDESC	131
10.8	Local Dynamic Model Code Sequence, II	131
10.9	Local Exec Model Code Sequence With Lea	131
10.10	Local Exec Model Code Sequence With Add	131
10.11	Local Exec Model Code Sequence, II	132
10.12	Local Exec Model Code Sequence, III	132
10.13	GD -> IE Code Transition	132
10.14	GDdesc -> IE Code Transition	132
10.15	GD -> LE Code Transition	133
10.16	GDdesc -> LE Code Transition	133
10.17	IE -> LE Code Transition With Lea	133
10.18	IE -> LE Code Transition With Add	133
10.19	IE -> LE Code Transition, II	134
10.20	LD -> LE Code Transition With Lea	134
10.21	LD -> LE Code Transition With Add	134
10.22	LD -> LE Code Transition, II	134
10.23	Indirect Branch	135
11.1	General Dynamic Model Code Sequence (LP64)	138
11.2	General Dynamic Model Code Sequence (ILP32)	138
11.3	Local Dynamic Model Code Sequence (LP64)	139
11.4	Local Dynamic Model Code Sequence (ILP32)	139
11.5	GD -> IE Code Transition (LP64)	139
11.6	GD -> IE Code Transition (ILP32)	140
11.7	GD -> LE Code Transition (LP64)	140
11.8	GD -> LE Code Transition (ILP32)	140
11.9	LD -> LE Code Transition (LP64)	141
11.10	LD -> LE Code Transition (ILP32)	141

B.1	Call and Jmp Conversion	148
B.2	Mov Conversion	149
B.3	Test and Binop Conversion	149

List of Figures

3.1	Scalar Types	17
3.2	Bit-Field Ranges	20
3.3	Stack Frame with Base Pointer	22
3.4	Register Usage	27
3.5	Parameter Passing Example	29
3.6	Register Allocation Example	30
3.7	Virtual Address Configuration	32
3.8	Conventional Segment Arrangements	33
3.9	Initial Process Stack	36
3.10	<code>auxv_t</code> Type Definition	37
3.11	Auxiliary Vector Types	38
3.12	Position-Independent Function Prologue Code	44
3.13	Absolute Load and Store (Small Model)	45
3.14	Position-Independent Load and Store (Small PIC Model)	46
3.15	Absolute Load and Store (Medium Model)	47
3.16	Position-Independent Load and Store (Medium PIC Model)	48
3.17	Position-Independent Load and Store (Medium PIC Model), continued	48
3.18	Absolute Global Data Load and Store	49
3.19	Faster Absolute Global Data Load and Store	49
3.20	Position-Independent Global Data Load and Store	50
3.21	Faster Position-Independent Global Data Load and Store	50
3.22	Position-Independent Direct Function Call (Small and Medium Model)	50
3.23	Position-Independent Indirect Function Call	51
3.24	Absolute Direct and Indirect Function Call	51
3.25	Position-Independent Direct and Indirect Function Call	52
3.26	Absolute Branching Code	53
3.27	Implicit Calculation of Target Address	54
3.28	Position-Independent Branching Code	54
3.29	Absolute Switch Code	55

3.30	Position-Independent Switch Code	56
3.31	Parameter Passing Example with Variable-Argument List	57
3.32	Register Allocation Example for Variable-Argument List	57
3.33	Register Save Area	58
3.34	<code>va_list</code> Type Declaration	58
3.35	Sample Implementation of <code>va_arg(1, int)</code>	60
3.36	DWARF Register Number Mapping	62
3.37	Pointer Encoding Specification Byte	63
4.1	Relocatable Fields	72
5.1	Global Offset Table	82
5.2	Procedure Linkage Table (small and medium models)	83
5.3	Final Large Code Model PLT	86
5.4	AMD64 Program Interpreter	87
6.1	Examples for Unwinding in Assembler	108
9.1	Example mapping of names	114
9.2	Mapping of Fortran to C types	114
11.1	Function Call without PLT (Small and Medium Models)	136
11.2	Function Address without PLT (Small and Medium Models)	137
11.3	<code>__tls_get_addr</code> Call	138
12.1	The IBT-enabled PLT (small and medium models)	144
12.2	The Second IBT-enabled PLT (small and medium models)	144
B.1	Procedure Linkage Table Entry Via GOTPLT Slot	147
B.2	Procedure Linkage Table Entry Via GOT Slot	148

Revision History

Intel Changes to this document made by Intel Corporation are copyright Intel Corporation and licensed under CC BY 4.0: <https://creativecommons.org/licenses/by/4.0/legalcode>.

1.0 Document `_Unwind_GetIPInfo`. Update to match C++ ABI. Add chapter for alternate code sequences For security.

- 0.99** Add description of TLS relocations (thanks to Alexandre Oliva) and mention the decimal floating point and AVX types (thanks to H.J. Lu).
- 0.98** Various clarifications and fixes according to feedback from Sun, thanks to Terrence Miller. DWARF register numbers for some system registers, thanks to Jan Beulich. Add `R_X86_64_SIZE32` and `R_X86_64_SIZE64` relocations; extend meaning of `e_phnum` to handle more than 0xffff program headers, thanks to Rod Evans. Add footnote about passing of `decimal` datatypes. Specify that `_Bool` is booleanized at the caller.
- 0.97** Integrate Fortran ABI.
- 0.96** Use `SHF_X86_64_LARGE` instead `SHF_AMD64_LARGE` (thanks to Evandro Menezes). Correct various grammatical errors noted by Mark F. Haigh, who also noted that there are no global VLAs in C99. Thanks also to Robert R. Henry.
- 0.95** Include description of the medium PIC memory model (thanks to Jan Hubička) and large model (thanks to Evandro Menezes).
- 0.94** Add sections in Development Environment, Program Loading, a description of `EH_FRAME` sections and general cleanups to make text in this ABI self-contained. Thanks to Michael Walker and Terrence Miller.
- 0.93** Add sections about program headers, new section types and special sections for unwinding information. Thanks to Michael Walker.
- 0.92** Fix some typos (thanks to Bryan Ford), add section about stack layout in the Linux kernel. Fix example in figure 3.5 (thanks to Tom Horsley). Add section on unwinding through assembler (written by Michal Ludvig). Remove `mmxext` feature (thanks to Evandro Menezes). Add section on Fortran (by Steven Bosscher) and stack unwinding (by Jan Hubička).
- 0.91** Clarify that x87 is default mode, not MMX (by Hans Peter Anvin).
- 0.90** Change DWARF register numbers again; mention that `__m128` needs alignment; fix typo in figure 3.3; add some comments on kernel expectations; mention TLS extensions; add example for passing of variable-argument lists; change semantics of `%rax` in variable-argument lists; improve formatting; mention that X87 class is not used for passing; make `/lib64` a Linux specific section; rename x86-64 to AMD64; describe passing of complex types. Special thanks to Andi Kleen, Michal Ludvig, Michael Matz, David O'Brien and Eric Young for their comments.

- 0.21** Define `__int128` as class `INTEGER` in register passing. Mention that `%al` is used for variadic argument lists. Fix some textual problems. Thanks to H. Peter Anvin, Bo Thorsen, and Michael Matz.
- 0.20 — 2002-07-11** Change DWARF register number values of `%rbx`, `%rsi`, `%rsi` (thanks to Michal Ludvig). Fix footnotes for fundamental types (thanks to H. Peter Anvin). Specify `size_t` (thanks to Bo Thorsen and Andreas Schwab). Add new section on floating point environment functions.
- 0.19 — 2002-03-27** Set name of Linux dynamic linker, mention `%fs`. Incorporate changes from H. Peter Anvin <hpa@zytor.com> for booleans and define handling of sub-64-bit integer types in registers.

Chapter 1

Introduction

The AMD64¹ architecture² is an extension of the x86 architecture. Any processor implementing the AMD64 architecture specification will also provide compatibility modes for previous descendants of the Intel 8086 architecture, including 32-bit processors such as the Intel 386, Intel Pentium, and AMD K6-2 processor. Operating systems conforming to the AMD64 ABI may provide support for executing programs that are designed to execute in these compatibility modes. The AMD64 ABI does not apply to such programs; this document applies only to programs running in the “long” mode provided by the AMD64 architecture.

Binaries using the AMD64 instruction set may program to either a 32-bit model, in which the C data types `int`, `long` and all pointer types are 32-bit objects (ILP32); or to a 64-bit model, in which the C `int` type is 32-bits but the C `long` type and all pointer types are 64-bit objects (LP64). This specification covers both LP64 and ILP32 programming models.

Except where otherwise noted, the AMD64 architecture ABI follows the conventions described in the Intel386 ABI. Rather than replicate the entire contents of the Intel386 ABI, the AMD64 ABI indicates only those places where changes have been made to the Intel386 ABI.

This specification is mostly written using terms and concepts of the C programming language, and so provides only an ABI for C. However, it is assumed that many programming languages will wish to link with code written in C, so that the ABI specifications documented here apply there too.³

¹AMD64 has been previously called x86-64. The latter name is used in a number of places out of historical reasons instead of AMD64.

²The architecture specification is available on the web at <http://www.x86-64.org/documentation>.

³See e.g. section 9.1 for details on C++ ABI and section 9.2 for Fortran.

Chapter 2

Software Installation

This document does not specify how software must be installed on an AMD64 architecture machine.

Chapter 3

Low Level System Information

3.1 Machine Interface

3.1.1 Processor Architecture

Any program can expect that an AMD64 processor implements the baseline features mentioned in table 3.1. Most feature names correspond to CPUID bits, as described in the processor manual. Exceptions are OSFXSR and SCE, which are controlled by bits in the `%cr4` register and the `IA32_EFER` MSR.

In addition to the AMD64 baseline architecture, several micro-architecture levels implemented by later CPU modules have been defined, starting at level `x86-64-v2`. These levels are intended to support loading of optimized implementations on those systems that are compatible with them (see below). The levels are cumulative in the sense that features from previous levels are implicitly included in later levels.

Levels `x86-64-v3` and `x86-64-v4` are only available if the corresponding features have been fully enabled. This means that the system must pass the full sequence of checks in the processor manual for these features, including verification of the XCR0 feature flags obtained using `xgetbv`.

Recommended Uses of Micro-Architecture Levels

The names for the micro-architecture levels of table 3.1 are expected to be used as directory names (to be searched by the dynamic linker, based on the levels supported by the current CPU), and by compilers, to select groups of CPU features. Distributions may also specify that they require CPU support for a certain level.

For example, to select the second level, `x86-64-v3`, a programmer would build a

Table 3.1: Micro-Architecture Levels

Level Name	CPU Feature	Example instruction
(baseline)	CMOV	cmov
	CX8	cmpxchg8b
	FPU	fld
	FXSR	fxsave
	MMX	emms
	OSFXSR	fxsave
	SCE	syscall
	SSE	cvtss2si
	SSE2	cvtpti2pd
x86-64-v2	CMPXCHG16B	cmpxchg16b
	LAHF-SAHF	lahf
	POPCNT	popcnt
	SSE3	addsubpd
	SSE4_1	blendpd
	SSE4_2	pcmpestri
	SSSE3	phaddb
x86-64-v3	AVX	vzeroall
	AVX2	vpermd
	BMI1	andn
	BMI2	bzhi
	F16C	vcvtph2ps
	FMA	vfmadd132pd
	LZCNT	lzcmt
	MOVBE	movbe
	OSXSAVE	xgetbv
x86-64-v4	AVX512F	kmovw
	AVX512BW	vdbpsadbw
	AVX512CD	vplzcntd
	AVX512DQ	vpmullq
	AVX512VL	n/a

shared object with the `-march=x86-64-v3` GCC flag. The resulting shared object needs to be installed into the directory `/usr/lib64/glibc-hwcap/x86-64-v3` or `/usr/lib/x86_64-linux-gnu/glibc-hwcap/x86-64-v3` (in case of distributions with a multi-arch file system layout). In order to support systems that only implement the K8 baseline, a fallback implementation must be installed into the default locations, `/usr/lib64` or `/usr/lib/x86_64-linux-gnu`. It has to be built with `-march=x86-64` (the upstream GCC default). If this guideline is not followed, loading the library will fail on systems that do not support the level for which the optimized shared object was built.

Shared objects that are installed under the matching `glibc-hwcap` subdirectory can use the CPU features for this level and earlier levels without further detection logic. Run-time detection for other CPU features not listed in this section, or listed only under later levels, is still required (even if all current CPUs implement certain CPU features together).

If a distribution requires support for a certain level, they build everything with the appropriate `-march=` option and install the built binaries in the default file system locations. When targeting such distributions, programmers can build their binaries with the same `-march=` option and install them into the default locations. Optimized shared objects for later levels can still be installed into subdirectories with the appropriate name.

3.1.2 Data Representation

Within this specification, the term *byte* refers to a 8-bit object, the term *twobyte* refers to a 16-bit object, the term *fourbyte* refers to a 32-bit object, the term *eightbyte* refers to a 64-bit object, and the term *sixteenbyte* refers to a 128-bit object.¹

Fundamental Types

Figure 3.1 shows the correspondence between ISO C's scalar types and the processor's. `__int128`, `_Float16`, `__float80`, `__float128`, `__m64`, `__m128`, `__m256` and `__m512` types are optional.

The `__float128` type uses a 15-bit exponent, a 113-bit mantissa (the high order significant bit is implicit) and an exponent bias of 16383.²

¹The Intel386 ABI uses the term *halfword* for a 16-bit object, the term *word* for a 32-bit object, the term *doubleword* for a 64-bit object. But most IA-32 processor specific documentation define a *word* as a 16-bit object, a *doubleword* as a 32-bit object, a *quadword* as a 64-bit object and a *double quadword* as a 128-bit object.

²Initial implementations of the AMD64 architecture are expected to support operations on the `__float128` type only via software emulation.

Figure 3.1: Scalar Types

Type	C	sizeof	Alignment (bytes)	AMD64 Architecture
Integral	<code>_Bool</code> [†]	1	1	boolean
	<code>char</code>	1	1	signed byte
	<code>signed char</code>			
	<code>unsigned char</code>	1	1	unsigned byte
	<code>signed short</code>	2	2	signed twobyte
	<code>unsigned short</code>	2	2	unsigned twobyte
	<code>signed int</code>	4	4	signed fourbyte
	<code>enum</code> ^{†††}			
	<code>unsigned int</code>	4	4	unsigned fourbyte
	<code>signed long (LP64)</code>	8	8	signed eightbyte
	<code>unsigned long (LP64)</code>	8	8	unsigned eightbyte
	<code>signed long (ILP32)</code>	4	4	signed fourbyte
	<code>unsigned long (ILP32)</code>	4	4	unsigned fourbyte
	<code>signed long long</code>	8	8 ^{††††}	signed eightbyte
	<code>unsigned long long</code>	8	8 ^{††††}	unsigned eightbyte
Pointer	<code>__int128</code> ^{††}	16	16	signed sixteenbyte
	<code>signed __int128</code> ^{††}	16	16	signed sixteenbyte
	<code>unsigned __int128</code> ^{††}	16	16	unsigned sixteenbyte
	<code>any-type *</code> (LP64) <code>any-type (*)</code> () (LP64) <code>any-type *</code> (ILP32) <code>any-type (*)</code> () (ILP32)	8 8 4 4	8 8 4 4	unsigned eightbyte unsigned eightbyte unsigned fourbyte unsigned fourbyte
Floating-point	<code>_Float16</code> ^{†††††}	2	2	16-bit (IEEE-754)
	<code>float</code>	4	4	single (IEEE-754)
	<code>double</code>	8	8 ^{††††}	double (IEEE-754)
	<code>__float80</code> ^{††}	16	16	80-bit extended (IEEE-754)
	<code>long double</code> ^{††††}	16	16	80-bit extended (IEEE-754)
	<code>__float128</code> ^{††}	16	16	128-bit extended (IEEE-754)
	<code>long double</code> ^{††††}	16	16	128-bit extended (IEEE-754)
Decimal-floating-point	<code>_Decimal32</code>	4	4	32bit BID (IEEE-754R)
	<code>_Decimal64</code>	8	8	64bit BID (IEEE-754R)
	<code>_Decimal128</code>	16	16	128bit BID (IEEE-754R)
Packed	<code>__m64</code> ^{††}	8	8	MMX and 3DNow!
	<code>__m128</code> ^{††}	16	16	SSE and SSE-2
	<code>__m256</code> ^{††}	32	32	AVX
	<code>__m512</code> ^{††}	64	64	AVX-512

[†] This type is called `bool` in C++.

^{††} These types are optional.

^{†††} C++ and some implementations of C permit enums larger than an int. The underlying type is bumped to an unsigned int, long int or unsigned long int, in that order.

^{††††} The long long, signed long long, unsigned long long and double types have 4-byte alignment in the Intel386 ABI.

^{†††††} The long double type is 128-bit, the same as the `__float128` type, on the Android™ platform. More information on the Android™ platform is available from <http://www.android.com/>.

^{††††††} The `_Float16` type, from ISO/IEC TS 18661-3:2015, is optional.

The `long double` type uses a 15 bit exponent, a 64-bit mantissa with an explicit high order significant bit and an exponent bias of 16383.³ Although a `long double` requires 16 bytes of storage, only the first 10 bytes are significant. The remaining six bytes are tail padding, and the contents of these bytes are undefined.

The `__int128` type is stored in little-endian order in memory, i.e., the 64 low-order bits are stored at a lower address than the 64 high-order bits.

The value of `_Alignof(max_align_t)` is 16.

A null pointer (for all types) has the value zero.

The type `size_t` is defined as `unsigned long` for LP64 and `unsigned int` for ILP32.

Booleans, when stored in a memory object, are stored as single byte objects the value of which is always 0 (`false`) or 1 (`true`). When stored in integer registers (except for passing as arguments), all 8 bytes of the register are significant; any nonzero value is considered `true`.

Like the Intel386 architecture, the AMD64 architecture in general does not require all data accesses to be properly aligned. Misaligned data accesses are slower than aligned accesses but otherwise behave identically. The only exceptions are that `__m128`, `__m256` and `__m512` must always be aligned properly.

Fundamental Types of N-bit Integers

`_BitInt(N)` from ISO/IEC WG14 N2763, where `N` is an integral constant expression representing the exact number of bits to be used to represent the type, is a family of integer types.

- `_BitInt(N)` types are signed by default, and `unsigned _BitInt(N)` types are unsigned.
- `_BitInt(N)` types are stored in little-endian order in memory. Bits in each byte are allocated from right to left.
- For `N ≤ 64`, they have the same size and alignment as the smallest of (signed and unsigned) `char`, `short`, `int`, `long` and `long long` types that can contain them.
- For `N > 64`, they are treated as struct of 64-bit integer chunks. The number of chunks is the smallest number that can contain the type. `_BitInt(N)` types are byte-aligned to 64 bits. The size of these types is the smallest multiple of the 64-bit chunks greater than or equal to `N`.

³This type is the x87 double extended precision data type.

- The value of the unused bits beyond the width of the `_BitInt (N)` value but within the size of the `_BitInt (N)` are unspecified when stored in memory or register.

This permits the use of these types in allocated arrays using the common `sizeof(Array)/sizeof(ElementType)` pattern.

Special Types

The `__bf16`⁴ type is an alternate encoding format for 16-bit float with 8-bit exponent and 7-bit mantissa. `__bf16` represents Brain Floating Point Format⁵ which is a truncated (16-bit) version of the 32-bit IEEE 754 single-precision floating-point format. It has the same size, alignment, parameter passing and return rules as `_Float16`.

Aggregates and Unions

Structures and unions assume the alignment of their most strictly aligned component. Each member is assigned to the lowest available offset with the appropriate alignment. The size of any object is always a multiple of the object's alignment.

An array uses the same alignment as its elements, except that a local or global array variable of length at least 16 bytes or a C99 variable-length array variable always has alignment of at least 16 bytes.⁶

Structure and union objects can require padding to meet size and alignment constraints. The contents of any padding is undefined.

Bit-Fields

C struct and union definitions may include bit-fields that define integral values of a specified size.

The ABI does not permit bit-fields having the type `__m64`, `__m128`, `__m256` or `__m512`. Programs using bit-fields of these types are not portable.

Bit-fields that are neither signed nor unsigned always have non-negative values. Although they may have type `char`, `short`, `int`, or `long` (which can have negative values), these bit-fields have the same range as a bit-field of the same size with the corresponding unsigned type. Bit-fields obey the same size and alignment rules as other structure and union members.

⁴This type is used in `BF16` intrinsics.

⁵This type is used to accelerate machine learning (deep learning training, in particular) algorithms.

⁶The alignment requirement allows the use of SSE instructions when operating on the array. The compiler cannot in general calculate the size of a variable-length array (VLA), but it is expected that most VLAs will require at least 16 bytes, so it is logical to mandate that VLAs have at least a 16-byte alignment.

Figure 3.2: Bit-Field Ranges

Bit-field Type	Width w	Range
signed char	1 to 8	-2^{w-1} to $2^{w-1} - 1$
char		0 to $2^w - 1$
unsigned char		0 to $2^w - 1$
signed short	1 to 16	-2^{w-1} to $2^{w-1} - 1$
short		0 to $2^w - 1$
unsigned short		0 to $2^w - 1$
signed int	1 to 32	-2^{w-1} to $2^{w-1} - 1$
int		0 to $2^w - 1$
unsigned int		0 to $2^w - 1$
signed long (LP64)	1 to 64	-2^{w-1} to $2^{w-1} - 1$
long (LP64)		0 to $2^w - 1$
unsigned long (LP64)		0 to $2^w - 1$
long (ILP32)	1 to 32	0 to $2^w - 1$
unsigned long (ILP32)		0 to $2^w - 1$
signed long long	1 to 64	-2^{w-1} to $2^{w-1} - 1$
long long		0 to $2^w - 1$
unsigned long long		0 to $2^w - 1$

Also:

- bit-fields are allocated from right to left
- bit-fields must be contained in a storage unit appropriate for its declared type
- bit-fields may share a storage unit with other struct / union members

Unnamed bit-fields' types do not affect the alignment of a structure or union.

3.2 Function Calling Sequence

This section describes the standard function calling sequence, including stack frame layout, register usage, parameter passing and so on.

The standard calling sequence requirements apply only to global functions. Local functions that are not reachable from other compilation units may use different conventions. Nevertheless, it is recommended that all functions use the standard calling sequence when possible.

3.2.1 Registers

The AMD64 architecture provides 16 general purpose 64-bit registers. In addition the architecture provides 16 SSE registers, each 128 bits wide and 8 x87 floating point registers, each 80 bits wide. Each of the x87 floating point registers may be referred to in *MMX/3DNow!* mode as a 64-bit register. All of these registers are global to all procedures active for a given thread.

Intel AVX (Advanced Vector Extensions) provides 16 256-bit wide AVX registers (`%ymm0` - `%ymm15`). The lower 128-bits of `%ymm0` - `%ymm15` are aliased to the respective 128b-bit SSE registers (`%xmm0` - `%xmm15`). Intel AVX-512 provides 32 512-bit wide SIMD registers (`%zmm0` - `%zmm31`). The lower 128-bits of `%zmm0` - `%zmm31` are aliased to the respective 128b-bit SSE registers (`%xmm0` - `%xmm31`⁷). The lower 256-bits of `%zmm0` - `%zmm31` are aliased to the respective 256-bit AVX registers (`%ymm0` - `%ymm31`⁸). For purposes of parameter passing and function return, `%xmmN`, `%ymmN` and `%zmmN` refer to the same register. Only one of them can be used at the same time. We use vector register to refer to either SSE, AVX or AVX-512 register. In addition, Intel AVX-512 also provides 8 vector mask registers (`%k0` - `%k7`), each 64-bit wide.

Intel Advanced Matrix Extensions (Intel AMX) is a programming paradigm consisting of two components: a set of 2-dimensional registers (tiles) representing sub-arrays from a larger 2-dimensional memory image, and accelerators able to operate on tiles. Capability of Intel AMX implementation is enumerated by palettes. Two palettes are supported: palette 0 represents the initialized state and palette 1 consists of 8 tile registers (`%tmm0` - `%tmm7`) of up to 1 KB size, which is controlled by a tile control register.

Intel APX (Advanced Performance Extensions) provides 16 general purpose 64-bit registers (`%r16` - `%r31`).

This subsection discusses usage of each register. Registers `%rbp`, `%rbx` and `%r12` through `%r15` “belong” to the calling function and the called function is required to preserve their values. In other words, a called function must preserve these registers’ values for its caller. Remaining registers “belong” to the called function.⁹ If a calling function wants to preserve such a register value across a function call, it must save the value in its local

⁷ `%xmm16` - `%xmm31` are only available with Intel AVX-512.

⁸ `%ymm16` - `%ymm31` are only available with Intel AVX-512.

⁹ Note that in contrast to the Intel386 ABI, `%rdi`, and `%rsi` belong to the called function, not the caller.

Figure 3.3: Stack Frame with Base Pointer

Position	Contents	Frame
$8n+16(\%rbp)$	memory argument eightbyte n	Previous
$16(\%rbp)$	memory argument eightbyte 0	
$8(\%rbp)$	return address	Current
$0(\%rbp)$	previous $\%rbp$ value	
$-8(\%rbp)$	unspecified	
$0(\%rsp)$	variable size	
$-128(\%rsp)$	red zone	

stack frame.

The CPU shall be in x87 mode upon entry to a function. Therefore, every function that uses the *MMX* registers is required to issue an *emms* or *femms* instruction after using *MMX* registers, before returning or calling another function.¹⁰ The direction flag *DF* in the *%rFLAGS* register must be clear (set to “forward” direction) on function entry and return. Other user flags have no specified role in the standard calling sequence and are *not* preserved across calls.

The control bits of the *MXCSR* register are callee-saved (preserved across calls), while the status bits are caller-saved (not preserved). The x87 status word register is caller-saved, whereas the x87 control word is callee-saved.

3.2.2 The Stack Frame

In addition to registers, each function has a frame on the run-time stack. This stack grows downwards from high addresses. Figure 3.3 shows the stack organization.

The end of the input argument area shall be aligned on a 16 (32 or 64, if *__m256* or *__m512* is passed on stack) byte boundary.¹¹ In other words, the stack needs to be 16 (32 or 64) byte aligned immediately before the call instruction is executed. Once control has

¹⁰All x87 registers are caller-saved, so callees that make use of the *MMX* registers may use the faster *femms* instruction.

¹¹The maximum aligned boundary is the maximum alignment of all variables passed on stack. In C11, variable of type `typedef struct { _Alignas (512) int i; } var_t;` is aligned to 512 bytes.

been transferred to the function entry point, i.e. immediately after the return address has been pushed, `%rsp` points to the return address, and the value of $(\text{\code{\%rsp}} + 8)$ is a multiple of 16 (32 or 64).¹²

The 128-byte area beyond the location pointed to by `%rsp` is considered to be reserved and shall not be modified by signal or interrupt handlers.¹³ Therefore, functions may use this area for temporary data that is not needed across function calls. In particular, leaf functions may use this area for their entire stack frame, rather than adjusting the stack pointer in the prologue and epilogue. This area is known as the red zone.

3.2.3 Parameter Passing

After the argument values have been computed, they are placed either in registers or pushed on the stack. The way how values are passed is described in the following sections.

Definitions We first define a number of classes to classify arguments. The classes are corresponding to AMD64 register classes and defined as:

INTEGER This class consists of integral types that fit into one of the general purpose registers.

SSE The class consists of types that fit into a vector register.

SSEUP The class consists of types that fit into a vector register and can be passed and returned in the upper bytes of it.

X87, X87UP These classes consists of types that will be returned via the x87 FPU.

COMPLEX_X87 This class consists of types that will be returned via the x87 FPU.

NO_CLASS This class is used as initializer in the algorithms. It will be used for padding and empty structures and unions.

MEMORY This class consists of types that will be passed and returned in memory via the stack.

¹²The conventional use of `%rbp` as a frame pointer for the stack frame may be avoided by using `%rsp` (the stack pointer) to index into the stack frame. This technique saves two instructions in the prologue and epilogue and makes one additional general-purpose register (`%rbp`) available.

¹³Locations within 128 bytes can be addressed using one-byte displacements.

Classification The size of each argument gets rounded up to eightbytes.¹⁴

The basic types are assigned their natural classes:

- Arguments of types (signed and unsigned) `_Bool`, `char`, `short`, `int`, `long`, `long long`, and pointers are in the `INTEGER` class.
- Arguments of types `_Float16`, `float`, `double`, `_Decimal32`, `_Decimal64` and `__m64` are in class `SSE`.
- Arguments of types `__float128`, `_Decimal128` and `__m128` are split into two halves. The least significant ones belong to class `SSE`, the most significant one to class `SSEUP`.
- Arguments of type `__m256` are split into four eightbyte chunks. The least significant one belongs to class `SSE` and all the others to class `SSEUP`.
- Arguments of type `__m512` are split into eight eightbyte chunks. The least significant one belongs to class `SSE` and all the others to class `SSEUP`.
- The 64-bit mantissa of arguments of type `long double` belongs to class `X87`, the 16-bit exponent plus 6 bytes of padding belongs to class `X87UP`.
- Arguments of type `__int128` offer the same operations as `INTEGER`s, yet they do not fit into one general purpose register but require two registers. For classification purposes `__int128` is treated as if it were implemented as:

```
typedef struct {  
    long low, high;  
} __int128;
```

with the exception that arguments of type `__int128` that are stored in memory must be aligned on a 16-byte boundary.

- Arguments of type `_BitInt(N)` with $N \leq 64$ are in the `INTEGER` class.
- Arguments of type `_BitInt(N)` with $N > 64$ are classified as if they were implemented as struct of 64-bit integer fields.
- Arguments of `complex T` where `T` is one of the types `_Float16`, `float`, `double` or `__float128` are treated as if they are implemented as:

¹⁴Therefore the stack will always be eightbyte aligned.

```

struct complexT {
    T real;
    T imag;
};

```

- A variable of type `complex long double` is classified as type `COMPLEX_X87`.

The classification of aggregate (structures and arrays) and union types works as follows:

1. If the size of an object is larger than eight eightbytes, or it contains unaligned fields, it has class `MEMORY` ¹⁵.
2. If a C++ object is non-trivial for the purpose of calls, as specified in the C++ ABI ¹⁶, it is passed by invisible reference (the object is replaced in the parameter list by a pointer that has class `INTEGER`) ¹⁷.
3. If the size of the aggregate exceeds a single eightbyte, each is classified separately. Each eightbyte gets initialized to class `NO_CLASS`.
4. Each field of an object is classified recursively so that always two fields ¹⁸ are considered. The resulting class is calculated according to the classes of the fields in the eightbyte:
 - (a) If both classes are equal, this is the resulting class.
 - (b) If one of the classes is `NO_CLASS`, the resulting class is the other class.
 - (c) If one of the classes is `MEMORY`, the result is the `MEMORY` class.
 - (d) If one of the classes is `INTEGER`, the result is the `INTEGER`.

¹⁵The post merger clean up described later ensures that, for the processors that do not support the `__m256` type, if the size of an object is larger than two eightbytes and the first eightbyte is not SSE or any other eightbyte is not SSEUP, it still has class `MEMORY`. This in turn ensures that for processors that do support the `__m256` type, if the size of an object is four eightbytes and the first eightbyte is SSE and all other eightbytes are SSEUP, it can be passed in a register. This also applies to the `__m512` type. That is for processors that support the `__m512` type, if the size of an object is eight eightbytes and the first eightbyte is SSE and all other eightbytes are SSEUP, it can be passed in a register, otherwise, it will be passed in memory.

¹⁶See section 9.1 for details on C++ ABI.

¹⁷An object whose type is non-trivial for the purpose of calls cannot be passed by value because such objects must have the same address in the caller and the callee. Similar issues apply when returning an object from a function. See C++17 [class.temporary] paragraph 3.

¹⁸Zero-width bit-fields are `NO_CLASS`.

- (e) If one of the classes is X87, X87UP, COMPLEX_X87 class, MEMORY is used as class.
 - (f) Otherwise class SSE is used.
5. Then a post merger cleanup is done:
- (a) If one of the classes is MEMORY, the whole argument is passed in memory.
 - (b) If X87UP is not preceded by X87, the whole argument is passed in memory.
 - (c) If the size of the aggregate exceeds two eightbytes and the first eightbyte isn't SSE or any other eightbyte isn't SSEUP, the whole argument is passed in memory.
 - (d) If SSEUP is not preceded by SSE or SSEUP, it is converted to SSE.

Passing Once arguments are classified, the registers get assigned (in left-to-right order) for passing as follows:

1. If the class is MEMORY, pass the argument on the stack at an address respecting the arguments alignment (which might be more than its natural alignment).
2. If the class is INTEGER, the next available register of the sequence `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8` and `%r9` is used¹⁹.
3. If the class is SSE, the next available vector register is used, the registers are taken in the order from `%xmm0` to `%xmm7`.
4. If the class is SSEUP, the eightbyte is passed in the next available eightbyte chunk of the last used vector register.
5. If the class is X87, X87UP or COMPLEX_X87, it is passed in memory.

When a value of a type of class INTEGER is returned or passed in a register or on the stack, the excess bits that would not be present in the memory representation of the type (see figure 3.1) are unspecified.²⁰

¹⁹Note that `%r11` is neither required to be preserved, nor is it used to pass arguments. Making this register available as scratch register means that code in the PLT need not spill any registers when computing the address to which control needs to be transferred. `%al` is used to indicate the number of vector arguments passed to a function requiring a variable number of arguments. `%r10` is used for passing a function's static chain pointer.

²⁰That is, the consumer side of those values needs to extend them or use short form instruction variants. As in the memory representation, for a value of type `_Bool`, the lowest 8 bits are significant, together forming the value 0 or 1.

Figure 3.4: Register Usage

Register	Usage	callee saved
%rax	temporary register; with variable arguments passes information about the number of vector registers used; 1 st return register	No
%rbx	callee-saved register	Yes
%rcx	used to pass 4 th integer argument to functions	No
%rdx	used to pass 3 rd argument to functions; 2 nd return register	No
%rsp	stack pointer	Yes
%rbp	callee-saved register; optionally used as frame pointer	Yes
%rsi	used to pass 2 nd argument to functions	No
%rdi	used to pass 1 st argument to functions	No
%r8	used to pass 5 th argument to functions	No
%r9	used to pass 6 th argument to functions	No
%r10	temporary register, used for passing a function's static chain pointer	No
%r11	temporary register	No
%r12-%r14	callee-saved registers	Yes
%r15	callee-saved register; optionally used as GOT base pointer	Yes
%r16-%r31	temporary registers	No
%xmm0-%xmm1	used to pass and return floating point arguments	No
%xmm2-%xmm7	used to pass floating point arguments	No
%xmm8-%xmm15	temporary registers	No
%xmm16-%xmm31	temporary registers	No
%tmm0-%tmm7	temporary registers	No
%mm0-%mm7	temporary registers	No
%k0-%k7	temporary registers	No
%st0,%st1	temporary registers, used to return long double arguments	No
%st2-%st7	temporary registers	No
%fs	thread pointer	Yes
mxcsr	SSE2 control and status word	partial
x87 SW	x87 status word	No
x87 CW	x87 control word	Yes
tilecfg	Tile control register	No

If there are no registers available for any eightbyte of an argument, the whole argument is passed on the stack. If registers have already been assigned for some eightbytes of such an argument, the assignments get reverted.

Once registers are assigned, the arguments passed in memory are pushed on the stack in reversed (right-to-left²¹) order.

For calls that may call functions that use varargs or stdargs (prototype-less calls or calls to functions containing ellipsis (...) in the declaration) `%a1`²² is used as hidden argument to specify the number of vector registers used. The contents of `%a1` do not need to match exactly the number of registers, but must be an upper bound on the number of vector registers used and is in the range 0–8 inclusive.

When passing `__m256` or `__m512` arguments to functions that use varargs or stdarg, function prototypes must be provided. Otherwise, the run-time behavior is undefined.

Returning of Values The returning of values is done according to the following algorithm:

1. Classify the return type with the classification algorithm.
2. If the type has class MEMORY, then the caller provides space for the return value and passes the address of this storage in `%rdi` as if it were the first argument to the function. In effect, this address becomes a “hidden” first argument. This storage must not overlap any data visible to the callee through other names than this argument.

On return `%rax` will contain the address that has been passed in by the caller in `%rdi`.

3. If the class is INTEGER, the next available register of the sequence `%rax`, `%rdx` is used.
4. If the class is SSE, the next available vector register of the sequence `%xmm0`, `%xmm1` is used.
5. If the class is SSEUP, the eightbyte is returned in the next available eightbyte chunk of the last used vector register.

²¹Right-to-left order on the stack makes the handling of functions that take a variable number of arguments simpler. The location of the first argument can always be computed statically, based on the type of that argument. It would be difficult to compute the address of the first argument if the arguments were pushed in left-to-right order.

²²Note that the rest of `%rax` is undefined, only the contents of `%a1` is defined.

6. If the class is X87, the value is returned on the X87 stack in `%st0` as 80-bit x87 number.
7. If the class is X87UP, the value is returned together with the previous X87 value in `%st0`.
8. If the class is COMPLEX_X87, the real part of the value is returned in `%st0` and the imaginary part in `%st1`.

As an example of the register passing conventions, consider the declarations and the function call shown in Figure 3.5. The corresponding register allocation is given in Figure 3.6, the stack frame offset given shows the frame before calling the function.

Figure 3.5: Parameter Passing Example

```
typedef struct {
    int a, b;
    double d;
} structparm;
structparm s;
int e, f, g, h, i, j, k;
long double ld;
double m, n;
__m256 y;
__m512 z;

extern void func (int e, int f,
                  structparm s, int g, int h,
                  long double ld, double m,
                  __m256 y,
                  __m512 z,
                  double n, int i, int j, int k);

func (e, f, s, g, h, ld, m, y, z, n, i, j, k);
```

Figure 3.6: Register Allocation Example

General Purpose Registers	Floating Point Registers	Stack Frame Offset
%rdi: e	%xmm0: s.d	0: ld
%rsi: f	%xmm1: m	16: j
%rdx: s.a, s.b	%ymm2: y	24: k
%rcx: g	%zmm3: z	
%r8: h	%xmm4: n	
%r9: i		

3.3 Operating System Interface

3.3.1 Exception Interface

As the AMD64 manuals describe, the processor changes mode to handle *exceptions*, which may be synchronous, floating-point/coprocessor or asynchronous. Synchronous and floating-point/coprocessor exceptions, being caused by instruction execution, can be explicitly generated by a process. This section, therefore, specifies those exception types with defined behavior. The AMD64 architecture classifies exceptions as *faults*, *traps*, and *aborts*. See the Intel386 ABI for more information about their differences.

Hardware Exception Types

The operating system defines the correspondence between hardware exceptions and the signals specified by `signal` (BA_OS) as shown in table 3.2. Contrary to the i386 architecture, the AMD64 does not define any instructions that generate a bounds check fault in long mode.

3.3.2 Virtual Address Space

Although the AMD64 architecture uses 64-bit pointers, implementations are only required to handle 48-bit addresses. Therefore, conforming processes may only use addresses from `0x00000000 00000000` to `0x00007fff ffffffff`²³.

²³`0x0000ffff ffffffff` is not a canonical address and cannot be used.

Table 3.2: Hardware Exceptions and Signals

Number	Exception name	Signal
0	divide error fault	SIGFPE
1	single step trap/fault	SIGTRAP
2	non-maskable interrupt	none
3	breakpoint trap	SIGTRAP
4	overflow trap	SIGSEGV
5	(reserved)	
6	invalid opcode fault	SIGILL
7	no coprocessor fault	SIGFPE
8	double fault abort	none
9	coprocessor overrun abort	SIGSEGV
10	invalid TSS fault	none
11	segment not present fault	none
12	stack exception fault	SIGSEGV
13	general protection fault/abort	SIGSEGV
14	page fault	SIGSEGV
15	(reserved)	
16	coprocessor error fault	SIGFPE
other	(unspecified)	SIGILL

Table 3.3: Floating-Point Exceptions

Code	Reason
FPE_FLTDIV	floating-point divide by zero
FPE_FLTOVF	floating-point overflow
FPE_FLTUND	floating-point underflow
FPE_FLTRES	floating-point inexact result
FPE_FLTINV	invalid floating-point operation

Processes begin with three logical segments, commonly called text, data, and stack. Use of shared libraries add other segments and a process may dynamically create segments.

3.3.3 Page Size

Systems are permitted to use any power-of-two page size between 4KB and 64KB, inclusive.

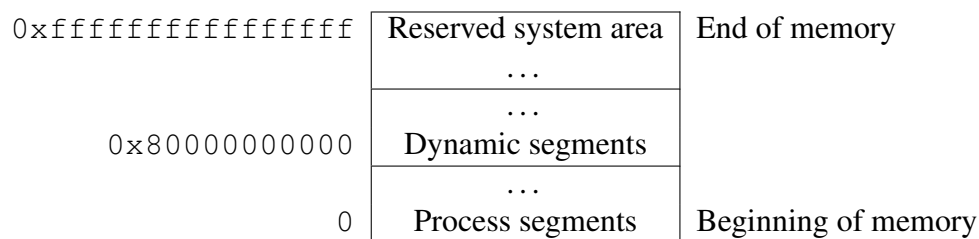
3.3.4 Virtual Address Assignments

Conceptually processes have the full address space available. In practice, however, several factors limit the size of a process.

- The system reserves a configuration dependent amount of virtual space.
- The system reserves a configuration dependent amount of space per process.
- A process whose size exceeds the system's available combined physical memory and secondary storage cannot run. Although some physical memory must be present to run any process, the system can execute processes that are bigger than physical memory, paging them to and from secondary storage. Nonetheless, both physical memory and secondary storage are shared resources. System load, which can vary from one program execution to the next, affects the available amount.

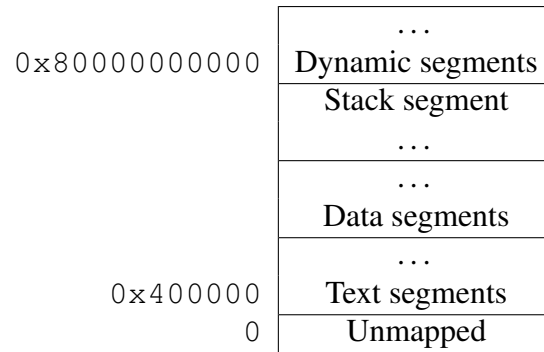
Programs that dereference null pointers are erroneous and a process should not expect 0x0 to be a valid address.

Figure 3.7: Virtual Address Configuration



Although applications may control their memory assignments, the typical arrangement appears in figure 3.8.

Figure 3.8: Conventional Segment Arrangements



3.4 Process Initialization

3.4.1 Initial Stack and Register State

Special Registers

The AMD64 architecture defines floating point instructions. At process startup the two floating point units, SSE2 and x87, both have all floating-point exception status flags cleared. The status of the control words is as defined in tables 3.4 and 3.5.

Table 3.4: x87 Floating-Point Control Word

Field	Value	Note
RC	0	Round to nearest
PC	11	Double extended precision
PM	1	Precision masked
UM	1	Underflow masked
OM	1	Overflow masked
ZM	1	Zero divide masked
DM	1	De-normal operand masked
IM	1	Invalid operation masked

Table 3.5: MXCSR Status Bits

Field	Value	Note
FZ	0	Do not flush to zero
RC	0	Round to nearest
PM	1	Precision masked
UM	1	Underflow masked
OM	1	Overflow masked
ZM	1	Zero divide masked
DM	1	De-normal operand masked
IM	1	Invalid operation masked
DAZ	0	De-normals are not zero

The `rFLAGS` register contains the system flags, such as the direction flag and the carry flag. The low 16 bits (FLAGS portion) of `rFLAGS` are accessible by application software. The state of them at process initialization is shown in table 3.6.

Table 3.6: `rFLAGS` Bits

Field	Value	Note
DF	0	Direction forward
CF	0	No carry
PF	0	Even parity
AF	0	No auxiliary carry
ZF	0	No zero result
SF	0	Unsigned result
OF	0	No overflow occurred

Stack State

This section describes the machine state that `exec` (`BA_OS`) creates for new processes. Various language implementations transform this initial program state to the state required by the language standard.

For example, a C program begins executing at a function named `main` declared as:

```
extern int main ( int argc , char *argv[ ] , char* envp[ ] );
```

where

argc is a non-negative argument count

argv is an array of argument strings, with `argv[argc] == 0`

envp is an array of environment strings, terminated by a null pointer.

When `main()` returns its value is passed to `exit()` and if that has been over-ridden and returns, `_exit()` (which must be immune to user interposition).

The initial state of the process stack, i.e. when `_start` is called is shown in figure 3.9.

Figure 3.9: Initial Process Stack

Purpose	Start Address	Length
Unspecified	High Addresses	
Information block, including argument strings, environment strings, auxiliary information ...		varies
Unspecified		
Null auxiliary vector entry		1 eightbyte
Auxiliary vector entries ...		2 eightbytes each
0		eightbyte
Environment pointers ...		1 eightbyte each
0	$8+8*\text{argc}+\text{\%rsp}$	eightbyte
Argument pointers	$8+\text{\%rsp}$	argc eightbytes
Argument count	$\text{\%rsp}$	eightbyte
Undefined	Low Addresses	

Argument strings, environment strings, and the auxiliary information appear in no specific order within the information block and they need not be compactly allocated.

Only the registers listed below have specified values at process entry:

- $\text{\%rbp}$** The content of this register is unspecified at process initialization time, but the user code should mark the deepest stack frame by setting the frame pointer to **zero**.
- $\text{\%rsp}$** The stack pointer holds the address of the byte with lowest address which is part of the stack. It is guaranteed to be **16-byte aligned at process entry**.
- $\text{\%rdx}$** a function pointer that the application should register with `atexit (BA_OS)`.

It is unspecified whether the data and stack segments are initially mapped with execute permissions or not. Applications which need to execute code on the stack or data segments should take proper precautions, e.g., by calling `mprotect()`.

3.4.2 Thread State

New threads inherit the floating-point state of the parent thread and the state is private to the thread thereafter.

3.4.3 Auxiliary Vector

The auxiliary vector is an array of the following structures (ref. [figure 3.10](#)), interpreted according to the `a_type` member.

Figure 3.10: `auxv_t` Type Definition

```
typedef struct
{
    int a_type;
    union {
        long a_val;
        void *a_ptr;
        void (*a_fnc)();
    } a_un;
} auxv_t;
```

The AMD64 ABI uses the auxiliary vector types defined in [figure 3.11](#).

Figure 3.11: Auxiliary Vector Types

Name	Value	a_un
AT_NULL	0	ignored
AT_IGNORE	1	ignored
AT_EXECFD	2	a_val
AT_PHDR	3	a_ptr
AT_PHEMT	4	a_val
AT_PHNUM	5	a_val
AT_PAGESZ	6	a_val
AT_BASE	7	a_ptr
AT_FLAGS	8	a_val
AT_ENTRY	9	a_ptr
AT_NOTELF	10	a_val
AT_UID	11	a_val
AT_EUID	12	a_val
AT_GID	13	a_val
AT_EGID	14	a_val
AT_PLATFORM	15	a_ptr
AT_HWCAP	16	a_val
AT_CLKTCK	17	a_val
AT_SECURE	23	a_val
AT_BASE_PLATFORM	24	a_ptr
AT_RANDOM	25	a_ptr
AT_HWCAP2	26	a_val
AT_EXECPN	31	a_ptr

AT_NULL The auxiliary vector has no fixed length; instead its last entry's `a_type` member has this value.

AT_IGNORE This type indicates the entry has no meaning. The corresponding value of `a_un` is undefined.

AT_EXECFD At process creation the system may pass control to an interpreter program. When this happens, the system places either an entry of type `AT_EXECFD` or one of type `AT_PHDR` in the auxiliary vector. The entry for type `AT_EXECFD` uses the `a_val`

member to contain a file descriptor open to read the application program's object file.

AT_PHDR The system may create the memory image of the application program before passing control to the interpreter program. When this happens, the `a_ptr` member of the `AT_PHDR` entry tells the interpreter where to find the program header table in the memory image.

AT_PHERENT The `a_val` member of this entry holds the size, in bytes, of one entry in the program header table to which the `AT_PHDR` entry points.

AT_PHNUM The `a_val` member of this entry holds the number of entries in the program header table to which the `AT_PHDR` entry points.

AT_PAGESZ If present, this entry's `a_val` member gives the system page size, in bytes.

AT_BASE The `a_ptr` member of this entry holds the base address at which the interpreter program was loaded into memory. See “Program Header” in the System V ABI for more information about the base address.

AT_FLAGS If present, the `a_val` member of this entry holds one-bit flags. Bits with undefined semantics are set to zero.

AT_ENTRY The `a_ptr` member of this entry holds the entry point of the application program to which the interpreter program should transfer control.

AT_NOTELF The `a_val` member of this entry is non-zero if the program is in another format than ELF.

AT_UID The `a_val` member of this entry holds the real user id of the process.

AT_EUID The `a_val` member of this entry holds the effective user id of the process.

AT_GID The `a_val` member of this entry holds the real group id of the process.

AT_EGID The `a_val` member of this entry holds the effective group id of the process.

AT_PLATFORM The `a_ptr` member of this entry points to a string containing the platform name.

AT_HWCAP The `a_val` member of this entry contains an bitmask of CPU features. It mask to the value returned by CPUID 1.EDX.

AT_CLKTCK The `a_val` member of this entry contains the frequency at which times() increments.

AT_SECURE The `a_val` member of this entry contains one if the program is in secure mode (for example started with `suid`). Otherwise zero.

AT_BASE_PLATFORM The `a_ptr` member of this entry points to a string identifying the base architecture platform (which may be different from the platform).

AT_RANDOM The `a_ptr` member of this entry points to 16 securely generated random bytes.

AT_HWCAP2 The `a_val` member of this entry contains the extended hardware feature mask. Currently it is 0, but may contain additional feature bits in the future.

AT_EXECFN The `a_ptr` member of this entry is a pointer to the file name of the executed program.

3.5 Coding Examples

This section discusses example code sequences for fundamental operations such as calling functions, accessing static objects, and transferring control from one part of a program to another. Unlike previous material, this material is not normative.

3.5.1 Architectural Constraints

The AMD64 architecture usually does not allow an instruction to encode arbitrary 64-bit constants as immediate operand. Most instructions accept 32-bit immediates that are sign extended to the 64-bit ones. Additionally the 32-bit operations with register destinations implicitly perform zero extension making loads of 64-bit immediates with upper half set to 0 even cheaper.

Additionally the branch instructions accept **32-bit immediate** operands that are sign extended and used to adjust the instruction pointer. Similarly an instruction pointer relative addressing mode exists for data accesses with equivalent limitations.

In order to **improve performance and reduce code size**, it is desirable to use different code models depending on the requirements.

Code models define constraints for symbolic values that allow the compiler to generate better code. Basically **code models differ in addressing (absolute versus position independent), code size, data size and address range**. We define only a small number of code models that are of general interest:

eventually, 32-bit immediate operand is calculated from the symbol's address and the offset so, symbol's address + offset must fit in 32-bit immediate!

sign extended immediate: $[0, 2^{31} - 2^{24} - 1] + [-2^{31}, 2^{24}] \sim [-2^{31}, 2^{31}-1]$

zero extended immediate: $[0, 2^{31} - 2^{24} - 1] + [0, 2^{31} - 2^{24}] \sim [0, 2^{32} - \dots]$

Small code model The virtual address of code executed is known at link time. Additionally all symbols are known to be located in the virtual addresses in the range from 0 to $2^{31} - 2^{24} - 1$ or from $0x00000000$ to $0x7effffff^{24}$.

This allows the compiler to encode symbolic references with offsets in the range from $-(2^{31})$ to 2^{24} or from $0x80000000$ to $0x01000000$ directly in the sign extended immediate operands, with offsets in the range from 0 to $2^{31} - 2^{24}$ or from $0x00000000$ to $0x7f000000$ in the zero extended immediate operands and use instruction pointer relative addressing for the symbols with offsets in the range $-(2^{24})$ to 2^{24} or $0xf0000000$ to $0x01000000$.

This is the fastest code model and we expect it to be suitable for the vast majority of programs.

Kernel code model The kernel of an operating system is usually rather small but runs in the negative half of the address space. So we define all symbols to be in the range from $2^{64} - 2^{31}$ to $2^{64} - 2^{24}$ or from $0xffffffff80000000$ to $0xfffffffff0000000$.

This code model has advantages similar to those of the small model, but allows encoding of zero extended symbolic references only for offsets from 2^{31} to $2^{31} + 2^{24}$ or from $0x80000000$ to $0x81000000$. The range offsets for sign extended reference changes to 0 to $2^{31} + 2^{24}$ or $0x00000000$ to $0x81000000$.

Medium code model The data sections are split into two parts — the regular data sections still limited in the same way as in the small code model and the large data sections having no limits except for available addressing space. The program layout must be set in a way so that large data sections (`.ldata`, `.lrodata`, `.lbss`) are not between regular text and data sections.

This model requires the compiler to use `movabs` instructions to access large static data and to load addresses into registers, but keeps the advantages of the small code model for manipulation of addresses in the small data and text sections (specially needed for branches).

By default only data larger than 65535 bytes will be placed in the large data section.

Large code model The large code model makes no assumptions about addresses and sizes of sections.

²⁴ The number 24 is chosen arbitrarily. It allows for all memory of objects of size up to 2^{24} or 16M bytes to be addressed directly because the base address of such objects is constrained to be less than $2^{31} - 2^{24}$ or $0x7f000000$. Without such constraint only the base address would be accessible directly, but not any offsetted variant of it.

Although not strictly necessary, the data sections can be split into normal and large parts like in the medium model, to improve interoperability.

The compiler is required to use the `movabs` instruction, as in the medium code model, even for dealing with addresses inside the text section. Additionally, indirect branches are needed when branching to addresses whose offset from the current instruction pointer is unknown.

It is possible to avoid the limitation on the text section in the small and medium models by breaking up the program into multiple shared libraries, so this model is strictly only required if the text of a single function becomes larger than what the medium model allows.

Small position independent code model (PIC) Unlike the previous models, the virtual addresses of instructions and data are not known until dynamic link time. So all addresses have to be relative to the instruction pointer.

Additionally the maximum distance between a symbol and the end of an instruction is limited to $2^{31} - 2^{24} - 1$ or `0x7effffff`, allowing the compiler to use instruction pointer relative branches and addressing modes supported by the hardware for every symbol with an offset in the range $-(2^{24})$ to 2^{24} or `0xffffffff` to `0x01000000`.

Medium position independent code model (PIC) This model is like the previous model, but similarly to the medium static model adds large data sections at the end of object files.

In the medium PIC model, the instruction pointer relative addressing can not be used directly for accessing large static data, since the offset can exceed the limitations on the size of the displacement field in the instruction. Instead an unwind sequence consisting of `movabs`, `lea` and `add` needs to be used.

Large position independent code model (PIC) This model is like the previous model, but makes no assumptions about the distance of symbols.

The large PIC model implies the same limitation as the medium PIC model regarding addressing of static data. Additionally, references to the global offset table and to the procedure linkage table and branch destinations need to be calculated in a similar way. Further the size of the text segment is allowed to be up to 16EB in size, hence similar restrictions apply to all address references into the text segments, including branches.

Only small code model and small position independent code model (PIC) are used in ILP32 binaries.

3.5.2 Conventions

In this document some special assembler symbols are used in the coding examples and discussion. They are:

- `name@GOT`: specifies the offset to the **GOT entry** for the symbol `name` from the base of the GOT.
- `name@GOTOFF`: specifies the offset to the **location** of the symbol `name` from the base of the GOT.
- `name@GOTPCREL`: specifies the offset to the GOT entry for the symbol `name` from the current code location.
- `name@PLT`: specifies the offset to the PLT entry of symbol `name` from the current code location.
- `name@PLTOFF`: specifies the offset to the PLT entry of symbol `name` from the base of the GOT.
- `_GLOBAL_OFFSET_TABLE_`: specifies the offset to the base of the GOT from the current code location.

3.5.3 Position-Independent Function Prologue

In the **small** code model **all addresses** (including GOT entries) are **accessible via the IP-relative addressing** provided by the AMD64 architecture. Hence there is **no need for an explicit GOT pointer** and therefore no function prologue for setting it up is necessary.

In the **medium and large** code models **a register has to be allocated to hold the address of the GOT** in position-independent objects, because the AMD64 ISA does not support an immediate displacement larger than 32 bits.

As `%r15` is **preserved across function calls**, it is **initialized** in the function prologue to **hold the GOT address**²⁵ for non-leaf functions which call other functions through the PLT. Other functions are free to use any other register. Throughout this document, `%r15` will be used in examples.

²⁵If, at code generation-time, it is determined that either no other functions are called (leaf functions), the called functions addresses can be resolved and are within 2GB, or no global data objects are referred to, it is not necessary to store the GOT address in `%r15` and the prologue code that initializes it may be omitted.

Figure 3.12: Position-Independent Function Prologue Code
medium model:

```
leaq    _GLOBAL_OFFSET_TABLE_(%rip),%r15 # GOTPC32 reloc
```

large model:

```
pushq    %r15                # save %r15
leaq     lf(%rip),%r11        # absolute %rip
1: movabs $_GLOBAL_OFFSET_TABLE_,%r15 # offset to the GOT (R_X86_64_GOTPC64)
leaq     (%r11,%r15),%r15     # absolute address of the GOT
```

For the medium model the GOT pointer is directly loaded, for the large model the absolute value of `%rip` is added to the relative offset to the base of the GOT in order to obtain its absolute address (see figure 3.12).

3.5.4 Data Objects

This section describes only objects with static storage. Stack-resident objects are excluded since programs always compute their virtual address relative to the stack or frame pointers.

Because only the `movabs` instruction uses 64-bit addresses directly, depending on the code model either `%rip`-relative addressing or building addresses in registers and accessing the memory through the register has to be used.

For absolute addresses `%rip`-relative encoding can be used in the small model. In the medium model the `movabs` instruction has to be used for accessing addresses.

Position-independent code cannot contain absolute address. To access a global symbol the address of the symbol has to be loaded from the Global Offset Table. The address of the entry in the GOT can be obtained with a `%rip`-relative instruction in the small model.

Small models

Figure 3.13: Absolute Load and Store (Small Model)

<pre>extern int src[65536]; extern int dst[65536]; extern int *ptr; static int lsrc[65536]; static int ldst[65536]; static int *lptr; dst[0] = src[0]; ptr = dst[0]; *ptr = src[0]; ldst[0] = lsrc[0]; lptr = ldst; *lptr = lsrc[0];</pre>	<pre>.extern src .extern dst .extern ptr .local lsrc .comm lsrc,262144,4 .local ldst .comm ldst,262144,4 .local lptr .comm lptr,8,8 .text movl src(%rip), %eax movl %eax, dst(%rip) movq \$dst, ptr(%rip) movq ptr(%rip),%rax movl src(%rip),%edx movl %edx, (%rax) movl lsrc(%rip), %eax movl %eax, ldst(%rip) movq \$dst, lptr(%rip) movq lptr(%rip),%rax movl lsrc(%rip),%edx movl %edx, (%rax)</pre>
--	---

Figure 3.14: Position-Independent Load and Store (Small PIC Model)

<pre>extern int src[65536]; extern int dst[65536]; extern int *ptr; static int lsrc[65536]; static int ldst[65536]; static int *lptr; dst[0] = src[0]; ptr = dst; *ptr = src[0]; ldst[0] = lsrc[0]; lptr = ldst; *lptr = lsrc[0];</pre>	<pre>.extern src .extern dst .extern ptr .local lsrc .comm lsrc,262144,4 .local ldst .comm ldst,262144,4 .local lptr .comm lptr,8,8 .text movq src@GOTPCREL(%rip), %rax movl (%rax), %edx movq dst@GOTPCREL(%rip), %rax movl %edx, (%rax) movq ptr@GOTPCREL(%rip), %rax movq dst@GOTPCREL(%rip), %rdx movq %rdx, (%rax) movq ptr@GOTPCREL(%rip),%rax movq (%rax), %rdx movq src@GOTPCREL(%rip), %rax movl (%rax), %eax movl %eax, (%rdx) movl lsrc(%rip), %eax movl %eax, ldst(%rip) lea ldst(%rip),%rdx movq %rdx, lptr(%rip) movq lptr(%rip),%rax movl lsrc(%rip),%edx movl %edx, (%rax)</pre>
---	--

Medium models

Figure 3.15: Absolute Load and Store (Medium Model)

<pre>extern int src[65536]; extern int dst[65536]; extern int *ptr; static int lsrc[65536]; static int ldst[65536]; static int *lptr; dst[0] = src[0]; ptr = dst; *ptr = src[0]; ldst[0] = lsrc[0]; lptr = ldst; *lptr = lsrc[0];</pre>	<pre>.extern src .extern dst .extern ptr .local lsrc .comm lsrc,262144,4²⁶ .local ldst .comm ldst,262144,4 .local lptr .comm lptr,8,8 .text movabsl src, %eax movabsl %eax, dst movabsq \$dst,%rdx movq %rdx, ptr movq ptr(%rip),%rdx movabsl src,%eax movl %eax, (%rdx) movabsl lsrc, %eax movabsl %eax, ldst movabsq \$ldst,%rdx movabsq %rdx, lptr movq lptr(%rip),%rdx movabsl lsrc,%eax movl %eax, (%rdx)</pre>
---	---

Figure 3.16: Position-Independent Load and Store (Medium PIC Model)

extern int src[65536]; extern int dst[65536]; extern int *ptr; static int lsrc[65536]; static int ldst[65536]; static int *lptr; dst[0] = src[0]; ptr = dst; *ptr = src[0];	.extern src .extern dst .extern ptr .local lsrc .comm lsrc,262144,4 .local ldst .comm ldst,262144,4 .local lptr .comm lptr,8,8 .text movq src@GOTPCREL(%rip), %rax movl (%rax), %edx movq dst@GOTPCREL(%rip), %rax movl %edx, (%rax) movq ptr@GOTPCREL(%rip), %rax movq dst@GOTPCREL(%rip), %rdx movq %rdx, (%rax) movq ptr@GOTPCREL(%rip), %rax movq (%rax), %rdx movq src@GOTPCREL(%rip), %rax movl (%rax), %eax movl %eax, (%rdx)
---	---

Figure 3.17: Position-Independent Load and Store (Medium PIC Model), continued

ldst[0] = lsrc[0]; lptr = ldst; *lptr = lsrc[0];	movabsq lsrc@GOTOFF, %rax movl (%rax,%r15), %eax movabsq ldst@GOTOFF, %rdx movl %eax, (%rdx,%r15) movabsq ldst@GOTOFF, %rax addq %r15, %rax movq %rax, lptr(%rip) movabsq lsrc@GOTOFF, %rax movl (%rax,%r15), %eax movq lptr(%rip), %rdx movl %eax, (%rdx)
--	--

Large Models

Again, in order to access data at any position in the 64-bit addressing space, it is necessary to calculate the address explicitly²⁷, not unlike the medium code model.

Figure 3.18: Absolute Global Data Load and Store

static int src; static int dst; extern int *ptr;	Lsrc: .long Ldst: .long .extern ptr
dst = src;	movabs \$Lsrc,%rax ;R_X86_64_64 movabs \$Ldst,%rdx ;R_X86_64_64 movl (%rax),%ecx movl %ecx,(%rdx)
ptr = &dst;	movabs \$ptr,%rax ;R_X86_64_64 movabs \$Ldst,%rdx ;R_X86_64_64 movq %rdx,(%rax)
*ptr = src;	movabs \$Lsrc,%rax ;R_X86_64_64 movabs \$ptr,%rdx ;R_X86_64_64 movl (%rax),%ecx movq (%rdx),%rdx movl %ecx,(%rdx)

Figure 3.19: Faster Absolute Global Data Load and Store

*ptr = src;	movabs \$ptr,%rdx ;R_X86_64_64 movl Lsrc(%rip),%ecx movq (%rdx),%rdx movl %ecx,(%rdx)
-------------	--

For position-independent code access to both static and external global data assumes that the GOT address is stored in a dedicated register. In these examples we assume it is in %r15²⁸ (see Function Prologue):

²⁷ If, at code generation-time, it is determined that a referred to global data object address is resolved within 2GB, the %rip-relative addressing mode can be used instead. See example in figure 3.19.

²⁸ If, at code generation-time, it is determined that a referred to global data object address is resolved within 2GB, the %rip-relative addressing mode can be used instead. See example in figure 3.21.

Figure 3.20: Position-Independent Global Data Load and Store

static int src; static int dst; extern int *ptr;	Lsrc: .long Ldst: .long .extern ptr
dst = src;	movabs \$Lsrc@GOTOFF,%rax ;R_X86_64_GOTOFF64 movabs \$Ldst@GOTOFF,%rdx ;R_X86_64_GOTOFF64 movl (%rax,%r15),%ecx movl %ecx,(%rdx,%r15)
ptr = &dst;	movabs \$ptr@GOT,%rax ;R_X86_64_GOT64 movabs \$Ldst@GOTOFF,%rdx ;R_X86_64_GOTOFF64 movq (%rax,%r15),%rax leaq (%rdx,%r15),%rcx movq %rcx,(%rax)
*ptr = src;	movabs \$Lsrc@GOTOFF,%rax ;R_X86_64_GOTOFF64 movabs \$ptr@GOT,%rdx ;R_X86_64_GOT64 movl (%rax,%r15),%ecx movq (%rdx,%r15),%rdx movl %ecx,(%rdx)

Figure 3.21: Faster Position-Independent Global Data Load and Store

*ptr = src;	movabs \$ptr@GOT,%rdx ;R_X86_64_GOT64 movl Lsrc(%rip),%ecx movq (%rdx,%r15),%rdx movl %ecx,(%rdx)
-------------	--

3.5.5 Function Calls

Small and Medium Models

Figure 3.22: Position-Independent Direct Function Call (Small and Medium Model)

extern void function (); function ();	.globl function call function@PLT
--	--------------------------------------

Figure 3.23: Position-Independent Indirect Function Call

extern void (*ptr) (); extern void name (); ptr = name; (*ptr) ();	.globl ptr, name movq ptr@GOTPCREL(%rip), %rax movq name@GOTPCREL(%rip), %rdx movq %rdx, (%rax) movq ptr@GOTPCREL(%rip), %rax call *(%rax)
---	---

Large models

It cannot be assumed that a function is within 2GB in general. Therefore, it is necessary to explicitly calculate the desired address reaching the whole 64-bit address space.

Figure 3.24: Absolute Direct and Indirect Function Call

static (*ptr) (void); extern foo (void); static bar (void);	Lptr: .quad .globl foo Lbar: ...
foo ();	movabs \$foo,%r11 ;R_X86_64_64 call *%r11
bar ();	movabs \$Lbar,%r11 ;R_X86_64_64 call *%r11
ptr = foo;	movabs \$Lptr,%rax ;R_X86_64_64 movabs \$foo,%r11 ;R_X86_64_64 movq %r11, (%rax)
ptr = bar;	movabs \$Lbar,%r11 ;R_X86_64_64 movq %r11, (%rax)
(*ptr) ();	movabs \$Lptr,%r11 ;R_X86_64_64 call *(%r11)

And in the case of position-independent objects ²⁹:

²⁹See subsection “Implementation advice” for some optimizations.

Figure 3.25: Position-Independent Direct and Indirect Function Call

static (*ptr) (void); extern foo (void); static bar (void);	Lptr: .quad .globl foo Lbar: ...
foo (); bar ();	movabs \$foo@PLTOFF,%r11 ;R_X86_64_PLTOFF64 call *(%r11,%r15) movabs \$Lbar@GOTOFF,%r11 ;R_X86_64_GOTOFF64 leaq (%r11,%r15),%r11 call *%r11
ptr = foo; ptr = bar;	movabs \$Lptr@GOTOFF,%rax ;R_X86_64_GOTOFF64 movabs \$foo@PLTOFF,%r11 ;R_X86_64_PLTOFF64 leaq (%r11,%r15),%r11 movq %r11, (%rax,%r15) movabs \$Lbar@GOTOFF,%r11 ;R_X86_64_GOTOFF64 leaq (%r11,%r15),%r11 movq %r11, (%rax,%r15)
(*ptr) ();	movabs \$Lptr@GOTOFF,%r11 ;R_X86_64_GOTOFF64 call *(%r11,%r15)

Implementation advice

If, at code generation-time, certain conditions are determined, it's possible to generate faster or smaller code sequences as the large model normally requires. When:

(absolute) target of function call is within 2GB , a direct call or `%rip`-relative addressing might be used:

bar ();	call Lbar
ptr = bar;	movabs \$Lptr,%rax ;R_X86_64_64 leaq \$Lbar(%rip),%r11 movq %r11, (%rax)

(PIC) the base of GOT is within 2GB an indirect call to the GOT entry might be implemented like so:

foo ();	call *(foo@GOT) ;R_X86_64_GOTPCREL
---------	------------------------------------

(PIC) the base of PLT is within 2GB , the PLT entry may be referred to relatively to `%rip`:

ptr = foo;	movabs \$Lptr@GOTOFF,%rax ;R_X86_64_GOTOFF64 leaq \$foo@PLT(%rip),%r11 ;R_X86_64_PLT32 movq %r11, (%rax,%r15)
------------	---

(PIC) target of function call is within 2GB and is **either not global or bound locally**, a direct call to the symbol may be used or it may be referred to relatively to `%rip`:

bar ();	call	Lbar
ptr = bar;	movabs	\$Lptr@GOTOFF,%rax ;R_X86_64_GOTOFF64
	leaq	\$Lbar(%rip),%r11
	movq	%r11, (%rax,%r15)

3.5.6 Branching

Small and Medium Models

As all labels are within 2GB no special care has to be taken when implementing branches. The full AMD64 ISA is usable.

Large Models

Because functions can be theoretically up to 16EB long, the maximum 32-bit displacement of conditional and unconditional branches in the AMD64 ISA are not enough to address the branch target. Therefore, a branch target address is calculated explicitly ³⁰. For absolute objects:

Figure 3.26: Absolute Branching Code

if (!a)	testl	%eax,%eax
{	jnz	1f
	movabs	\$2f,%r11 ;R_X86_64_64
	jmpq	*%r11
...	1:	...
}	2:	
goto Label;	movabs	\$Label,%r11 ;R_X86_64_64
	jmpq	*%r11
...	...	
Label:	Label:	

³⁰If, at code generation-time, it is determined that the target addresses are within 2GB, alternatively, branch target addresses may be calculated implicitly (see figure 3.27)

Figure 3.27: Implicit Calculation of Target Address

if (!a)	testl %eax,%eax
{	jz 2f
...	1: ...
}	2:
goto Label;	jmp Label
...	...
Label:	Label:

For position-independent objects:

Figure 3.28: Position-Independent Branching Code

if (!a)	testl %eax,%eax
{	jnz 1f
	movabs \$2f@GOTOFF,%r11 ;R_X86_64_GOTOFF64
	leaq (%r11,%r15),%r11
	jmpq *%r11
...	1: ...
}	2:
goto Label;	movabs \$Label@GOTOFF,%r11 ;R_X86_64_GOTOFF64
	leaq (%r11,%r15),%r11
	jmpq *%r11
...	
...	Label:
Label:	

For absolute objects, the implementation of the `switch` statement is:

Figure 3.29: Absolute Switch Code

switch (a)	
{	
	cmpl \$0,%eax
	jl .Ldefault
	cmpl \$2,%eax
	jg .Ldefault
	movabs \$.Ltable,%r11 ;R_X86_64_64
	jmpq *(%r11,%eax,8)
	.section .lrodata,"aLM",@progbits,8
	.align 8
	.Ltable:
	.quad .Lcase0 ;R_X86_64_64
	.quad .Ldefault ;R_X86_64_64
	.quad .Lcase2 ;R_X86_64_64
	.previous
default:	.Ldefault:
...	...
case 0:	.Lcase0:
...	...
case 2:	.Lcase2:
...	...
}	

When building position-independent objects, the `switch` statement implementation changes to:

Figure 3.30: Position-Independent Switch Code

switch (a)	cmpl \$0,%eax
{	jl .Ldefault
	cmpl \$2,%eax
	jg .Ldefault
	movabs \$.Ltable@GOTOFF,%r11 ;R_X86_64_GOTOFF64
	leaq (%r11,%r15),%r11
	movq *(%r11,%eax,8),%r11
	leaq (%r11,%r15),%r11
	jmpq *%r11
	.section .lrodata,"aLM",@progbits,8
	.align 8
	.Ltable:
	.quad .Lcase0@GOTOFF ;R_X86_64_GOTOFF64
	.quad .Ldefault@GOTOFF ;R_X86_64_GOTOFF64
	.quad .Lcase2@GOTOFF ;R_X86_64_GOTOFF64
	.previous
default:	.Ldefault:
...	...
case 0:	.Lcase0:
...	...
case 2:	.Lcase2:
...	...
}	

31

3.5.7 Variable Argument Lists

Some otherwise portable C programs depend on the argument passing scheme, implicitly assuming that all arguments are passed on the stack, and arguments appear in increasing order on the stack. Programs that make these assumptions never have been portable, but they have worked on many implementations. However, they do not work on the AMD64 architecture because some arguments are passed in registers. Portable C programs must use the header file `<stdarg.h>` in order to handle variable argument lists.

When a function taking variable-arguments is called, `%al` must be set to the total number of floating point parameters passed to the function in vector registers.³²

³¹The `jump-table` is emitted in a `different section` so as to occupy `cache lines without instruction bytes`, thus avoiding exclusive cache subsystems to `thrash`.

³²This implies that the only legal values for `%al` when calling a function with variable-argument lists are 0 to 8 (inclusive).

When `__m256` or `__m512` is passed as variable-argument, it should always be passed on stack. Only named `__m256` and `__m512` arguments may be passed in register as specified in section 3.2.3.

Figure 3.31: Parameter Passing Example with Variable-Argument List

```
int a, b;
long double ld;
double m, n;
__m256 u, y;
__m512 v, z;

extern void func (int a, double m, __m256 u, __m512 v, ...);

func (a, m, u, v, b, ld, y, z, n);
```

Figure 3.32: Register Allocation Example for Variable-Argument List

General Purpose Registers	Floating Point Registers	Stack Frame Offset
%rdi: a	%xmm0: m	0: ld
%rsi: b	%ymm1: u	32: y
%rax: 3	%zmm2: v	
	%xmm3: n	

The Register Save Area

The prologue of a function taking a variable argument list and known to call the macro `va_start` is expected to save the argument registers to the *register save area*. Each argument register has a fixed offset in the register save area as defined in the figure 3.33.

Only registers that might be used to pass arguments need to be saved. Other registers are not accessed and can be used for other purposes. If a function is known to never accept arguments passed in registers³³, the register save area may be omitted entirely.

The prologue should use `%al` to avoid unnecessarily saving XMM registers. This is especially important for integer only programs to prevent the initialization of the XMM unit.

³³This fact may be determined either by exploring types used by the `va_arg` macro, or by the fact that the named arguments already are exhausted the argument registers entirely.

Figure 3.33: Register Save Area

Register	Offset
%rdi	0
%rsi	8
%rdx	16
%rcx	24
%r8	32
%r9	40
%xmm0	48
%xmm1	64
...	
%xmm7	160

The `va_list` Type

The `va_list` type is an array containing a single element of one structure containing the necessary information to implement the `va_arg` macro. The C definition of `va_list` type is given in figure 3.34.

Figure 3.34: `va_list` Type Declaration

```
typedef struct {
    unsigned int gp_offset;
    unsigned int fp_offset;
    void *overflow_arg_area;
    void *reg_save_area;
} va_list[1];
```

The `va_start` Macro

The `va_start` macro initializes the structure as follows:

`reg_save_area` The element points to the start of the register save area.

`overflow_arg_area` This pointer is used to fetch arguments passed on the stack. It is initialized with the address of the first argument passed on the stack, if any, and then always updated to point to the start of the next argument on the stack.

gp_offset The element holds the offset in bytes from `reg_save_area` to the place where the next available general purpose argument register is saved. In case all argument registers have been exhausted, it is set to the value 48 ($6 * 8$).

fp_offset The element holds the offset in bytes from `reg_save_area` to the place where the next available floating point argument register is saved. In case all argument registers have been exhausted, it is set to the value 176 ($6 * 8 + 8 * 16$).

The `va_arg` Macro

The algorithm for the generic `va_arg(l, type)` implementation is defined as follows:

1. Determine whether `type` may be passed in the registers. If not go to step 7.
2. Compute `num_gp` to hold the number of general purpose registers needed to pass `type` and `num_fp` to hold the number of floating point registers needed.
3. Verify whether arguments fit into registers. In the case:

$$l->gp_offset > 48 - num_gp * 8$$

or

$$l->fp_offset > 176 - num_fp * 16$$

go to step 7.

4. Fetch `type` from `l->reg_save_area` with an offset of `l->gp_offset` and/or `l->fp_offset`. This may require copying to a temporary location in case the parameter is passed in **different register classes** or requires an alignment greater than 8 for general purpose registers and 16 for XMM registers.
5. Set:
$$l->gp_offset = l->gp_offset + num_gp * 8$$
$$l->fp_offset = l->fp_offset + num_fp * 16.$$
6. Return the fetched `type`.
7. Align `l->overflow_arg_area` upwards to a 16 byte boundary if alignment needed by `type` exceeds 8 byte boundary.
8. Fetch `type` from `l->overflow_arg_area`.

9. Set `l->overflow_arg_area` to:

`l->overflow_arg_area + sizeof(type)`

10. Align `l->overflow_arg_area` upwards to an 8 byte boundary.

11. Return the fetched `type`.

The `va_arg` macro is usually implemented as a compiler builtin and expanded in simplified forms for each particular type. Figure 3.35 is a sample implementation of the `va_arg` macro.

Figure 3.35: Sample Implementation of `va_arg(l, int)`

	<code>movl</code>	<code>l->gp_offset, %eax</code>	
	<code>cmpl</code>	<code>\$48, %eax</code>	Is register available?
	<code>jae</code>	<code>stack</code>	If not, use stack
	<code>leal</code>	<code>\$8(%rax), %edx</code>	Next available register
	<code>addq</code>	<code>l->reg_save_area, %rax</code>	Address of saved register
	<code>movl</code>	<code>%edx, l->gp_offset</code>	Update <code>gp_offset</code>
	<code>jmp</code>	<code>fetch</code>	
stack:	<code>movq</code>	<code>l->overflow_arg_area, %rax</code>	Address of stack slot
	<code>leaq</code>	<code>8(%rax), %rdx</code>	Next available stack slot
	<code>movq</code>	<code>%rdx, l->overflow_arg_area</code>	Update
fetch:	<code>movl</code>	<code>(%rax), %eax</code>	Load argument

3.6 DWARF Definition

This section³⁴ defines the Debug With Arbitrary Record Format (DWARF) debugging format for the AMD64 processor family. The AMD64 ABI does not define a debug format. However, all systems that do implement DWARF on AMD64 shall use the following definitions.

DWARF is a specification developed for symbolic, source-level debugging. The debugging information format does not favor the design of any compiler or debugger. For more information on DWARF, see *DWARF Debugging Format Standard*, available at: <http://www.dwarfstd.org/>.

³⁴This section is structured in a way similar to the PowerPC psABI

3.6.1 DWARF Release Number

The DWARF definition requires some machine-specific definitions. The register number mapping needs to be specified for the AMD64 registers. In addition, starting with version 3 the DWARF specification requires processor-specific address class codes to be defined.

3.6.2 DWARF Register Number Mapping

Table 3.36³⁵ outlines the register number mapping for the AMD64 processor family.³⁶

3.7 Stack Unwind Algorithm

The stack frames are not self descriptive and where stack unwinding is desirable (such as for exception handling) additional unwind information needs to be generated. The information is stored in an allocatable section `.eh_frame` whose format is identical to `.debug_frame` defined by the DWARF debug information standard, see *DWARF Debugging Information Format*, with the following extensions:

Position independence In order to avoid load time relocations for position independent code, the FDE CIE offset pointer should be stored relative to the start of CIE table entry. Frames using this extension of the DWARF standard must set the CIE identifier tag to 1.

Outgoing arguments area delta To maintain the size of the temporarily allocated outgoing arguments area present on the end of the stack (when using `push` instructions), operation `GNU_ARGS_SIZE (0x2e)` can be used. This operation takes a single `uleb128` argument specifying the current size. This information is used to adjust the stack frame when jumping into the exception handler of the function after unwinding the stack frame. Additionally the CIE Augmentation shall contain an exact specification of the encoding used. It is recommended to use a PC relative encoding whenever possible and adjust the size according to the code model used.

CIE Augmentations: The augmentation field is formatted according to the augmentation field forming string stored in the CIE header.

The string may contain the following characters:

³⁵The table defines Return Address to have a register number, even though the address is stored in `0(%rsp)` and not in a physical register.

³⁶This document does not define mappings for privileged registers.

Figure 3.36: DWARF Register Number Mapping

Register Name	Number	Abbreviation
General Purpose Register RAX	0	%rax
General Purpose Register RDX	1	%rdx
General Purpose Register RCX	2	%rcx
General Purpose Register RBX	3	%rbx
General Purpose Register RSI	4	%rsi
General Purpose Register RDI	5	%rdi
Frame Pointer Register RBP	6	%rbp
Stack Pointer Register RSP	7	%rsp
Extended Integer Registers 8-15	8-15	%r8-%r15
Return Address RA	16	
Vector Registers 0-7	17-24	%xmm0-%xmm7
Extended Vector Registers 8-15	25-32	%xmm8-%xmm15
Floating Point Registers 0-7	33-40	%st0-%st7
MMX Registers 0-7	41-48	%mm0-%mm7
Flag Register	49	%rFLAGS
Segment Register ES	50	%es
Segment Register CS	51	%cs
Segment Register SS	52	%ss
Segment Register DS	53	%ds
Segment Register FS	54	%fs
Segment Register GS	55	%gs
Reserved	56-57	
FS Base address	58	%fs.base
GS Base address	59	%gs.base
Reserved	60-61	
Task Register	62	%tr
LDT Register	63	%ldtr
128-bit Media Control and Status	64	%mxcsr
x87 Control Word	65	%fcw
x87 Status Word	66	%fsw
Upper Vector Registers 16-31	67-82	%xmm16-%xmm31
Reserved	83-117	
Vector Mask Registers 0-7	118-125	%k0-%k7
Reserved	126-129	
APX Integer Registers 16-31	130-145	%r16-%r31
Tile Registers 0-7	146-153	%tmm0-%tmm7
Tile Control Register	154	%tilecfg

Figure 3.37: Pointer Encoding Specification Byte

Mask	Meaning
0x1	Values are stored as <code>uleb128</code> or <code>sleb128</code> type (according to flag 0x8)
0x2	Values are stored as 2 bytes wide integers (<code>udata2</code> or <code>sdata2</code>)
0x3	Values are stored as 4 bytes wide integers (<code>udata4</code> or <code>sdata4</code>)
0x4	Values are stored as 8 bytes wide integers (<code>udata8</code> or <code>sdata8</code>)
0x8	Values are signed
0x10	Values are PC relative
0x20	Values are text section relative
0x30	Values are data section relative
0x40	Values are relative to the start of function

z Indicates that a `uleb128` is present determining the size of the augmentation section.

L Indicates the encoding (and thus presence) of an LSDA pointer in the FDE augmentation.

The data filed consist of single byte specifying the way pointers are encoded. It is a mask of the values specified by the table 3.37.

The default DWARF pointer encoding (direct 4-byte absolute pointers) is represented by value 0.

R Indicates a non-default pointer encoding for FDE code pointers. The forming is represented by a single byte in the same way as in the ‘L’ command.

P Indicates the presence and an encoding of a language personality routine in the CIE augmentation. The encoding is represented by a single byte in the same way as in the ‘L’ command followed by a pointer to the personality function encoded by the specified encoding.

When the augmentation is present, the first command must always be ‘z’ to allow easy skipping of the information.

In order to simplify manipulation of the unwind tables, the runtime library provide higher level API to stack unwinding mechanism, for details see section 6.2.

Chapter 4

Object Files

4.1 ELF Header

4.1.1 Machine Information

Programming Model

As described in Section 1, binaries using the AMD64 instruction set may program to either a 32-bit model (ILP32) or to a 64-bit model (LP64). This specification describes both binaries that use the ILP32 and the LP64 model.

File Class

For AMD64 ILP32 objects, the file class value in `e_ident[EI_CLASS]` must be `ELFCLASS32`. For AMD64 LP64 objects, the file class value must be `ELFCLASS64`.

Data Encoding

For the data encoding in `e_ident[EI_DATA]`, AMD64 objects use `ELFDATA2LSB`.

Processor identification

Processor identification resides in the ELF headers `e_machine` member and must have the value `EM_X86_64`.¹

¹The value of this identifier is 62.

4.1.2 Number of Program Headers

The `e_phnum` member contains the number of entries in the program header table. The product of `e_phentsize` and `e_phnum` gives the table's size in bytes. If a file has no program header table, `e_phnum` holds the value zero.

If the number of program headers is greater than or equal to `PN_XNUM` (0xffff), this member has the value `PN_XNUM` (0xffff). The actual number of program header table entries is contained in the `sh_info` field of the section header at index 0. Otherwise, the `sh_info` member of the initial entry contains the value zero.

4.2 Sections

4.2.1 Section Flags

In order to allow linking object files of different code models, it is necessary to provide for a way to differentiate those sections which may hold more than 2GB from those which may not. This is accomplished by defining a processor-specific section attribute flag for `sh_flag` (see table 4.1).

Table 4.1: AMD64 Specific Section Header Flag, `sh_flags`

Name	Value
<code>SHF_X86_64_LARGE</code>	<code>0x10000000</code>

SHF_X86_64_LARGE If an object file section does *not* have this flag set, then it may not hold more than 2GB and can be freely referred to in objects using smaller code models. Otherwise, only objects using larger code models can refer to them. For example, a medium code model object can refer to data in a section that sets this flag besides being able to refer to data in a section that does not set it; likewise, a small code model object can refer only to code in a section that does not set this flag.

4.2.2 Section types

Table 4.2: Section Header Types

sh_type name	Value
SHT_X86_64_UNWIND	0x70000001

SHT_X86_64_UNWIND This section contains unwind function table entries for stack unwinding. The contents are described in Section 4.2.4 of this document.

4.2.3 Special Sections

Table 4.3: Special sections

Name	Type	Attributes
.got	SHT_PROGBITS	SHF_ALLOC+SHF_WRITE
.plt	SHT_PROGBITS	SHF_ALLOC+SHF_EXECINSTR
.eh_frame	SHT_X86_64_UNWIND	SHF_ALLOC

.got This section holds the global offset table.

.plt This section holds the procedure linkage table.

.eh_frame This section holds the unwind function table. The contents are described in Section 4.2.4 of this document.

The additional sections defined in table 4.4 are used by a system supporting the large code model.

Table 4.4: Additional Special Sections for the Medium and Large Code Models

Name	Type	Attributes
.lbss	SHT_NOBITS	SHF_ALLOC+SHF_WRITE+SHF_X86_64_LARGE
.ldata	SHT_PROGBITS	SHF_ALLOC+SHF_WRITE+SHF_X86_64_LARGE
.ldata1	SHT_PROGBITS	SHF_ALLOC+SHF_WRITE+SHF_X86_64_LARGE
.lgot	SHT_PROGBITS	SHF_ALLOC+SHF_WRITE+SHF_X86_64_LARGE
.lplt	SHT_PROGBITS	SHF_ALLOC+SHF_EXECINSTR+SHF_X86_64_LARGE
.lrodata	SHT_PROGBITS	SHF_ALLOC+SHF_X86_64_LARGE
.lrodata1	SHT_PROGBITS	SHF_ALLOC+SHF_X86_64_LARGE
.ltext	SHT_PROGBITS	SHF_ALLOC+SHF_EXECINSTR+SHF_X86_64_LARGE

In order to enable static linking of objects using different code models, the following section ordering is suggested:

`.plt .init .fini .text .got .rodata .rodata1 .data .data1 .bss` These sections can have a combined size of up to 2GB.

`.lplt .ltext .lgot .lrodata .lrodata1 .ldata .ldata1 .lbss` These sections plus the above can have a combined size of up to 16EB.

4.2.4 EH_FRAME sections

The call frame information needed for unwinding the stack is output into one or more ELF sections of type `SHT_X86_64_UNWIND`. In the simplest case there will be one such section per object file and it will be named `.eh_frame`. An `.eh_frame` section consists of one or more subsections. Each subsection contains a CIE (Common Information Entry) followed by varying number of FDEs (Frame Descriptor Entry). A FDE corresponds to an explicit or compiler generated function in a compilation unit, all FDEs can access the CIE that begins their subsection for data. If the code for a function is not one contiguous block, there will be a separate FDE for each contiguous sub-piece.

If an object file contains C++ template instantiations there shall be a separate CIE immediately preceding each FDE corresponding to an instantiation.

Using the preferred encoding specified below, the `.eh_frame` section can be entirely resolved at link time and thus can become part of the text segment.

`EH_PE` encoding below refers to the pointer encoding as specified in the enhanced LSB Chapter 7 for `Eh_Frame_Hdr`.

Table 4.5: Common Information Entry (CIE)

Field	Length (byte)	Description
Length	4	Length of the CIE (not including this 4-byte field)
CIE id	4	Value 0 for <code>.eh_frame</code> (used to distinguish CIEs and FDEs when scanning the section)
Version	1	Value One (1)
CIE Augmentation String	string	Null-terminated string with legal values being "" or 'z' optionally followed by single occurrences of 'P', 'L', or 'R' in any order. The presence of character(s) in the string dictates the content of field 8, the Augmentation Section. Each character has one or two associated operands in the AS (see table 4.6 for which ones). Operand order depends on position in the string ('z' must be first).
Code Align Factor	uleb128	To be multiplied with the "Advance Location" instructions in the Call Frame Instructions
Data Align Factor	sleb128	To be multiplied with all offsets in the Call Frame Instructions
Ret Address Reg	1/uleb128	A "virtual" register representation of the return address. In Dwarf V2, this is a byte, otherwise it is uleb128. It is a byte in gcc 3.3.x
Optional CIE Augmentation Section	varying	Present if Augmentation String in Augmentation Section field 4 is not 0. See table 4.6 for the content.
Optional Call Frame Instructions	varying	

Table 4.6: CIE Augmentation Section Content

Char	Operands	Length (byte)	Description
z	size	uleb128	Length of the remainder of the Augmentation Section
P	personality_enc	1	Encoding specifier - preferred value is a pc-relative, signed 4-byte
	personality routine	(encoded)	Encoded pointer to personality routine (actually to the PLT entry for the personality routine)
R	code_enc	1	Non-default encoding for the code-pointers (FDE members <code>initial_location</code> and <code>address_range</code> and the operand for <code>DW_CFA_set_loc</code>) - preferred value is pc-relative, signed 4-byte
L	lsda_enc	1	FDE augmentation bodies may contain LSDA pointers. If so they are encoded as specified here - preferred value is pc-relative, signed 4-byte possibly indirect thru a GOT entry

Table 4.7: Frame Descriptor Entry (FDE)

Field	Length (byte)	Description
Length	4	Length of the FDE (not including this 4-byte field)
CIE pointer	4	Distance from this field to the nearest preceding CIE (the value is subtracted from the current address). This value can never be zero and thus can be used to distinguish CIE's and FDE's when scanning the <code>.eh_frame</code> section
Initial Location	var	Reference to the function code corresponding to this FDE. If 'R' is missing from the CIE Augmentation String, the field is an 8-byte absolute pointer. Otherwise, the corresponding <code>EH_PE</code> encoding in the CIE Augmentation Section is used to interpret the reference
Address Range	var	Size of the function code corresponding to this FDE. If 'R' is missing from the CIE Augmentation String, the field is an 8-byte unsigned number. Otherwise, the size is determined by the corresponding <code>EH_PE</code> encoding in the CIE Augmentation Section (the value is always absolute)
Optional FDE Augmentation Section	var	Present if CIE Augmentation String is non-empty. See table 4.8 for the content.
Optional Call Frame Instructions	var	

Table 4.8: FDE Augmentation Section Content

Char	Operands	Length (byte)	Description
z	length	uleb128	Length of the remainder of the Augmentation Section
L	LSDA	var	LSDA pointer, encoded in the format specified by the corresponding operand in the CIE's augmentation body. (only present if length > 0).

The existence and size of the optional call frame instruction area must be computed based on the overall size and the offset reached while scanning the preceding fields of the CIE or FDE.

The overall size of a `.eh_frame` section is given in the ELF section header. The only way to determine the number of entries is to scan the section until the end, counting entries as they are encountered.

4.3 Symbol Table

The discussion of "Function Addresses" in Section 5.2 defines some special values for symbol table fields.

The `STT_GNU_IFUNC`² symbol type is optional. It is the same as `STT_FUNC` except that it always points to a function or piece of executable code which takes no arguments and returns a function pointer. If an `STT_GNU_IFUNC` symbol is referred to by a relocation, then evaluation of that relocation is delayed until load-time. The value used in the relocation is the function pointer returned by an invocation of the `STT_GNU_IFUNC` symbol.

The purpose of the `STT_GNU_IFUNC` symbol type is to allow the run-time to select between multiple versions of the implementation of a specific function. The selection made in general will take the currently available hardware into account and select the most appropriate version.

²It is specified in **Linux Extensions to gABI** at <https://gitlab.com/x86-psABIs/Linux-ABI>

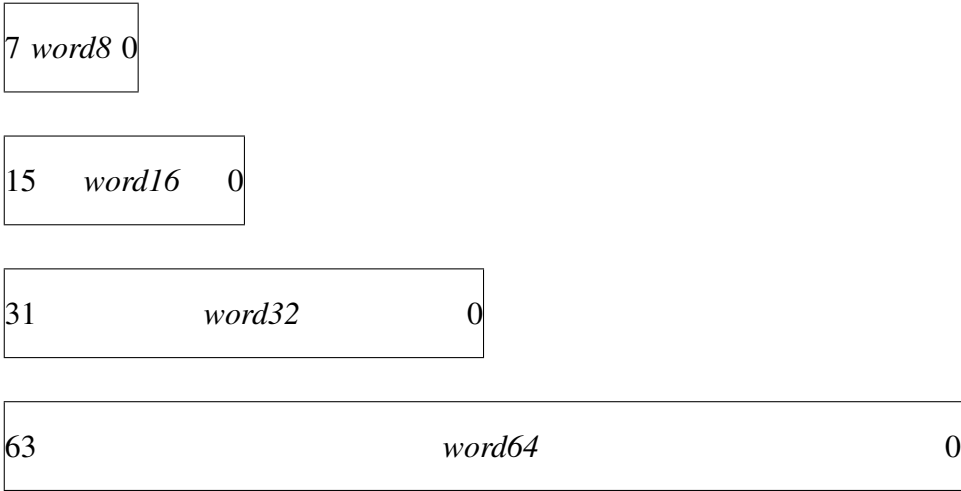
This section is essentially a rulebook for how assembler/linker should turn symbolic references in x86-64 machine code into concrete numeric values!

4.4 Relocation

4.4.1 Relocation Types

Figure 4.1 shows the allowed relocatable fields.

Figure 4.1: Relocatable Fields



<i>word8</i>	This specifies a 8-bit field occupying 1 byte.
<i>word16</i>	This specifies a 16-bit field occupying 2 bytes with arbitrary byte alignment. These values use the same byte order as other word values in the AMD64 architecture.
<i>word32</i>	This specifies a 32-bit field occupying 4 bytes with arbitrary byte alignment. These values use the same byte order as other word values in the AMD64 architecture.
<i>word64</i>	This specifies a 64-bit field occupying 8 bytes with arbitrary byte alignment. These values use the same byte order as other word values in the AMD64 architecture.
<i>wordclass</i>	This specifies <i>word64</i> for LP64 and specifies <i>word32</i> for ILP32.

The following notations are used for specifying relocations in table 4.9:

A Represents the addend used to compute the value of the relocatable field.

- B** Represents the base address at which a shared object has been loaded into memory during execution. Generally, a shared object is built with a 0 base virtual address, but the execution address will be different.
- G** Represents the offset into the global offset table at which the relocation entry's symbol will reside during execution.
- GOT** Represents the address of the global offset table.
- L** Represents the place (section offset or address) of the Procedure Linkage Table entry for a symbol.
- P** Represents the place (section offset or address) of the storage unit being relocated (computed using `r_offset`).
- S** Represents the value of the symbol whose index resides in the relocation entry.
- Z** Represents the size of the symbol whose index resides in the relocation entry.

The AMD64 LP64 ABI architecture uses only `Elf64_Rela` relocation entries with explicit addends. The `r_addend` member serves as the relocation addend.

The AMD64 ILP32 ABI architecture uses only `Elf32_Rela` relocation entries in relocatable files. Executable files or shared objects may use either `Elf32_Rela` or `Elf32_Rel` relocation entries.

Table 4.9: Relocation Types

Name	Value	Field	Calculation
R_X86_64_NONE	0	none	none
R_X86_64_64	1	word64	S + A
R_X86_64_PC32	2	word32	S + A - P
R_X86_64_GOT32	3	word32	G + A
R_X86_64_PLT32	4	word32	L + A - P
R_X86_64_COPY	5	none	none
R_X86_64_GLOB_DAT	6	wordclass	S
R_X86_64_JUMP_SLOT	7	wordclass	S
R_X86_64_RELATIVE	8	wordclass	B + A
R_X86_64_GOTPCREL	9	word32	G + GOT + A - P
R_X86_64_32	10	word32	S + A
R_X86_64_32S	11	word32	S + A
R_X86_64_16	12	word16	S + A
R_X86_64_PC16	13	word16	S + A - P
R_X86_64_8	14	word8	S + A
R_X86_64_PC8	15	word8	S + A - P
R_X86_64_DTPMOD64	16	word64	
R_X86_64_DTPOFF64	17	word64	
R_X86_64_TPOFF64	18	word64	
R_X86_64_TLSGD	19	word32	
R_X86_64_TLSLD	20	word32	
R_X86_64_DTPOFF32	21	word32	
R_X86_64_GOTTPOFF	22	word32	
R_X86_64_TPOFF32	23	word32	
R_X86_64_PC64 [†]	24	word64	S + A - P
R_X86_64_GOTOFF64 [†]	25	word64	S + A - GOT
R_X86_64_GOTPC32	26	word32	GOT + A - P
R_X86_64_SIZE32	32	word32	Z + A
R_X86_64_SIZE64 [†]	33	word64	Z + A
R_X86_64_GOTPC32_TLSDESC	34	word32	
R_X86_64_TLSDESC_CALL	35	none	
R_X86_64_TLSDESC	36	word64×2	
R_X86_64_IRELATIVE	37	wordclass	indirect (B + A)
R_X86_64_RELATIVE64 ^{††}	38	word64	B + A
Deprecated	39		
Deprecated	40		
R_X86_64_GOTPCRELX	41	word32	G + GOT + A - P
R_X86_64_REX_GOTPCRELX	42	word32	G + GOT + A - P
R_X86_64_CODE_4_GOTPCRELX	43	word32	G + GOT + A - P
R_X86_64_CODE_4_GOTTPOFF	44	word32	
R_X86_64_CODE_4_GOTPC32_TLSDESC	45	word32	
R_X86_64_CODE_5_GOTPCRELX	46	word32	G + GOT + A - P
R_X86_64_CODE_5_GOTTPOFF	47	word32	
R_X86_64_CODE_5_GOTPC32_TLSDESC	48	word32	
R_X86_64_CODE_6_GOTPCRELX	49	word32	G + GOT + A - P
R_X86_64_CODE_6_GOTTPOFF	50	word32	
R_X86_64_CODE_6_GOTPC32_TLSDESC	51	word32	

[†] This relocation is used only for LP64.

^{††} This relocation only appears in ILP32 executable files or shared objects.

The special semantics for most of these relocation types are identical to those used for the Intel386 ABI.^{3 4}

The `R_X86_64_GOTPCREL` relocation has different semantics from the `R_X86_64_GOT32` or equivalent i386 `R_386_GOTPC` relocation. In particular, because the AMD64 architecture has an addressing mode relative to the instruction pointer, it is possible to load an address from the GOT using a single instruction. The calculation done by the `R_X86_64_GOTPCREL` relocation gives the difference between the location in the GOT where the symbol's address is given and the location where the relocation is applied.

For the occurrence of `name@GOTPCREL` in the following assembler instructions:

```
call    *name@GOTPCREL(%rip)
jmp     *name@GOTPCREL(%rip)
mov     name@GOTPCREL(%rip), %reg
test    %reg, name@GOTPCREL(%rip)
binop   name@GOTPCREL(%rip), %reg
```

where `binop` is one of `adc`, `add`, `and`, `cmp`, `or`, `sbb`, `sub`, `xor` instructions, the following GOTPCRELX relocations should be generated (see also section B.2):

`R_X86_64_GOTPCRELX` The instruction starts at 2 bytes before the relocation offset.

`R_X86_64_REX_GOTPCRELX` The instruction starts at 3 bytes before the relocation offset with the `REX` prefix.

`R_X86_64_CODE_4_GOTPCRELX` The instruction starts at 4 bytes before the relocation offset.

`R_X86_64_CODE_5_GOTPCRELX` The instruction starts at 5 bytes before the relocation offset.

`R_X86_64_CODE_6_GOTPCRELX` The instruction starts at 6 bytes before the relocation offset.

³Even though the AMD64 architecture supports IP-relative addressing modes, a GOT is still required since the offset from a particular instruction to a particular data item cannot be known by the static linker.

⁴Note that the AMD64 architecture assumes that offsets into GOT are 32-bit values, not 64-bit values. This choice means that a maximum of $2^{32}/8 = 2^{29}$ entries can be placed in the GOT. However, that should be more than enough for most programs. In the event that it is not enough, the linker could create multiple GOTs. Because 32-bit offsets are used, loads of global data do not require loading the offset into a displacement register; the base plus immediate displacement addressing form can be used.

The `R_X86_64_32` and `R_X86_64_32S` relocations truncate the computed value to 32-bits. The linker must verify that the generated value for the `R_X86_64_32` (`R_X86_64_32S`) relocation zero-extends (sign-extends) to the original 64-bit value.

A program or object file using `R_X86_64_8`, `R_X86_64_16`, `R_X86_64_PC16` or `R_X86_64_PC8` relocations is not conformant to this ABI, these relocations are only added for documentation purposes. The `R_X86_64_16`, and `R_X86_64_8` relocations truncate the computed value to 16-bits resp. 8-bits.

The relocations `R_X86_64_DTPMOD64`, `R_X86_64_DTPOFF64`, `R_X86_64_TPOFF64`, `R_X86_64_TLSD`, `R_X86_64_TLSD`, `R_X86_64_DTPOFF32`, `R_X86_64_GOTTPOFF` and `R_X86_64_TPOFF32` are listed for completeness. They are part of the Thread-Local Storage ABI extensions and are documented in the document called “ELF Handling for Thread-Local Storage”⁵. The following `GOTTPOFF` relocations should be generated:

`R_X86_64_GOTTPOFF` The instruction starts at 3 bytes before the relocation offset with the `REX` byte.

`R_X86_64_CODE_4_GOTTPOFF` The instruction starts at 4 bytes before the relocation offset.

`R_X86_64_CODE_5_GOTTPOFF` The instruction starts at 5 bytes before the relocation offset.

`R_X86_64_CODE_6_GOTTPOFF` The instruction starts at 6 bytes before the relocation offset.

Linker optimization must take the different instruction encoding into account.

The relocations `R_X86_64_GOTPC32_TLSDESC`, `R_X86_64_TLSDESC_CALL` and `R_X86_64_TLSDESC` are also used for Thread-Local Storage, but are not documented there as of this writing. A description can be found in the document “Thread-Local Storage Descriptors for IA32 and AMD64/EM64T”⁶. The following `GOTPC32_TLSDESC` relocations should be generated:

`R_X86_64_GOTPC32_TLSDESC` The instruction starts at 3 bytes before the relocation offset.

`R_X86_64_CODE_4_GOTPC32_TLSDESC` The instruction starts at 4 bytes before the relocation offset.

⁵This document is currently available via <http://www.akkadia.org/drepper/tls.pdf>

⁶This document is currently available via <http://www.fsfla.org/~lxoliva/writeups/TLS/RFC-TLSDESC-x86.txt>

`R_X86_64_CODE_5_GOTPC32_TLSDESC` The instruction starts at 5 bytes before the relocation offset.

`R_X86_64_CODE_6_GOTPC32_TLSDESC` The instruction starts at 6 bytes before the relocation offset.

Linker optimization must take the different instruction encoding into account.

In order to make this document self-contained, a description of the TLS relocations follows.

The `%fs` segment register is used to implement the thread pointer. The linear address of the thread pointer is stored at offset 0 relative to the `%fs` segment register. The following code loads the thread pointer in the `%rax` register:

```
movq    %fs:0, %rax
```

`R_X86_64_DTPMOD64` resolves to the index of the dynamic thread vector entry that points to the base address of the TLS block corresponding to the module that defines the referenced symbol. `R_X86_64_DTPOFF64` and `R_X86_64_DTPOFF32` compute the offset from the pointer in that entry to the referenced symbol. The linker generates such relocations in adjacent entries in the GOT, in response to `R_X86_64_TLSGD` and `R_X86_64_TLSLD` relocations. If the linker can compute the offset itself, because the referenced symbol binds locally, the relocations `R_X86_64_64` and `R_X86_64_32` may be used instead. Otherwise, such relocations are always in pairs, such that the `R_X86_64_DTPOFF64` relocation applies to the word64 right past the corresponding `R_X86_64_DTPMOD64` relocation.

`R_X86_64_TPOFF64` and `R_X86_64_TPOFF32` resolve to the offset from the thread pointer to a thread-local variable. The former is generated in response to `R_X86_64_GOTTPOFF`, that resolves to a PC-relative address of a GOT entry containing such a 64-bit offset.

`R_X86_64_TLSGD` and `R_X86_64_TLSLD` both resolve to PC-relative offsets to a DTPMOD GOT entry. The difference between them is that, for `R_X86_64_TLSGD`, the following GOT entry will contain the offset of the referenced symbol into its TLS block, whereas, for `R_X86_64_TLSLD`, the following GOT entry will contain the offset for the base address of the TLS block. The idea is that adding this offset to the result of `R_X86_64_DTPMOD32` for a symbol ought to yield the same as the result of `R_X86_64_DTPMOD64` for the same symbol.

`R_X86_64_TLSDESC` resolves to a pair of word64s, called TLS Descriptor, the first of which is a pointer to a function, followed by an argument. The function is passed a pointer to the this pair of entries in `%rax` and, using the argument in the second entry, it must compute and return in `%rax` the offset from the thread pointer to the symbol

referenced in the relocation, without modifying any registers other than processor flags. `R_X86_64_GOTPC32_TLSDESC` resolves to the PC-relative address of a TLS descriptor corresponding to the named symbol. `R_X86_64_TLSDESC_CALL` must annotate the instruction used to call the TLS Descriptor resolver function, so as to enable relaxation of that instruction.

`R_X86_64_IRELATIVE` is similar to `R_X86_64_RELATIVE` except that the value used in this relocation is the program address returned by the function, which takes no arguments, at the address of the result of the corresponding `R_X86_64_RELATIVE` relocation.

One use of the `R_X86_64_IRELATIVE` relocation is to avoid name lookup for the locally defined `STT_GNU_IFUNC` symbols at load-time. Support for this relocation is optional, but is required for the `STT_GNU_IFUNC` symbols.

4.4.2 Large Models

In order to extend both the PLT and the GOT beyond 2GB, it is necessary to add appropriate relocation types to handle full 64-bit addressing. See figure 4.10.

Table 4.10: Large Model Relocation Types

Name	Value	Field	Calculation
<code>R_X86_64_GOT64</code>	27	word64	$G + A$
<code>R_X86_64_GOTPCREL64</code>	28	word64	$G + GOT - P + A$
<code>R_X86_64_GOTPC64</code>	29	word64	$GOT - P + A$
Deprecated	30		
<code>R_X86_64_PLTOFF64</code>	31	word64	$L - GOT + A$

Chapter 5

Program Loading and Dynamic Linking

5.1 Program Loading

Program loading is a process of mapping file segments to virtual memory segments. For efficient mapping executable and shared object files must have segments whose file offsets and virtual addresses are congruent modulo the page size.

To save space the file page holding the last page of the text segment may also contain the first page of the data segment. The last data page may contain file information not relevant to the running process. Logically, the system enforces the memory permissions as if each segment were complete and separate; segments' addresses are adjusted to ensure each logical page in the address space has a single set of permissions. In the example above, the region of the file holding the end of text and the beginning of data will be mapped twice: at one virtual address for text and at a different virtual address for data.

The end of the data segment requires special handling for uninitialized data, which the system defines to begin with zero values. Thus if a file's last data page includes information not in the logical memory page, the extraneous data must be set to zero, not the unknown contents of the executable file. "Impurities" in the other three pages are not logically part of the process image; whether the system expunges them is unspecified.

One aspect of segment loading differs between executable files and shared objects. Executable file segments typically contain absolute code (see section 3.5 "Coding Examples"). For the process to execute correctly, the segments must reside at the virtual addresses used to build the executable file. Thus the system uses the `p_vaddr` values unchanged as virtual addresses.

On the other hand, shared object segments typically contain position-independent code. This lets a segments virtual address change from one process to another, without invalidating execution behavior. Though the system chooses virtual addresses for individ-

ual processes, it maintains the segments’ relative positions. Because position-independent code uses relative addressing between segments, the difference between virtual addresses in memory must match the difference between virtual addresses in the file.

5.1.1 Program header

The following AMD64 program header types are defined:

Table 5.1: Program Header Types

Name	Value
PT_GNU_EH_FRAME	0x6474e550
PT_SUNW_EH_FRAME	0x6474e550
PT_SUNW_UNWIND	0x6464e550

PT_GNU_EH_FRAME, PT_SUNW_EH_FRAME and PT_SUNW_UNWIND The segment contains the stack unwind tables. See Section 4.2.4 of this document. ¹

5.2 Dynamic Linking

Dynamic Section

Dynamic section entries give information to the dynamic linker. Some of this information is processor-specific, including the interpretation of some entries in the dynamic structure.

The following optional AMD64 Dynamic Array Tags are defined:

¹ The value for these program headers have been placed in the PT_LOOS and PT_HIOS (os specific range) in order to adapt to the existing GNU implementation. New OS’s wanting to agree on these program header should also add it into their OS specific range.

Table 5.2: Optional Dynamic Array Tags, d_tag

Name	Value	d_un
DT_X86_64_PLT	DT_LOPROC + 0	d_ptr
DT_X86_64_PLTSZ	DT_LOPROC + 1	d_val
DT_X86_64_PLTENT	DT_LOPROC + 3	d_val

DT_X86_64_PLT The address of the procedure linkage table.

DT_X86_64_PLTSZ The total size, in bytes, of the procedure linkage table.

DT_X86_64_PLTENT The size, in bytes, of a procedure linkage table entry. Always a power of 2.

DT_X86_64_PLT, DT_X86_64_PLTSZ and DT_X86_64_PLTENT are provided only if

- The procedure linkage table entries transfer control to external functions via indirect branch over the corresponding entry in the global offset table referenced by the r_offset field of the R_X86_64_JUMP_SLOT relocation. The r_addend field of the R_X86_64_JUMP_SLOT relocation stores the procedure linkage table offset of indirect branch in the corresponding procedure linkage table entry.
- All such procedure linkage table entries have the same layout. Each entry starts with an indirect branch over the global offset table entry. An ENDBR64 instruction may precede the indirect branch. There should be no functional side effect when the indirect branch is replaced by a direct branch.
- The only entry point of a procedure linkage table entry is the first byte of the entry.
- All entries have the same size and are aligned to the entry size.

If permitted, using DT_X86_64_PLT, DT_X86_64_PLTSZ and DT_X86_64_PLTENT together with the r_offset and r_addend fields of R_X86_64_JUMP_SLOT relocation, the dynamic linker may retrieve the indirect branch target address stored at the global offset table entry and rewrite the indirect branch in the procedure linkage table entry to a direct branch.

Global Offset Table (GOT)

Position-independent code cannot, in general, contain absolute virtual addresses. Global offset tables hold absolute addresses in private data, thus making the addresses available without compromising the position-independence and shareability of a program's text. A program references its global offset table using position-independent addressing and extracts absolute values, thus redirecting position-independent references to absolute locations.

If a program requires direct access to the absolute address of a symbol, that symbol will have a global offset table entry. Because the executable file and shared objects have separate global offset tables, a symbol's address may appear in several tables. The dynamic linker processes all the global offset table relocations before giving control to any code in the process image, thus ensuring the absolute addresses are available during execution.

The table's first entry (number zero) is reserved to hold the address of the dynamic structure, referenced with the symbol `_DYNAMIC`. This allows a program, such as the dynamic linker, to find its own dynamic structure without having yet processed its relocation entries. This is especially important for the dynamic linker, because it must initialize itself without relying on other programs to relocate its memory image. On the AMD64 architecture, entries one and two in the global offset table also are reserved.

The global offset table contains 64-bit addresses.

For the large models the GOT is allowed to be up to 16EB in size.

Figure 5.1: Global Offset Table

```
extern Elf64_Addr _GLOBAL_OFFSET_TABLE_ [];
```

The symbol `_GLOBAL_OFFSET_TABLE_` may reside in the middle of the `.got` section, allowing both negative and non-negative offsets into the array of addresses.

Function Addresses

References to the address of a function from an executable file and the shared objects associated with it might not resolve to the same value. References from within shared objects will normally be resolved by the dynamic linker to the virtual address of the function itself. References from within the executable file to a function defined in a shared object will normally be resolved by the link editor to the address of the procedure linkage table entry for that function within the executable file.

To allow comparisons of function addresses to work as expected, if an executable file references a function defined in a shared object, the link editor will place the address of the procedure linkage table entry for that function in its associated symbol table entry. This will result in symbol table entries with section index of `SHN_UNDEF` but a type of `STT_FUNC` and a non-zero `st_value`. A reference to the address of a function from within a shared library will be satisfied by such a definition in the executable.

Some relocations are associated with procedure linkage table entries. These entries are used for direct function calls rather than for references to function addresses. These relocations do not use the special symbol value described above. Otherwise a very tight endless loop would be created.

Procedure Linkage Table

Much as the global offset table redirects position-independent address calculations to absolute locations, the procedure linkage table redirects position-independent function calls to absolute locations. The link editor cannot resolve execution transfers (such as function calls) from one executable or shared object to another. Consequently, the link editor arranges to have the program transfer control to entries in the procedure linkage table. On the AMD64 architecture, procedure linkage tables reside in shared text, but they use addresses in the private global offset table. The dynamic linker determines the destinations' absolute addresses and modifies the global offset table's memory image accordingly. The dynamic linker thus can redirect the entries without compromising the position-independence and shareability of the program's text. Executable files and shared object files have separate procedure linkage tables. Unlike Intel386 ABI, this ABI uses the same procedure linkage table for both programs and shared objects (see figure 5.2).

Figure 5.2: Procedure Linkage Table (small and medium models)

```
.PLT0: pushq    GOT+8(%rip)           # GOT[1]
      jmp     *GOT+16(%rip)         # GOT[2]
      nopl    0x0(%rax)
.PLT1: jmp     *name1@GOTPCREL(%rip) # 16 bytes from .PLT0
      pushq   $index1
      jmp     .PLT0
.PLT2: jmp     *name2@GOTPCREL(%rip) # 16 bytes from .PLT1
      pushq   $index2
      jmp     .PLT0
.PLT3: ...
```

Following the steps below, the dynamic linker and the program “cooperate” to resolve symbolic references through the procedure linkage table and the global offset table.

1. When first creating the memory image of the program, the dynamic linker sets the second and the third entries in the global offset table to special values. Steps below explain more about these values.
2. Each shared object file in the process image has its own procedure linkage table, and control transfers to a procedure linkage table entry only from within the same object file.
3. For illustration, assume the program calls `name1`, which transfers control to the label `.PLT1`.
4. The first instruction jumps to the address in the global offset table entry for `name1`. Initially the global offset table holds the address of the following `pushq` instruction, not the real address of `name1`.
5. Now the program pushes a relocation index (*index*) on the stack. The relocation index is a 32-bit, non-negative index into the relocation table addressed by the `DT_JMPREL` dynamic section entry. The designated relocation entry will have type `R_X86_64_JUMP_SLOT`, and its offset will specify the global offset table entry used in the previous `jmp` instruction. The relocation entry contains a symbol table index that will reference the appropriate symbol, `name1` in the example.
6. After pushing the relocation index, the program then jumps to `.PLT0`, the first entry in the procedure linkage table. The `pushq` instruction places the value of the second global offset table entry (`GOT+8`) on the stack, thus giving the dynamic linker one word of identifying information. The program then jumps to the address in the third global offset table entry (`GOT+16`), which transfers control to the dynamic linker.
7. When the dynamic linker receives control, it unwinds the stack, looks at the designated relocation entry, finds the symbol’s value, stores the “real” address for `name1` in its global offset table entry, and transfers control to the desired destination.
8. Subsequent executions of the procedure linkage table entry will transfer directly to `name1`, without calling the dynamic linker a second time. That is, the `jmp` instruction at `.PLT1` will transfer to `name1`, instead of “falling through” to the `pushq` instruction.

The `LD_BIND_NOW` environment variable can change the dynamic linking behavior. If its value is non-null, the dynamic linker evaluates procedure linkage table entries before

transferring control to the program. That is, the dynamic linker processes relocation entries of type `R_X86_64_JUMP_SLOT` during process initialization. Otherwise, the dynamic linker evaluates procedure linkage table entries lazily, delaying symbol resolution and relocation until the first execution of a table entry.

Relocation entries of type `R_X86_64_TLSDESC` may also be subject to lazy relocation, using a single entry in the procedure linkage table and in the global offset table, at locations given by `DT_TLSDESC_PLT` and `DT_TLSDESC_GOT`, respectively, as described in “Thread-Local Storage Descriptors for IA32 and AMD64/EM64T”².

For self-containment, `DT_TLSDESC_GOT` specifies a GOT entry in which the dynamic loader should store the address of its internal TLS Descriptor resolver function, whereas `DT_TLSDESC_PLT` specifies the address of a PLT entry to be used as the TLS descriptor resolver function for lazy resolution from within this module. The PLT entry must push the linkmap of the module onto the stack and tail-call the internal TLS Descriptor resolver function.

Large Models

In the small and medium code models the size of both the PLT and the GOT is limited by the maximum 32-bit displacement size. Consequently, the base of the PLT and the top of the GOT can be at most 2GB apart.

Therefore, in order to support the available addressing space of 16EB, it is necessary to extend both the PLT and the GOT. Moreover, the PLT needs to support the GOT being over 2GB away and the GOT can be over 2GB in size.³

The PLT is extended as shown in figure 5.3 with the assumption that the GOT address is in `%r15`⁴.

²This document is currently available via <http://www.fsfla.org/~lxoliva/writeups/TLS/RFC-TLSDESC-x86.txt>

³If it is determined that the base of the PLT is within 2GB of the top of the GOT, it is also allowed to use the same PLT layout for a large code model object as that of the small and medium code models.

⁴See Function Prologue.

Figure 5.3: Final Large Code Model PLT

```
.PLT0:  pushq    8(%r15)           # GOT[1]
        jmpq    *16(%r15)      # GOT[2]
        nopl    0x0(%rax,%rax,1)
.PLT1:  movabs  $name1@GOT,%r11 # 16 bytes from .PLT0
        jmp     *(%r11,%r15)
.PLT1a: pushq    $index1        # "call" dynamic linker
        jmp     .PLT0
.PLT2:  ...                    # 21 bytes from .PLT1
.PLTx:  movabs  $namex@GOT,%r11 # 102261125th entry
        jmp     *(%r11,%r15)
.PLTxa: pushq    $indexx
        pushq   8(%r15)         # repeat .PLT0 code
        jmpq    *16(%r15)
.PLTy:  ...                    # 27 bytes from .PLTx
```

This way, for the first 102261125 entries, each PLT entry besides `.PLT0` uses only 21 bytes. Afterwards, the PLT entry code changes by repeating that of `.PLT0`, when each PLT entry is 27 bytes long. Notice that any alignment consideration is dropped in order to keep the PLT size down.

Each extended PLT entry is thus 5 to 11 bytes larger than the small and medium code model PLT entries.

The functionality of entry `.PLT0` remains unchanged from the small and medium code models.

Note that the **symbol index is still limited to 32 bits**, which would allow for up to 4G global and external functions.

Typically, UNIX compilers support **two types of PLT**, generally through the options **`-fpic` and `-fPIC`**. When building position-independent objects using the large code model, only `-fPIC` is allowed. Using the option `-fpic` with the large code model remains reserved for future use.

5.2.1 Program Interpreter

The valid program interpreter for programs conforming to the AMD64 ABI is listed in Table 5.4, which also contains the program interpreter used by Linux.

Figure 5.4: AMD64 Program Interpreter

Data Model	Path	Linux Path
LP64	/lib/ld64.so.1	/lib64/ld-linux-x86-64.so.2
ILP32	/lib/ldx32.so.1	/libx32/ld-linux-x32.so.2

5.2.2 Initialization and Termination Functions

The implementation is responsible for executing the initialization functions specified by `DT_INIT`, `DT_INIT_ARRAY`, and `DT_PREINIT_ARRAY` entries in the executable file and shared object files for a process, and the termination (or finalization) functions specified by `DT_FINI` and `DT_FINI_ARRAY`, as specified by the *System V ABI*. The user program plays no further part in executing the initialization and termination functions specified by these dynamic tags.

AI generated

AND (Strict Security Enforcement): if even one file is unverified or lacks the feature, it is forced OFF for the entire program to prevent crashes

OR (Permissive Feature Tracking): missing or legacy files are ignored; the feature turns ON if any single file uses it

OR-AND (Verified Requirements Tracking): confirms 100% of files were inspected, and turns the feature ON if at least one file requires it

5.3 Program Property

The following processor-specific program property type ranges ⁵ are defined:

Table 5.3: Program Property Type Ranges

Name	Value
<code>GNU_PROPERTY_X86_UINT32_AND_LO</code>	<code>0xc0000002</code>
<code>GNU_PROPERTY_X86_UINT32_AND_HI</code>	<code>0xc0007fff</code>
<code>GNU_PROPERTY_X86_UINT32_OR_LO</code>	<code>0xc0008000</code>
<code>GNU_PROPERTY_X86_UINT32_OR_HI</code>	<code>0xc000ffff</code>
<code>GNU_PROPERTY_X86_UINT32_OR_AND_LO</code>	<code>0xc0010000</code>
<code>GNU_PROPERTY_X86_UINT32_OR_AND_HI</code>	<code>0xc0017fff</code>

The `pr_data` field of each property contains a 4-byte unsigned integer.

AI rewrite: A feature will only be turned on in the final executable if that feature is turned on in every single .o file used to build it

`GNU_PROPERTY_X86_UINT32_AND_LO..GNU_PROPERTY_X86_UINT32_AND_HI`

A bit in the output `pr_data` field is set only if it is set in all relocatable input `pr_data`

⁵See **Linux Extensions to gABI** at <https://gitlab.com/x86-psABIs/Linux-ABI>

fields. If all bits in the the output `pr_data` field are zero, this property should be removed from output. If the bit is 1, all input relocatables have the feature. If the bit is 0 or the property is missing, it is unknown whether all input relocatables have the feature.

GNU_PROPERTY_X86_UINT32_OR_LO..GNU_PROPERTY_X86_UINT32_OR_HI

A bit in the output `pr_data` field is set if it is set in any relocatable input `pr_data` fields. If all bits in the the output `pr_data` field are zero, this property should be removed from output. If the bit is 1, some input relocatables have the feature. If the bit is 0 or the property is missing, it is unknown whether any input relocatables have the feature.

GNU_PROPERTY_X86_UINT32_OR_AND_LO..GNU_PROPERTY_X86_UINT32_OR_AND_HI

A bit in the output `pr_data` field is set if it is set in any relocatable input `pr_data` fields and this property is present in all relocatable input files. A missing property implies that its bits have unknown values. When all bits in the the output `pr_data` field are zero, this property should not be removed from output to indicate it has zero in all bits. If the property is in output, all input relocatables have the property. If the bit is 1, some input relocatables have the feature. If the bit is 0, none of input relocatables have the feature.

The following property types are defined:

Table 5.4: Program Property Types

Name	Value
GNU_PROPERTY_X86_FEATURE_1_AND	GNU_PROPERTY_X86_UINT32_AND_LO + 0
GNU_PROPERTY_X86_FEATURE_2_USED	GNU_PROPERTY_X86_UINT32_OR_AND_LO + 1
GNU_PROPERTY_X86_FEATURE_2_NEEDED	GNU_PROPERTY_X86_UINT32_OR_LO + 1
GNU_PROPERTY_X86_ISA_1_USED	GNU_PROPERTY_X86_UINT32_OR_AND_LO + 2
GNU_PROPERTY_X86_ISA_1_NEEDED	GNU_PROPERTY_X86_UINT32_OR_LO + 2

GNU_PROPERTY_X86_FEATURE_1_AND The x86 processor features indicated by the corresponding bits are used in program.

GNU_PROPERTY_X86_FEATURE_2_USED The x86 processor features indicated by the corresponding bits are used in program. Their support in the hardware is optional. Its absence in an x86 ELF binary implies that any x86 processor features may be used. `GNU_PROPERTY_X86_FEATURE_2_USED` can be used to check for features used in the program.

GNU_PROPERTY_X86_FEATURE_2_NEEDED The x86 processor features indicated by the corresponding bits are needed in program and they must be supported by the hardware. Loader may refuse to load the program whose `GNU_PROPERTY_X86_FEATURE_2_NEEDED` features aren't supported by the hardware.

GNU_PROPERTY_X86_ISA_1_USED The x86 instruction sets indicated by the corresponding bits are used in program. Their support in the hardware is optional. `GNU_PROPERTY_X86_ISA_1_USED` can be used to check for ISAs used in the program.

GNU_PROPERTY_X86_ISA_1_NEEDED The x86 instruction sets indicated by the corresponding bits are needed in program and they must be supported by the hardware. Loader may refuse to load the program whose `GNU_PROPERTY_X86_ISA_1_NEEDED` ISAs aren't supported by the hardware.

The following bits are defined for `GNU_PROPERTY_X86_FEATURE_1_AND`:

Table 5.5: `GNU_PROPERTY_X86_FEATURE_1_AND` Bit Flags

Name	Value
<code>GNU_PROPERTY_X86_FEATURE_1_IBT</code>	<code>1U << 0</code>
<code>GNU_PROPERTY_X86_FEATURE_1_SHSTK</code>	<code>1U << 1</code>

GNU_PROPERTY_X86_FEATURE_1_IBT This indicates that all executable sections are compatible with IBT (see Section 12.1.1) when `endbr64` instruction starts each valid target where an indirect branch instruction can land.

GNU_PROPERTY_X86_FEATURE_1_SHSTK This indicates that all executable sections are compatible with SHSTK (see Section 12.1.2) where return address popped from shadow stack always matches return address popped from normal stack.

The following bits are defined for `GNU_PROPERTY_X86_FEATURE_2_USED` and `GNU_PROPERTY_X86_FEATURE_2_NEEDED`:

Table 5.6: Bit Flags For X86 Processor Features

Name	Value
GNU_PROPERTY_X86_FEATURE_2_X86 [†]	1U « 0
GNU_PROPERTY_X86_FEATURE_2_X87	1U « 1
GNU_PROPERTY_X86_FEATURE_2_MMX	1U « 2
GNU_PROPERTY_X86_FEATURE_2_XMM	1U « 3
GNU_PROPERTY_X86_FEATURE_2_YMM	1U « 4
GNU_PROPERTY_X86_FEATURE_2_ZMM	1U « 5
GNU_PROPERTY_X86_FEATURE_2_FXSR	1U « 6
GNU_PROPERTY_X86_FEATURE_2_XSAVE	1U « 7
GNU_PROPERTY_X86_FEATURE_2_XSAVEOPT	1U « 8
GNU_PROPERTY_X86_FEATURE_2_XSAVEC	1U « 9
GNU_PROPERTY_X86_FEATURE_2_TMM	1U « 10
GNU_PROPERTY_X86_FEATURE_2_MASK	1U « 11

[†] This bit should always be set when GNU_PROPERTY_X86_FEATURE_2_USED is used.

GNU_PROPERTY_X86_FEATURE_2_X86 This indicates that basic x86 processor features are used.

GNU_PROPERTY_X86_FEATURE_2_X87 This indicates that x87 FPU is used.

GNU_PROPERTY_X86_FEATURE_2_MMX This indicates that MMX register feature is used.

GNU_PROPERTY_X86_FEATURE_2_XMM This indicates that XMM register feature is used.

GNU_PROPERTY_X86_FEATURE_2_YMM This indicates that YMM register feature is used.

GNU_PROPERTY_X86_FEATURE_2_ZMM This indicates that ZMM register feature is used.

GNU_PROPERTY_X86_FEATURE_2_FXSR This indicates that FXSR feature is used.

GNU_PROPERTY_X86_FEATURE_2_XSAVEOPT This indicates that XSAVEOPT feature is used.

GNU_PROPERTY_X86_FEATURE_2_XSAVEC This indicates that XSAVEC feature is used.

GNU_PROPERTY_X86_FEATURE_2_TMM This indicates that TMM register feature is used.

GNU_PROPERTY_X86_FEATURE_2_MASK This indicates that mask register feature is used.

The following bits are defined for `GNU_PROPERTY_X86_ISA_1_USED` and `GNU_PROPERTY_X86_ISA_1_NEEDED` based on micro-architecture levels in table 3.1:

Table 5.7: Bit Flags For X86 Instruction Sets

Name	Value
<code>GNU_PROPERTY_X86_ISA_1_BASELINE</code>	<code>1U << 0</code>
<code>GNU_PROPERTY_X86_ISA_1_V2</code>	<code>1U << 1</code>
<code>GNU_PROPERTY_X86_ISA_1_V3</code>	<code>1U << 2</code>
<code>GNU_PROPERTY_X86_ISA_1_V4</code>	<code>1U << 3</code>

GNU_PROPERTY_X86_ISA_1_BASELINE This indicates that `baseline` micro-architecture level must be supported by the hardware (see table 3.1).

GNU_PROPERTY_X86_ISA_1_V2 This indicates that micro-architecture level `x86-64-v2` must be supported by the hardware (see table 3.1).

GNU_PROPERTY_X86_ISA_1_V3 This indicates that micro-architecture level `x86-64-v3` must be supported by the hardware (see table 3.1).

GNU_PROPERTY_X86_ISA_1_V4 This indicates that micro-architecture level `x86-64-v4` must be supported by the hardware (see table 3.1).

Chapter 6

Libraries

A further review of the Intel386 ABI is needed.

6.1 C Library

6.1.1 Global Data Symbols

The symbols `_fp_hw`, `__flt_rounds` and `__huge_val` are not provided by the AMD64 ABI.

6.1.2 Floating Point Environment Functions

ISO C 99 defines the floating point environment functions from `<fenv.h>`. Since AMD64 has two floating point units with separate control words, the programming environment has to keep the control values in sync. On the other hand this means that routines accessing the control words only need to access one unit, and the SSE unit is the unit that should be accessed in these cases. The function `fegetround` therefore only needs to report the rounding value of the SSE unit and can ignore the x87 unit.

6.2 Unwind Library Interface

This section defines the Unwind Library interface¹, expected to be provided by any AMD64 psABI-compliant system. This is the interface on which the C++ ABI exception-handling facilities are built. We assume as a basis the Call Frame Information tables described in the DWARF Debugging Information Format document.

¹The overall structure and the external interface is derived from the IA-64 UNIX System V ABI

This section is meant to specify a language-independent interface that can be used to provide higher level exception-handling facilities such as those defined by C++.

The unwind library interface consists of at least the following routines:

```
_Unwind_RaiseException ,  
_Unwind_Resume ,  
_Unwind_DeleteException ,  
_Unwind_GetGR ,  
_Unwind_SetGR ,  
_Unwind_GetIP ,  
_Unwind_GetIPInfo ,  
_Unwind_SetIP ,  
_Unwind_GetRegionStart ,  
_Unwind_GetLanguageSpecificData ,  
_Unwind_ForcedUnwind ,  
_Unwind_GetCFA
```

In addition, two data types are defined (`_Unwind_Context` and `_Unwind_Exception`) to interface a calling runtime (such as the C++ runtime) and the above routine. All routines and interfaces behave as if defined `extern "C"`. In particular, the names are not mangled. All names defined as part of this interface have a `"_Unwind_"` prefix.

Lastly, a language and vendor specific personality routine will be stored by the compiler in the unwind descriptor for the stack frames requiring exception processing. The personality routine is called by the unwinder to handle language-specific tasks such as identifying the frame handling a particular exception.

6.2.1 Exception Handler Framework

Reasons for Unwinding

There are two major reasons for unwinding the stack:

- exceptions, as defined by languages that support them (such as C++)
- “forced” unwinding (such as caused by `longjmp` or thread termination)

The interface described here tries to keep both similar. There is a major difference, however.

- In the case where an exception is thrown, the stack is unwound while the exception propagates, but it is expected that the personality routine for each stack frame knows

whether it wants to catch the exception or pass it through. This choice is thus delegated to the personality routine, which is expected to act properly for any type of exception, whether “native” or “foreign”. Some guidelines for “acting properly” are given below.

- During “forced unwinding”, on the other hand, an external agent is driving the unwinding. For instance, this can be the `longjmp` routine. This external agent, not each personality routine, knows when to stop unwinding. The fact that a personality routine is not given a choice about whether unwinding will proceed is indicated by the `_UA_FORCE_UNWIND` flag.

To accommodate these differences, two different routines are proposed. `_Unwind_RaiseException` performs exception-style unwinding, under control of the personality routines. `_Unwind_ForcedUnwind`, on the other hand, performs unwinding, but gives an external agent the opportunity to intercept calls to the personality routine. This is done using a proxy personality routine, that intercepts calls to the personality routine, letting the external agent override the defaults of the stack frame’s personality routine.

As a consequence, it is **not necessary for each personality routine to know about any of the possible external agents** that may cause an unwind. For instance, the C++ personality routine need deal only with C++ exceptions (and possibly disguising foreign exceptions), but it does not need to know anything specific about unwinding done on behalf of `longjmp` or pthreads cancellation.

The Unwind Process

The standard ABI exception handling/unwind process begins with the raising of an exception, in one of the forms mentioned above. This call specifies an exception object and an exception class.

The runtime framework then starts a two-phase process:

- In the *search* phase, the framework repeatedly calls the personality routine, with the `_UA_SEARCH_PHASE` flag as described below, first for the current `%rip` and register state, and then unwinding a frame to a new `%rip` at each step, until the personality routine reports either success (a handler found in the queried frame) or failure (no handler) in all frames. It does not actually restore the unwound state, and the personality routine must access the state through the API.
- If the search phase reports a failure, e.g. because no handler was found, it will call `terminate()` rather than commence phase 2.

If the search phase reports success, the framework restarts in the *cleanup* phase. Again, it repeatedly calls the personality routine, with the `_UA_CLEANUP_PHASE` flag as described below, first for the current `%rip` and register state, and then unwinding a frame to a new `%rip` at each step, until it gets to the frame with an identified handler. At that point, it restores the register state, and control is transferred to the user landing pad code.

Each of these two phases uses both the unwind library and the personality routines, since the validity of a given handler and the mechanism for transferring control to it are language-dependent, but the method of locating and restoring previous stack frames is language-independent.

A two-phase exception-handling model is not strictly necessary to implement C++ language semantics, but it does provide some benefits. For example, the first phase allows an exception-handling mechanism to *dismiss* an exception before stack unwinding begins, which allows *presumptive* exception handling (correcting the exceptional condition and resuming execution at the point where it was raised). While C++ does not support presumptive exception handling, other languages do, and the two-phase model allows C++ to coexist with those languages on the stack.

Note that even with a two-phase model, we may execute each of the two phases more than once for a single exception, as if the exception was being thrown more than once. For instance, since it is not possible to determine if a given catch clause will re-throw or not without executing it, the exception propagation effectively stops at each catch clause, and if it needs to restart, restarts at phase 1. This process is not needed for destructors (cleanup code), so the phase 1 can safely process all destructor-only frames at once and stop at the next enclosing catch clause.

For example, if the first two frames unwound contain only cleanup code, and the third frame contains a C++ catch clause, the personality routine in phase 1, does not indicate that it found a handler for the first two frames. It must do so for the third frame, because it is unknown how the exception will propagate out of this third frame, e.g. by re-throwing the exception or throwing a new one in C++.

The API specified by the AMD64 psABI for implementing this framework is described in the following sections.

6.2.2 Data Structures

Reason Codes

The unwind interface uses reason codes in several contexts to identify the reasons for failures or other actions, defined as follows:

```
typedef enum {
    _URC_NO_REASON = 0,
    _URC_FOREIGN_EXCEPTION_CAUGHT = 1,
    _URC_FATAL_PHASE2_ERROR = 2,
    _URC_FATAL_PHASE1_ERROR = 3,
    _URC_NORMAL_STOP = 4,
    _URC_END_OF_STACK = 5,
    _URC_HANDLER_FOUND = 6,
    _URC_INSTALL_CONTEXT = 7,
    _URC_CONTINUE_UNWIND = 8
} _Unwind_Reason_Code;
```

The interpretations of these codes are described below.

Exception Header

The unwind interface uses a pointer to an exception header object as its representation of an exception being thrown. In general, the full representation of an exception object is language- and implementation-specific, but is prefixed by a header understood by the unwind interface, defined as follows:

```
typedef void (*_Unwind_Exception_Cleanup_Fn)
    (_Unwind_Reason_Code reason,
     struct _Unwind_Exception *exc);
struct _Unwind_Exception {
    uint64_t exception_class;
    _Unwind_Exception_Cleanup_Fn exception_cleanup;
    uint64_t private_1;
    uint64_t private_2;
};
```

An `_Unwind_Exception` object must be eightbyte aligned. The first two fields are set by user code prior to raising the exception, and the latter two should never be touched except by the runtime.

The `exception_class` field is a language- and implementation-specific identifier of the kind of exception. It allows a personality routine to distinguish between native and foreign exceptions, for example. By convention, the high 4 bytes indicate the vendor (for instance AMD\0), and the low 4 bytes indicate the language. For the C++ ABI described in this document, the low four bytes are C++\0.

The `exception_cleanup` routine is called whenever an exception object needs to be destroyed by a different runtime than the runtime which created the exception object, for instance if a Java exception is caught by a C++ catch handler. In such a case, a reason code (see above) indicates why the exception object needs to be deleted:

`_URC_FOREIGN_EXCEPTION_CAUGHT = 1` This indicates that a different runtime caught this exception. Nested foreign exceptions, or re-throwing a foreign exception, result in undefined behavior.

`_URC_FATAL_PHASE1_ERROR = 3` The personality routine encountered an error during phase 1, other than the specific error codes defined.

`_URC_FATAL_PHASE2_ERROR = 2` The personality routine encountered an error during phase 2, for instance a stack corruption.

Normally, all errors should be reported during phase 1 by returning from `_Unwind_RaiseException`. However, landing pad code could cause stack corruption between phase 1 and phase 2. For a C++ exception, the runtime should call `terminate()` in that case.

The private unwinder state (`private_1` and `private_2`) in an exception object should be neither read by nor written to by personality routines or other parts of the language-specific runtime. It is used by the specific implementation of the unwinder on the host to store internal information, for instance to remember the final handler frame between unwinding phases.

In addition to the above information, a typical runtime such as the C++ runtime will add language-specific information used to process the exception. This is expected to be a contiguous area of memory after the `_Unwind_Exception` object, but this is not required as long as the matching personality routines know how to deal with it, and the `exception_cleanup` routine de-allocates it properly.

Unwind Context

The `_Unwind_Context` type is an opaque type used to refer to a system-specific data structure used by the system unwinder. This context is created and destroyed by the system, and passed to the personality routine during unwinding.

```
struct _Unwind_Context
```

6.2.3 Throwing an Exception

`_Unwind_RaiseException`

```
_Unwind_Reason_Code _Unwind_RaiseException  
( struct _Unwind_Exception *exception_object );
```

Raise an exception, passing along the given exception object, which should have its `exception_class` and `exception_cleanup` fields set. The exception object has been allocated by the language-specific runtime, and has a language-specific format, except that it must contain an `_Unwind_Exception` struct (see Exception Header above). `_Unwind_RaiseException` does not return, unless an error condition is found (such as no

handler for the exception, bad stack format, etc.). In such a case, an `_Unwind_Reason_Code` value is returned.

Possibilities are:

`_URC_END_OF_STACK` The unwinder encountered the end of the stack during phase 1, without finding a handler. The unwind runtime will not have modified the stack. The C++ runtime will normally call `uncaught_exception()` in this case.

`_URC_FATAL_PHASE1_ERROR` The unwinder encountered an unexpected error during phase 1, e.g. stack corruption. The unwind runtime will not have modified the stack. The C++ runtime will normally call `terminate()` in this case.

If the unwinder encounters an unexpected error during phase 2, it should return `_URC_FATAL_PHASE2_ERROR` to its caller. In C++, this will usually be `__cxa_throw`, which will call `terminate()`.

The unwind runtime will likely have modified the stack (e.g. popped frames from it) or register context, or landing pad code may have corrupted them. As a result, the caller of `_Unwind_RaiseException` can make no assumptions about the state of its stack or registers.

`_Unwind_ForcedUnwind`

```
typedef _Unwind_Reason_Code (*_Unwind_Stop_Fn)
(int version,
 _Unwind_Action actions,
 uint64 exceptionClass,
 struct _Unwind_Exception *exceptionObject,
 struct _Unwind_Context *context,
 void *stop_parameter );
_Unwind_Reason_Code _Unwind_ForcedUnwind
( struct _Unwind_Exception *exception_object,
  _Unwind_Stop_Fn stop,
  void *stop_parameter );
```

Raise an exception for forced unwinding, passing along the given exception object, which should have its `exception_class` and `exception_cleanup` fields set. The exception object has been allocated by the language-specific runtime, and has a language-specific format, except that it must contain an `_Unwind_Exception` struct (see Exception Header above).

Forced unwinding is a single-phase process (phase 2 of the normal exception-handling process). The `stop` and `stop_parameter` parameters control the termination of the unwind process, instead of the usual personality routine query. The `stop` function parameter is called for each unwind frame, with the parameters described for the usual personality routine below, plus an additional `stop_parameter`.

When the `stop` function identifies the destination frame, it transfers control (according to its own, unspecified, conventions) to the user code as appropriate without returning, normally after calling `_Unwind_DeleteException`. If not, it should return an `_Unwind_Reason_Code` value as follows:

`_URC_NO_REASON` This is not the destination frame. The unwind runtime will call the frame's personality routine with the `_UA_FORCE_UNWIND` and `_UA_CLEANUP_PHASE` flags set in actions, and then unwind to the next frame and call the stop function again.

`_URC_END_OF_STACK` In order to allow `_Unwind_ForcedUnwind` to perform special processing when it reaches the end of the stack, the unwind runtime will call it after the last frame is rejected, with a `NULL` stack pointer in the context, and the stop function must catch this condition (i.e. by noticing the `NULL` stack pointer). It may return this reason code if it cannot handle end-of-stack.

`_URC_FATAL_PHASE2_ERROR` The stop function may return this code for other fatal conditions, e.g. stack corruption.

If the stop function returns any reason code other than `_URC_NO_REASON`, the stack state is indeterminate from the point of view of the caller of `_Unwind_ForcedUnwind`. Rather than attempt to return, therefore, the unwind library should return `_URC_FATAL_PHASE2_ERROR` to its caller.

Example: `longjmp_unwind()`

The expected implementation of `longjmp_unwind()` is as follows. The `setjmp()` routine will have saved the state to be restored in its customary place, including the frame pointer. The `longjmp_unwind()` routine will call `_Unwind_ForcedUnwind` with a stop function that compares the frame pointer in the context record with the saved frame pointer. If equal, it will restore the `setjmp()` state as customary, and otherwise it will return `_URC_NO_REASON` or `_URC_END_OF_STACK`.

If a future requirement for two-phase forced unwinding were identified, an alternate routine could be defined to request it, and an actions parameter flag defined to support it.

`_Unwind_Resume`

```
void _Unwind_Resume
(struct _Unwind_Exception *exception_object);
```

Resume propagation of an existing exception e.g. after executing cleanup code in a partially unwound stack. A call to this routine is inserted at the end of a landing pad that

performed cleanup, but did not resume normal execution. It causes unwinding to proceed further.

`_Unwind_Resume` should not be used to implement re-throwing. To the unwinding runtime, the catch code that re-throws was a handler, and the previous unwinding session was terminated before entering it. Re-throwing is implemented by calling `_Unwind_RaiseException` again with the same exception object.

This is the only routine in the unwind library which is expected to be called directly by generated code: it will be called at the end of a landing pad in a "landing-pad" model.

6.2.4 Exception Object Management

`_Unwind_DeleteException`

```
void _Unwind_DeleteException
(struct _Unwind_Exception *exception_object);
```

Deletes the given exception object. If a given runtime resumes normal execution after catching a foreign exception, it will not know how to delete that exception. Such an exception will be deleted by calling `_Unwind_DeleteException`. This is a convenience function that calls the function pointed to by the `exception_cleanup` field of the exception header.

6.2.5 Context Management

These functions are used for communicating information about the unwind context (i.e. the unwind descriptors and the user register state) between the unwind library and the personality routine and landing pad. They include routines to read or set the context record images of registers in the stack frame corresponding to a given unwind context, and to identify the location of the current unwind descriptors and unwind frame.

`_Unwind_GetGR`

```
uint64 _Unwind_GetGR
(struct _Unwind_Context *context, int index);
```

This function returns the 64-bit value of the given general register. The register is identified by its index as given in figure 3.36.

During the two phases of unwinding, no registers have a guaranteed value.

`_Unwind_SetGR`

```
void _Unwind_SetGR
(struct _Unwind_Context *context,
 int index,
 uint64 new_value);
```

This function sets the 64-bit value of the given register, identified by its index as for `_Unwind_GetGR`.

The behavior is guaranteed only if the function is called during phase 2 of unwinding, and applied to an unwind context representing a handler frame, for which the personality routine will return `_URC_INSTALL_CONTEXT`. In that case, only registers `%rdi`, `%rsi`, `%rdx`, `%rcx` should be used. These scratch registers are reserved for passing arguments between the personality routine and the landing pads.

`_Unwind_GetIP`

```
uint64 _Unwind_GetIP
(struct _Unwind_Context *context);
```

This function returns the 64-bit value of the instruction pointer (IP).

During unwinding, the value is guaranteed to be the address of the instruction immediately following the call site in the function identified by the unwind context. This value may be outside of the procedure fragment for a function call that is known to not return (such as `_Unwind_Resume`).

Applications which unwind through asynchronous signals and other non-call locations should use `_Unwind_GetIPInfo` below, and the additional flag that function provides.

`_Unwind_GetIPInfo`

```
uint64 _Unwind_GetIPInfo
(struct _Unwind_Context *context, int *ip_before_insn);
```

This function returns the same value as `_Unwind_GetIP`. In addition, the argument `ip_before_insn` must not be null, and `*ip_before_insn` is updated with a flag which indicates whether the returned pointer is at or after the first not yet fully executed instruction.

If `*ip_before_insn` is false, the application calling `_Unwind_GetIPInfo` should assume that the instruction pointer provided points after a call instruction which has not yet returned. In general, this means that the application should use the preceding call instruction as the instruction pointer location of the unwind context. Typically, this can be approximated by subtracting one from the returned instruction pointer.

If `*ip_before_insn` is true, then the instruction pointer does not refer to an active call site. Usually, this means that the instruction pointer refers to the point at which an asynchronous signal arrived. In this case, the application should use the instruction pointer returned from `_Unwind_GetIPInfo` as the instruction pointer location of the unwind context, without adjustment.

`_Unwind_SetIP`

```
void _Unwind_SetIP
(struct _Unwind_Context *context,
 uint64 new_value);
```

This function sets the value of the instruction pointer (IP) for the routine identified by the unwind context.

The behavior is guaranteed only when this function is called for an unwind context representing a handler frame, for which the personality routine will return `_URC_INSTALL_CONTEXT`. In this case, control will be transferred to the given address, which should be the address of a landing pad.

`_Unwind_GetLanguageSpecificData`

```
uint64 _Unwind_GetLanguageSpecificData
(struct _Unwind_Context *context);
```

This routine returns the address of the language-specific data area for the current stack frame.

This routine is not strictly required: it could be accessed through `_Unwind_GetIP` using the documented format of the DWARF Call Frame Information Tables, but since this work has been done for finding the personality routine in the first place, it makes sense to cache the result in the context. We could also pass it as an argument to the personality routine.

`_Unwind_GetRegionStart`

```
uint64 _Unwind_GetRegionStart
(struct _Unwind_Context *context);
```

This routine returns the address of the beginning of the procedure or code fragment described by the current unwind descriptor block.

This information is required to access any data stored relative to the beginning of the procedure fragment. For instance, a call site table might be stored relative to the beginning of the procedure fragment that contains the calls. During unwinding, the function returns the start of the procedure fragment containing the call site in the current stack frame.

`_Unwind_GetCFA`

```
uint64 _Unwind_GetCFA
(struct _Unwind_Context *context);
```

This function returns the 64-bit Canonical Frame Address which is defined as the value of `%rsp` at the call site in the previous frame. This value is guaranteed to be correct any time the context has been passed to a personality routine or a stop function.

6.2.6 Personality Routine

```
_Unwind_Reason_Code (*__personality_routine)
(int version,
 _Unwind_Action actions,
 uint64 exceptionClass,
 struct _Unwind_Exception *exceptionObject,
 struct _Unwind_Context *context);
```

The personality routine is the function in the C++ (or other language) runtime library which serves as an interface between the system unwind library and language-specific exception handling semantics. It is specific to the code fragment described by an unwind info block, and it is always referenced via the pointer in the unwind info block, and hence it has no psABI-specified name.

Parameters

The personality routine parameters are as follows:

version Version number of the unwinding runtime, used to detect a mis-match between the unwinder conventions and the personality routine, or to provide backward compatibility. For the conventions described in this document, version will be 1.

actions Indicates what processing the personality routine is expected to perform, as a bit mask. The possible actions are described below.

exceptionClass An 8-byte identifier specifying the type of the thrown exception. By convention, the high 4 bytes indicate the vendor (for instance AMD\0), and the low 4 bytes indicate the language. For the C++ ABI described in this document, the low four bytes are C++\0. This is not a null-terminated string. Some implementations may use no null bytes.

exceptionObject The pointer to a memory location recording the necessary information for processing the exception according to the semantics of a given language (see the Exception Header section above).

context Unwinder state information for use by the personality routine. This is an opaque handle used by the personality routine in particular to access the frame's registers (see the Unwind Context section above).

return value The return value from the personality routine indicates how further unwind should happen, as well as possible error conditions. See the following section.

Personality Routine Actions

The actions argument to the personality routine is a bitwise OR of one or more of the following constants:

```
typedef int _Unwind_Action;
const _Unwind_Action _UA_SEARCH_PHASE = 1;
const _Unwind_Action _UA_CLEANUP_PHASE = 2;
const _Unwind_Action _UA_HANDLER_FRAME = 4;
const _Unwind_Action _UA_FORCE_UNWIND = 8;
```

_UA_SEARCH_PHASE Indicates that the personality routine should check if the current frame contains a handler, and if so return `_URC_HANDLER_FOUND`, or otherwise return `_URC_CONTINUE_UNWIND`. `_UA_SEARCH_PHASE` cannot be set at the same time as `_UA_CLEANUP_PHASE`.

_UA_CLEANUP_PHASE Indicates that the personality routine should perform cleanup for the current frame. The personality routine can perform this cleanup itself, by calling nested procedures, and return `_URC_CONTINUE_UNWIND`. Alternatively, it can setup the registers (including the IP) for transferring control to a "landing pad", and return `_URC_INSTALL_CONTEXT`.

_UA_HANDLER_FRAME During phase 2, indicates to the personality routine that the current frame is the one which was flagged as the handler frame during phase 1. The personality routine is not allowed to change its mind between phase 1 and phase 2, i.e. it must handle the exception in this frame in phase 2.

_UA_FORCE_UNWIND During phase 2, indicates that no language is allowed to "catch" the exception. This flag is set while unwinding the stack for `longjmp` or during thread cancellation. User-defined code in a catch clause may still be executed, but the catch clause must resume unwinding with a call to `_Unwind_Resume` when finished.

Transferring Control to a Landing Pad

If the personality routine determines that it should transfer control to a landing pad (in phase 2), it may set up registers (including IP) with suitable values for entering the landing pad (e.g. with landing pad parameters), by calling the context management routines above. It then returns `_URC_INSTALL_CONTEXT`.

Prior to executing code in the landing pad, the unwind library restores registers not altered by the personality routine, using the context record, to their state in that frame

before the call that threw the exception, as follows. All registers specified as callee-saved by the base ABI are restored, as well as scratch registers `%rdi`, `%rsi`, `%rdx`, `%rcx` (see below). Except for those exceptions, scratch (or caller-saved) registers are not preserved, and their contents are undefined on transfer.

The landing pad can either resume normal execution (as, for instance, at the end of a C++ catch), or resume unwinding by calling `_Unwind_Resume` and passing it the `exceptionObject` argument received by the personality routine. `_Unwind_Resume` will never return.

`_Unwind_Resume` should be called if and only if the personality routine did not return `_Unwind_HANDLER_FOUND` during phase 1. As a result, the unwinder can allocate resources (for instance memory) and keep track of them in the exception object reserved words. It should then free these resources before transferring control to the last (handler) landing pad. It does not need to free the resources before entering non-handler landing-pads, since `_Unwind_Resume` will ultimately be called.

The landing pad may receive arguments from the runtime, typically passed in registers set using `_Unwind_SetGR` by the personality routine. For a landing pad that can call to `_Unwind_Resume`, one argument must be the `exceptionObject` pointer, which must be preserved to be passed to `_Unwind_Resume`.

The landing pad may receive other arguments, for instance a switch value indicating the type of the exception. Four scratch registers are reserved for this use (`%rdi`, `%rsi`, `%rdx`, `%rcx`)².

Rules for Correct Inter-Language Operation

The following rules must be observed for correct operation between languages and/or run times from different vendors:

An exception which has an unknown class must not be altered by the personality routine. The semantics of foreign exception processing depend on the language of the stack frame being unwound. This covers in particular how exceptions from a foreign language are mapped to the native language in that frame.

If a runtime resumes normal execution, and the caught exception was created by another runtime, it should call `_Unwind_DeleteException`. This is true even if it understands the exception object format (such as would be the case between different C++ run times).

A runtime is not allowed to catch an exception if the `_UA_FORCE_UNWIND` flag was passed to the personality routine.

²GCC uses `%rax` and `%rdx` instead.

Example: Foreign Exceptions in C++. In C++, foreign exceptions can be caught by a `catch(...)` statement. They can also be caught as if they were of a `__foreign_exception` class, defined in `<exception>`. The `__foreign_exception` may have subclasses, such as `__java_exception` and `__ada_exception`, if the runtime is capable of identifying some of the foreign languages.

The behavior is **undefined** in the following cases:

- A `__foreign_exception` catch argument is accessed in any way (including taking its address).
- A `__foreign_exception` is active at the same time as another exception (either there is a nested exception while catching the foreign exception, or the foreign exception was itself nested).
- `uncaught_exception()`, `set_terminate()`, `set_unexpected()`, `terminate()`, or `unexpected()` is called at a time a foreign exception exists (for example, calling `set_terminate()` during unwinding of a foreign exception).

All these cases might involve accessing C++ specific content of the thrown exception, for instance to chain active exceptions.

Otherwise, a catch block catching a foreign exception is **allowed**:

- to resume normal execution, thereby stopping propagation of the foreign exception and deleting it, or
- to re-throw the foreign exception. In that case, the original exception object must be unaltered by the C++ runtime.

A catch-all block may be executed during forced unwinding. For instance, a `longjmp` may execute code in a `catch(...)` during stack unwinding. However, if this happens, unwinding will proceed at the end of the catch-all block, whether or not there is an explicit re-throw.

Setting the low 4 bytes of exception class to C++\0 is reserved for use by C++ run-times compatible with the common C++ ABI.

6.3 Unwinding Through Assembler Code

For successful unwinding on AMD64 every function must provide a valid debug information in the DWARF Debugging Information Format. In high level languages (e.g. C/C++,

Fortran, Ada, ...) this information is generated by the compiler itself. However for hand-written assembly routines the debug info must be provided by the author of the code. To ease this task some new assembler directives are added:

.cfi_startproc is used at the beginning of each function that should have an entry in `.eh_frame`. It initializes some internal data structures and emits architecture dependent initial CFI instructions. Each `.cfi_startproc` directive has to be closed by `.cfi_endproc`.

.cfi_endproc is used at the end of a function where it closes its unwind entry previously opened by `.cfi_startproc` and emits it to `.eh_frame`.

.cfi_def_cfa REGISTER, OFFSET defines a rule for computing CFA as: take address from REGISTER and add OFFSET to it.

.cfi_def_cfa_register REGISTER modifies a rule for computing CFA. From now on REGISTER will be used instead of the old one. The offset remains the same.

.cfi_def_cfa_offset OFFSET modifies a rule for computing CFA. The register remains the same, but OFFSET is new. Note that this is the absolute offset that will be added to a defined register to compute the CFA address.

.cfi_adjust_cfa_offset OFFSET is similar to `.cfi_def_cfa_offset` but OFFSET is a relative value that is added or subtracted from the previous offset.

.cfi_offset REGISTER, OFFSET saves the previous value of REGISTER at offset OFFSET from CFA.

.cfi_rel_offset REGISTER, OFFSET saves the previous value of REGISTER at offset OFFSET from the current CFA register. This is transformed to `.cfi_offset` using the known displacement of the CFA register from the CFA. This is often easier to use, because the number will match the code it is annotating.

.cfi_escape EXPRESSION[, ...] allows the user to add arbitrary bytes to the unwind info. One might use this to add OS-specific CFI opcodes, or generic CFI opcodes that the assembler does not support.

Figure 6.1: Examples for Unwinding in Assembler

```
# - function with local variable allocated on the stack
.type    func_locvars,@function
func_locvars:
    .cfi_startproc
    # allocate space for local vars
    sub    $0x1234, %rsp
    .cfi_adjust_cfa_offset    0x1234
    # body
    ...
    # release space of local vars and return
    add    $0x1234, %rsp
    .cfi_adjust_cfa_offset    -0x1234
    ret
    .cfi_endproc

# - function that moves frame pointer to another register
#   and then allocates space for local variables
.type    func_otherreg,@function
func_otherreg:
    .cfi_startproc
    # save frame pointer to r12
    movq    %rsp, %r12
    .cfi_def_cfa_register    r12
    # allocate space for local vars
    # (no .cfi_{def,adjust}_cfa_offset needed here,
    # because CFA is computed from r12!)
    sub    $100,%rsp
    # body
    ...
    # restore frame pointer from r12
    movq    %r12, %rsp
    .cfi_def_cfa_register    rsp
    ret
    .cfi_endproc
```

Chapter 7

Development Environment

During compilation of C or C++ code at least the symbols in table 7.1 are defined by the pre-processor ¹.

Table 7.1: Predefined Pre-Processor Symbols

<code>__amd64</code>	Defined for both LP64 and ILP32 programming models.
<code>__amd64__</code>	Defined for both LP64 and ILP32 programming models.
<code>__x86_64</code>	Defined for both LP64 and ILP32 programming models.
<code>__x86_64__</code>	Defined for both LP64 and ILP32 programming models.
<code>_LP64</code>	Defined for LP64 programming model.
<code>__LP64__</code>	Defined for LP64 programming model.
<code>_ILP32</code>	Defined for ILP32 programming model.
<code>__ILP32__</code>	Defined for ILP32 programming model.

¹ `__LP64` and `__ILP64__` were added to GCC 3.3 in March, 2003.

Chapter 8

Execution Environment

Not done yet.

Chapter 9

Conventions

¹

¹This chapter is used to document some features special to the AMD64 ABI. The different sections might be moved to another place or removed completely.

9.1 C++

For the C++ ABI we will use the IA-64 C++ ABI and instantiate it appropriately. The current draft of that ABI is available at:

<http://mentorembdedded.github.io/cxx-abi/>

9.2 Fortran

A formal Fortran ABI does not exist. Most Fortran compilers are designed for very specific high performance computing applications, so Fortran compilers use different passing conventions and memory layouts optimized for their specific purpose. For example, Fortran applications that must run on distributed memory machines need a different data representation for array descriptors (also known as dope vectors, or fat pointers) than applications running on symmetric multiprocessor shared memory machines. A normative ABI for Fortran is therefore not desirable. However, for interoperability of different Fortran compilers, as well as for interoperability with other languages, this section provides some guidelines for data types representation, and argument passing. The guidelines in this section are derived from the GNU Fortran 77 (G77) compiler, and are also followed by the GNU Fortran 95 (`gfortran`) compiler (restricted to Fortran 77 features). Other Fortran compilers already available for AMD64 at the time of this writing may use different conventions, so compatibility is not guaranteed.

When this text uses the term *Fortran procedure*, the text applies to both Fortran `FUNCTION` and `SUBROUTINE` subprograms as well as for alternate `ENTRY` points, unless specifically stated otherwise.

Everything not explicitly defined in this ABI is left to the implementation.

9.2.1 Names

External names in Fortran are names of entities visible to all subprograms at link time. This includes names of `COMMON` blocks and Fortran procedures. To avoid name space conflicts with linked-in libraries, all external names have to be mangled. And to avoid name space conflicts of mangled external names with local names, all local names must also be mangled. The mangling scheme is straightforward as follows:

- all names that do not have any underscores in it should have *one* underscore appended
- all external names containing one or more underscores in it (wherever) should have *two* underscores appended ².
- all external names should be mapped to lower case, following the traditional UNIX model for Fortran compilers

For examples see figure 9.1:

²Historically, this is to be compatible with `f2c`.

Figure 9.1: Example mapping of names

Fortran external name	Linker name
FOO	foo_
foo	foo_
Foo	foo_
foo_	foo__
f_oo	f_oo__

The entry point of the main program unit is called `MAIN_`. The symbol name for the blank common block is `__BLNK__`. the external name of the unnamed `BLOCK DATA` routine is `__BLOCK_DATA__`.

9.2.2 Representation of Fortran Types

For historical reasons, GNU Fortran 77 maps Fortran programs to the C ABI, so the data representation can be explained best by providing the mapping of Fortran types to C types used by G77 on AMD64³ as in figure 9.2. The “`TYPE*N`” notation specifies that variables or aggregate members of type `TYPE` shall occupy `N` bytes of storage.

Figure 9.2: Mapping of Fortran to C types

Fortran	Data kind	Equivalent C type
INTEGER*4	Default integer	signed int
INTEGER*8	Double precision integer	signed long
REAL*4	Single precision FP number	float
REAL*8	Double precision FP number	double
COMPLEX*4	Single precision complex FP number	complex float
COMPLEX*8	Double precision complex FP number	complex double
LOGICAL	Boolean logical type	signed int
CHARACTER	Text string	char[] + length

The values for type `LOGICAL` are `.TRUE.` implemented as 1 and `.FALSE.` implemented as 0.

³G77 provides a header `g2c.h` with the equivalent C type definitions for all supported Fortran scalar types.

Data objects with a `CHARACTER` type⁴ are represented as an array of characters of the C char type (not guaranteed to be “\0” terminated) with a separate length counter to distinguish between `CHARACTER` data objects with a length parameter, and aggregate types of `CHARACTER` data objects, possibly also with a length parameter.

Layout of other aggregate types is implementation defined. GNU Fortran puts all arrays in contiguous memory in column-major order. GNU Fortran 95 builds an equivalent C struct for derived types without reordering the type fields. Other compilers may use other representations as needed. The representation and use of Fortran 90/95 array descriptors is implementation defined. Note that array indices start at 1 by default.

Fortran 90/95 allow different kinds of each basic type using the `kind` type parameter of a type. Kind type parameter values are implementation defined.

Layout of the commonly used Cray pointers is implementation defined.

9.2.3 Argument Passing

For each given Fortran 77 procedure, an equivalent C prototype can be derived. Once this equivalent C prototype is known, the C ABI conventions should be applied to determine how arguments are passed to the Fortran procedure.

G77 passes all (user defined) formal arguments of a procedure by reference. Specifically, pointers to the location in memory of a variable, array, array element, a temporary location that holds the result of evaluating an expression or a temporary or permanent location that holds the value of a constant (xf. g77 manual) are passed as actual arguments. Artificial compiler generated arguments may be passed by value or by reference as they are inherently compiler and hence implementation specific.

Data objects with a `CHARACTER` type are passed as a pointer to the character string and its length, so that each `CHARACTER` formal argument in a Fortran procedure results in two actual arguments in the equivalent C prototype. The first argument occupies the position in the formal argument list of the Fortran procedure. This argument is a pointer to the array of characters that make up the string, passed by the caller. The second argument is appended to the end of the user-specified formal argument list. This argument is of the default integer type and its value is the length of the array of characters, that is the length, passed as the first argument. This length is passed by value. When more than one `CHARACTER` argument is present in an argument list, the length arguments are appended in the order the original arguments appear. The above discussion also applies to sub-strings.

This ABI does not define the passing of optional arguments. They are allowed only in Fortran 90/95 and their passing is implementation defined.

⁴This includes sub-strings.

This ABI does not define array functions (function returning arrays). They are allowed only in Fortran 90/95 and requires the definition of array descriptors.

Note that Fortran 90/95 procedure arguments with the `INTENT (IN)` attribute should also be passed by reference if the procedure is to be linked with code written in Fortran 77. Fortran 77 does not and can not support the `INTENT` attribute because it has no concept of explicit interfaces. It is therefore not possible to declare the callee's arguments as `INTENT (IN)`. A Fortran 77 compiler must assume that all procedure arguments are `INTENT (INOUT)` in the Fortran 90/95 sense.

9.2.4 Functions

The calling of statement functions is implementation defined (as they are defined only locally, the compiler has the freedom to apply any calling convention it likes).

Subroutines with alternate returns (e.g. "SUBROUTINE X(*,*)" called as "CALL X(*10,*20)") are implemented as functions returning an `INTEGER` of the default kind. The value of this returned integer is whatever integer is specified in the "RETURN" statement for the subroutine⁵, or 0 for a `RETURN` statement without an argument. It is up to the caller to jump to the corresponding alternate return label. The actual alternate-return arguments are omitted from the calling sequence.

An example:

```

SUBROUTINE SHOW_ALTERNATE_RETURN (N)
  INTEGER N
  CALL ALTERNATE_RETURN_EXAMPLE (N, *10, *20, *30)
  WRITE (*,*) 'OK - Normal Return'
  RETURN
10  WRITE (*,*) '1st alternate return'
    RETURN
20  WRITE (*,*) '2nd alternate return'
    RETURN
30  WRITE (*,*) '2nd alternate return'
    RETURN
END

SUBROUTINE ALTERNATE_RETURN_EXAMPLE (N, *, *, *)
  INTEGER N
  IF (N .EQ. 0 ) RETURN      ! Implicit "RETURN 0"
  IF ( N .EQ. 1 ) RETURN 1
  IF ( N .EQ. 2 ) RETURN 2
  RETURN 3
END
```

⁵ This integer indicates the position of an alternate return from the subroutine in the formal argument list

Here the `SUBROUTINE ALTERNATE_RETURN_EXAMPLE` is implemented as a function returning an `INTEGER*4` with value 0 if `N` is 0, 1 if `N` is 1, 2 if `N` is 2 and 3 for all other values of `N`. This return value is used by the caller as if the actual call were replaced by this sequence:

```
INTEGER X
X = CALL ALTERNATE_RETURN_EXAMPLE (N)
GOTO (10, 20, 30), X
```

All in all the effect is that the index of the returned to label (starting from 1) will be contained in `%rax` after the call.

Alternate `ENTRY` points of a `SUBROUTINE` or `FUNCTION` should be treated as separate sub-programs, as mandated by the Fortran standard. I.e. arguments passed to an alternate `ENTRY` should be passed as if the alternate `ENTRY` is a separate `SUBROUTINE` or `FUNCTION`. If a `FUNCTION` has alternate `ENTRY` points, the result of each of the alternate `ENTRY` points must be returned as if the alternate `ENTRY` is a separate `FUNCTION` with the result type of the alternate `ENTRY`. The external naming of alternate `ENTRY` points follows section 9.2.1.

9.2.5 COMMON blocks

In absence of any `EQUIVALENCE` declaration involving variables in `COMMON` blocks the layout of a `COMMON` block is exactly the same as the layout of the equivalent C structure (with types of variables substituted according to section 9.2.2), including the alignment requirements.

This ABI defines the layout under presence of `EQUIVALENCE` statements only in some cases:

- the layout of the `COMMON` block must not change if one ignores the `EQUIVALENCE`, which amongst other things means:
- If two arrays are equivalenced, the larger array must be named in the `COMMON` block, and there must be complete inclusion, in particular the other array may not extend the size of the equivalenced segment. It may also not change the alignment requirement.
- If an array element and a scalar are equivalenced, the array must be named in the `COMMON` block and it must not be smaller than the scalar. The type of the scalar must not require bigger alignment than the array.
- if two scalars are equivalenced they must have the same size and alignment requirements.

Other cases are implementation defined.

Because the Fortran standard allows the blank `COMMON` block to have different sizes in different subprograms, it may be impossible to determine if it is small enough to fit in the `.bss` section. When compiling for the medium or large code models the blank `COMMON` block should therefore always be put in the `.lbss` section.

9.2.6 Intrinsics

This sections lists the set of intrinsics which has to be supported at minimum by a conforming compiler. They are separated by origin. They follow regular calling and naming conventions.

The signature of intrinsics uses the syntax *return – type(argtype1, argtype2, ...)*, where the individual types can be the following characters: **V** (as in void) designates a `SUBROUTINE`, **L** a `LOGICAL`, **I** an `INTEGER`, **R** a `REAL`, and **C** a `CHARACTER`. Hence `I(R, L)` designates a `FUNCTION` returning an `INTEGER` and taking a `REAL` and a `LOGICAL`. If an argument is an array, this is indicated using a trailing number, e.g. **I13** is an `INTEGER` array with 13 elements. If a `CHARACTER` argument or return value has a fixed length, this is indicated using an asterisk and a trailing number, for example **C*16** is a `CHARACTER(len=16)`. If a `CHARACTER` argument of arbitrary length must be passed, the trailing number is replaced with `N`, for example **C*N**.

Table 9.1: Mil intrinsics

Name	Signature	Meaning
<code>BTest</code>	<code>L(I,I)</code>	Test bit
<code>IAnd</code>	<code>I(I,I)</code>	Boolean AND
<code>IOr</code>	<code>I(I,I)</code>	Boolean OR
<code>IEOr</code>	<code>I(I,I)</code>	Boolean XOR
<code>Not</code>	<code>I(I)</code>	Boolean NOT
<code>IBClr</code>	<code>I(I,I)</code>	Clear a bit
<code>IBits</code>	<code>I(I,I,I)</code>	Extract a bit subfield of a variable
<code>IBSet</code>	<code>I(I,I)</code>	Set a bit
<code>IShft</code>	<code>I(I,I)</code>	Logical bit shift
<code>IShftC</code>	<code>I(I,I,I)</code>	Circular bit shift
<code>MvBits</code>	<code>V(I,I,I,I)</code>	Move a bit field

BTest (*I*, *Pos*) Returns `.TRUE.` if bit *Pos* in *I* is set, returns `.FALSE.` otherwise.

IAnd (*I*, *J*) Returns value resulting from a boolean AND on each pair of bits in *I* and *J*.

IOr (*I*, *J*) Returns value resulting from a boolean OR on each pair of bits in *I* and *J*.

IEOr (*I*, *J*) Returns value resulting from a boolean XOR on each pair of bits in *I* and *J*.

Not (*I*) Returns value resulting from a boolean NOT on each bit in *I*.

IBClr (*I*, *Pos*) Returns the value of *I* with bit *Pos* cleared (set to zero).

IBits (*I*, *Pos*, *Len*) Extracts a subfield starting from bit position *Pos* and with a length (towards the most significant bit) of *Len* bits from *I*. The result is right-justified and the remaining bits are zeroed.

IBSet (*I*, *Pos*) Returns the value of *I* with the bit in position *Pos* set to one.

IShft (*I*, *Shift*) All bits of *I* are shifted *Shift* places. `Shift.GT.0` indicates a left shift, `Shift.EQ.0` indicates no shift, and `Shift.LT.0` indicates a right shift. Bits shifted out from the least (when shifting right) or most (when shifting left) significant position are lost. Bits shifted in at the opposite end are not set (i.e. zero).

IShftC (*I*, *Shift*, *Size*) The rightmost *Size* bits of the argument *I* are shifted circularly *Shift* places. The unshifted bits of the result are the same as the unshifted bits of *I*.

MvBits (*From*, *FromPos*, *Len*, *To*, *ToPos*) Move *Len* bits of *From* from bit positions *FromPos* through *FromPos*+*Len*-1 to bit positions *ToPos* through *ToPos*+*Len*-1 of *To*. The bit portions of *To* that are not affected by the movement of bits are unchanged.

Table 9.2: F77 intrinsics

Name	Meaning
Abs	Absolute value
ACos	Arc cosine
AInt	Truncate to whole number
ANInt	Round to nearest whole number
ASin	Arc sine
ATan	Arc Tangent
ATan2	Arc Tangent
Char	Character from code
Cmplx	Construct <code>COMPLEX (KIND=1)</code> value
Conjg	Complex conjugate
Cos	Cosine
CosH	Hyperbolic cosine
Dble	Convert to double precision
DiM	Difference magnitude (non-negative subtract)
DProd	Double-precision product
Exp	Exponential
IChar	Code for character
Index	Locate a <code>CHARACTER</code> substring
Int	Convert to <code>INTEGER</code> value truncated to whole number
Len	Length of character entity
LGe	Lexically greater than or equal
LGt	Lexically greater than
LLe	Lexically less than or equal
LLt	Lexically less than
Log	Natural logarithm
Log10	Common logarithm
Max	Maximum value
Min	Minimum value
Mod	Remainder
NInt	Convert to <code>INTEGER</code> value rounded to nearest whole number
Real	Convert value to type <code>REAL (KIND=1)</code>
Sin	Sine
SinH	Hyperbolic sine
SqRt	Square root
Tan	Tangent
TanH	Hyperbolic tangent

Refer to the Fortran 77 language standard for signature and definition of the F77 intrinsics listed in table 9.2. These intrinsics can have a prefix as per the standard hence the table is not exhaustive.

Table 9.3: F90 intrinsics

Name	Meaning
AChar	ASCII character from code
Bit_Size	Number of bits in arguments type
CPU_Time	Get current CPU time
IACChar	ASCII code for character
Len_Trim	Get last non-blank character in string
System_Clock	Get current system clock value

Refer to the Fortran 90 language standard for signature and definition of the F90 intrinsics listed in table 9.3.

Table 9.4: Math intrinsics

Name	Signature	Meaning
BesJ0	R(R)	Bessel function
BesJ1	R(R)	Bessel function
BesJN	R(I,R)	Bessel function
BesY0	R(R)	Bessel function
BesY1	R(R)	Bessel function
BesYN	R(I,R)	Bessel function
ErF	R(R)	Error function
ErFC	R(R)	Complementary error function
IRand	I(I)	Random number
Rand	R(I)	Random number
SRand	V(I)	Random seed

BesJ0 (X) Calculates the Bessel function of the first kind of order 0 of X. Returns a `REAL` of the same kind as x.

BesJ1 (X) Calculates the Bessel function of the first kind of order 1 of X. Returns a `REAL` of the same kind as `x`.

BesJN (N, X) Calculates the Bessel function of the first kind of order N of X. Returns a `REAL` of the same kind as `x`.

BesY0 (X) Calculates the Bessel function of the second kind of order 0 of X. Returns a `REAL` of the same kind as `x`.

BesY1 (X) Calculates the Bessel function of the second kind of order 1 of X. Returns a `REAL` of the same kind as `x`.

BesYN (N, X) Calculates the Bessel function of the second kind of order N of X. Returns a `REAL` of the same kind as `x`.

ErF (X) Calculates the error function of X. Returns a `REAL` of the same kind as `x`.

ErFC (X) Calculates the complementary error function of X, i.e. $1 - \text{ERF}(X)$. Returns a `REAL` of the same kind as `x`.

IRand (Flag) Flag is optional. Returns a uniform quasi-random number up to a system-dependent limit. If `Flag .EQ. 0` or `Flag` is not passed, the next number in sequence is returned. If `Flag .EQ. 1`, the generator is restarted. If `Flag` has any other value, the generator is restarted with the value of `Flag` as the new seed.

Rand (Flag) Flag is optional. Returns a uniform quasi-random number between 0 and 1. If `Flag .EQ. 0` or `Flag` is not passed, the next number in sequence is returned. If `Flag .EQ. 1`, the generator is restarted. If `Flag` has any other value, the generator is restarted with the value of `Flag` as the new seed.

SRand (Seed) Reinitializes the random number generator for `IRand` and `Rand` with the seed in `Seed`.

Table 9.5: Unix intrinsics

Name	Signature	Meaning
Abort	V()	Abort the program
Access	I(C,C)	Check file accessibility
DTime	V(R2,R)	Get elapsed time since last call
ETime	V(R2,R)	Get elapsed time for process
Flush	V(I)	Flush buffered output
FNum	I(I)	Get file descriptor from Fortran unit number
FStat	V(I,I13,I)	Get file information
GError	V(C*N)	Get error message for last error
GetArg	V(I,C*N)	Obtain command-line argument
GetCWD	V(C*N,I)	Get current working directory
GetEnv	V(C*N,C*N)	Get environment variable
GetGId	I()	Get process group ID
GetPId	I()	Get process ID
GetUId	I()	Get process user ID
GetLog	V(C*N)	Get login name
HostNm	V(C*N,I)	Get host name
IArgC	I()	Obtain count of command-line arguments
IDate	V(I3)	Get local date info
IErrNo	I()	Get error number for last error
ITime	V(I3)	Get local time of day
LStat	V(C*N,I13,I)	Get file information
PError	V(C*N)	Print error message for last error
Rename	V(C*N,C*N,I)	Rename file
Sleep	V(I)	Sleep for a specified time
System	V(C*N,I)	Invoke shell (system) command

Abort () Prints a message and potentially causes a core dump.

Access (Name, Mode) Checks file `Name` for accessibility in the mode specified by `Mode`. Returns 0 if the file is accessible in that mode, otherwise an error code. `Name` must be a NULL-terminated string of CHARACTER (i.e. a C-style string). Trailing blanks in `Name` are ignored. `Mode` must be a concatenation of any of the following characters: **r** meaning test for read permission, **w** meaning test for write permission, **x** meaning test for execute/search permission, or a space meaning test for existence of the file.

DTime (TArray, Result) When called for the first time, returns the number of seconds of runtime since the start of the program in `Result`, the user component of this runtime in `TArray(1)`, and the system time in `TArray(2)`. Subsequent invocations values based on accumulations since the previous invocation.

ETime (TArray, Result) Returns the number of seconds of runtime since the start of the program in `Result`, the user component of this runtime in `TArray(1)`, and the system time in `TArray(2)`. Subsequent invocations values based on accumulations since the previous invocation.

Flush (Unit) Flushes the Fortran I/O unit with ID `Unit`. The unit must be open for output. If the optional `Unit` argument is omitted, all open units are flushed.

FNum (Unit) Returns the UNIX(tm) file descriptor number corresponding to the Fortran I/O unit `Unit`. The unit must be open.

FStat (Unit, SArray, Status) Obtains data about the file open on Fortran I/O unit `Unit` and places it in the array `SArray`. The values in this array are as follows:

1. Device ID
2. Inode number
3. File mode
4. Number of links
5. Owner's UID
6. Owner's GID
7. ID of device containing directory entry for file
8. File size (bytes)
9. Last access time
10. Last modification time
11. Last file status change time
12. Preferred I/O block size (-1 if not available)
13. Number of blocks allocated (-1 if not available)

If an element is not available, or not relevant on the host system, it is returned as 0 except when indicated otherwise in the above list. If the optional `Status` argument is supplied, it contains 0 on success or a nonzero error code upon return.

Error (*Message*) Returns the system error message corresponding to the last system error (*errno* in C). The message is returned in *Message*. If *Message* is longer than the error message, it is padded with blanks after the message. If *Message* is not long enough to hold the error message, the error message is truncated to the length of *Message*.

GetArg (*Pos*, *Value*) Returns in *Value* the command-line argument in position *Pos*. If there are fewer than *Pos* command-line arguments, *Value* is filled with blanks. If *Pos* is 0, the name of the program is returned. If *Value* is longer than the command-line argument, it is padded with blanks after the argument. If *Value* is not long enough to hold the command-line argument, the argument is truncated to the length of *Value*.

GetCWD (*Name*, *Status*) Returns in *Name* the current working directory. If the optional *Status* argument is supplied, it contains 0 on success or a nonzero error code upon return.

GetEnv (*Name*, *Value*) Returns in *Value* the environment variable identified with *Name*. If *Name* has not been set, *Value* is filled with blanks. A null character marks the end of the name in *Name*. Trailing blanks in *Name* are ignored. If *Value* is longer than the environment variable, it is padded with blanks after the variable. If *Value* is not long enough to hold the environment variable, the variable is truncated to the length of *Value*.

GetGid () Returns the group ID for the current process.

GetPid () Returns the process ID for the current process.

GetUid () Returns the user ID for the current process.

GetLog (*Login*) Returns the login name for the process in *Login*, or a blank string if the host system does not support `getlogin(3)`. If *Login* is longer than the login name, it is padded with blanks after the login name. If *Login* is not long enough to hold the login name, the login name is truncated to the length of *Login*.

HotNm (*Name*, *Status*) Returns in *Name* system's host name. If the optional *Status* argument is supplied, it contains 0 on success or a nonzero error code upon return. If *Name* is longer than the host name, it is padded with blanks after the host name. If *Name* is not long enough to hold the host name, the host name is truncated to the length of *Name*.

IArgC () Returns the number of command-line arguments. The program name itself is not included in this number.

`IDate (TArray)` Returns the current local date day, month, year in elements 1, 2, and 3 of `TArray`, respectively. The year has four significant digits.

`IErrno ()` Returns the last system error number (`errno` in C).

`ITime (TArray)` Returns the current local time hour, minutes, and seconds in elements 1, 2, and 3 of `TArray`, respectively.

`LStat (File, SArray, Status)` Obtains data about a file named `File` and places it in the array `SArray`. The values in this array are as follows:

1. Device ID
2. Inode number
3. File mode
4. Number of links
5. Owner's UID
6. Owner's GID
7. ID of device containing directory entry for file
8. File size (bytes)
9. Last access time
10. Last modification time
11. Last file status change time
12. Preferred I/O block size (-1 if not available)
13. Number of blocks allocated (-1 if not available)

If an element is not available, or not relevant on the host system, it is returned as 0 except when indicated otherwise in the above list. If the optional `Status` argument is supplied, it contains 0 on success or a nonzero error code upon return.

`PError (MsgPrefix)` Prints a newline-terminated error message corresponding to the last system error. This is prefixed by the string `MsgPrefix`, a colon and a space. The error message is printed on the C `stderr` stream.

`Rename (Path1, Path2, Status)` Renames the file named `Path1` to `Path2`. A `null` character marks the end of the names. Trailing blanks are ignored. If the optional `Status` argument is supplied, it contains 0 on success or a nonzero error code upon return.

`Sleep (Seconds)` Causes the program to pause for `Seconds` seconds.

`System (Command, Status)` Passes the string in `Command` to a shell through `system(3)`. If the optional argument `Status` is present, it contains the value returned by `system(3)`.

Chapter 10

ILP32 Programming Model

"x32" is commonly used to refer to AMD64 ILP32 programming model.

10.1 Parameter Passing

When a value of pointer type is returned or passed in a register, bits 32 to 63 shall be zero.

10.2 Address Space

ILP32 binaries reside in the lower 32 bits of the 64-bit virtual address space and all addresses are 32 bits in size. They should conform to small code model or small position independent code model (PIC) described in Section [3.5.1](#).

10.3 Thread-Local Storage Support

ILP32 Thread-Local Storage (TLS) support is based on LP64 TLS implementation with some modifications.

10.3.1 Global Thread-Local Variable

For a global thread-local variable `x`:

```
extern __thread int x;
```

General Dynamic Model Load address of `x` into `%rax`

Table 10.1: General Dynamic Model Code Sequence

LP64			ILP32		
0x00	.byte	0x66	0x00	leaq	x@tlsgd(%rip),%rdi
0x01	leaq	x@tlsgd(%rip),%rdi	0x07	.word	0x6666
0x08	.word	0x6666	0x09	rex64	
0x0a	rex64		0x0a	call	__tls_get_addr@plt
0x0b	call	__tls_get_addr@plt			

In TLSDESC code sequence, `leal` instruction must be encoded with `rex` prefix even if it isn't required by destination register. If the `leal` encoding has a variable length, linker can't tell where it starts and can't safely perform GDesc -> IE/LE optimization.

Table 10.2: General Dynamic Model Code Sequence with TLSDESC

LP64			ILP32		
0x00	leaq	x@tlsdesc(%rip),%rax	0x00	rex leal	x@tlsdesc(%rip),%eax
0x07	call	*x@tlsdesc(%rax)	0x07	call	*x@tlsdesc(%eax)
0x08	add	%fs:0x0,%eax	0x09	add	%fs:0x0,%eax

Initial Exec Model Load address of `x` into `%rax`. Instruction `addl` must be encoded with `rex` prefix even if it isn't required by destination register. Otherwise linker can't safely perform IE -> LE optimization.

Table 10.3: Initial Exec Model Code Sequence

LP64			ILP32		
0x00	movq	%fs:0,%rax	0x00	movl	%fs:0,%eax
0x09	addq	x@gottpoff(%rip),%rax	0x08	rex addl	x@gottpoff(%rip),%eax

Initial Exec Model, II Load value of `x` into `%edi`. `%fs:(%eax)` memory operand can't be used for ILP32 since its effective address is the base address of `%fs` + value of `%eax` zero-extended to a 64-bit result, which is incorrect with negative value in `%eax`.

Table 10.4: Initial Exec Model Code Sequence, II

LP64			ILP32		
0x00	movq	x@gottpoff(%rip),%rax	0x00	movq	x@gottpoff(%rip),%rax
0x07	movl	%fs:(%rax),%edi	0x07	movl	%fs:(%rax),%edi

10.3.2 Static Thread-Local Variable

For a static thread-local variable x:

```
static __thread int x;
```

Local Dynamic Model Load address of x into %rax

Table 10.5: Local Dynamic Model Code Sequence With Lea

LP64			ILP32		
0x00	leaq	x@tlsld(%rip),%rdi	0x00	leaq	x@tlsld(%rip),%rdi
0x07	call	__tls_get_addr@plt	0x07	call	__tls_get_addr@plt
0x0c	leaq	x@dtppoff(%rax),%rax	0x0c	leal	x@dtppoff(%rax),%eax

or

Table 10.6: Local Dynamic Model Code Sequence With Add

LP64			ILP32		
0x00	leaq	x@tlsld(%rip),%rdi	0x00	leaq	x@tlsld(%rip),%rdi
0x07	call	__tls_get_addr@plt	0x07	call	__tls_get_addr@plt
0x0c	addq	\$x@dtppoff,%rax	0x0c	addl	\$x@dtppoff,%eax

For code sequence with TLSDESC, local dynamic model is similar to general dynamic model. The same encoding requirement for `leal` instruction also applies.

Table 10.7: General Dynamic Model Code Sequence with TLSDESC

LP64			ILP32		
0x00	leaq	x@tlsdesc(%rip),%rax	0x00	rex leal	x@tlsdesc(%rip),%eax
0x07	call	*x@tlsdesc(%rax)	0x07	call	*x@tlsdesc(%eax)
0x08	add	%fs:0x0,%eax	0x09	add	%fs:0x0,%eax

Local Dynamic Model, II Load value of x into $\%edi$

Table 10.8: Local Dynamic Model Code Sequence, II

LP64			ILP32		
0x00	movq	%fs:0,%rax	0x00	movl	%fs:0,%eax
0x09	movl	x@dtppoff(%rax),%edi	0x08	movl	x@dtppoff(%rax),%edi

Local Exec Model Load address of x into $\%rax$

Table 10.9: Local Exec Model Code Sequence With Lea

LP64			ILP32		
0x00	movq	%fs:0,%rax	0x00	movl	%fs:0,%eax
0x09	leaq	x@tpoff(%rax),%rax	0x08	leal	x@tpoff(%rax),%eax

or

Table 10.10: Local Exec Model Code Sequence With Add

LP64			ILP32		
0x00	movq	%fs:0,%rax	0x00	movl	%fs:0,%eax
0x09	addq	\$x@tpoff,%rax	0x08	addl	\$x@tpoff,%eax

Local Exec Model, II Load value of x into $\%edi$

Table 10.11: Local Exec Model Code Sequence, II

LP64			ILP32		
0x00	movq	%fs:0,%rax	0x00	movl	%fs:0,%eax
0x09	movl	x@tpoff(%rax),%edi	0x08	movl	x@tpoff(%rax),%edi

Local Exec Model, III Load value of x into %edi

Table 10.12: Local Exec Model Code Sequence, III

LP64			ILP32		
0x00	movl	%fs:x@tpoff,%edi	0x00	movl	%fs:x@tpoff,%edi

10.3.3 TLS Linker Optimization

General Dynamic To Initial Exec Load address of x into %rax

Table 10.13: GD -> IE Code Transition

GD			IE		
0x00	leaq	x@tlsgd(%rip),%rdi	0x00	movl	%fs:0,%eax
0x07	.word	0x6666	0x08	addq	x@gottpoff(%rip),%rax
0x09	rex64				
0x0a	call	__tls_get_addr@plt			

Table 10.14: GDesc -> IE Code Transition

GDesc			LE		
0x00	rex leal	x@tlsdes(%rip),%eax	0x00	rex movl	x@gottpoff(%rip), %eax
0x07	call	*foo@TLSCALL(%eax)	0x07	nopl	(%rax)

Table 10.15: GD -> LE Code Transition

GD			LE		
0x00	leaq	x@tlsgd(%rip),%rdi	0x00	movl	%fs:0,%eax
0x07	.word	0x6666	0x08	leal	x@tpoff(%rax),%eax
0x09	rex64				
0x0a	call	__tls_get_addr@plt			

Table 10.16: GDesc -> LE Code Transition

GDesc			LE		
0x00	rex leal	x@tlsdes(%rip),%eax	0x00	rex movl	\$x@tpoff,%eax
0x07	call	*foo@TLSCALL(%eax)	0x07	nopl	(%rax)

Initial Exec To Local Exec Load address of x into $\%rax$

Table 10.17: IE -> LE Code Transition With Lea

IE			LE		
0x00	movl	%fs:0,%eax	0x00	movl	%fs:0,%eax
0x08	rex addl	x@gottpoff(%rip),%eax	0x08	rex leal	x@tpoff(%rax),%eax

or

Table 10.18: IE -> LE Code Transition With Add

IE			LE		
0x00	movl	%fs:0,%r12d	0x00	movl	%fs:0,%r12d
0x09	addl	x@gottpoff(%rip),%r12d	0x09	addl	\$x@tpoff,%r12d

Initial Exec To Local Exec, II Load value of x into $\%edi$.

Table 10.19: IE -> LE Code Transition, II

IE			LE		
0x00	movq	x@gottpoff(%rip),%rax	0x00	movq	x@tpoff,%rax
0x07	movl	%fs:(%rax),%edi	0x07	movl	%fs:(%rax),%edi

Local Dynamic to Local Exec Load address of x into %rax

Table 10.20: LD -> LE Code Transition With Lea

LD			LE		
0x00	leaq	x@tlsld(%rip),%rdi	0x00	nopl	0x0(%rax)
0x07	call	__tls_get_addr@plt	0x04	movl	%fs:0,%eax
0x0c	leal	x@dtppoff(%rax),%eax	0x0c	leal	x@tpoff(%rax),%eax

or

Table 10.21: LD -> LE Code Transition With Add

LD			LE		
0x00	leaq	x@tlsld(%rip),%rdi	0x00	nopl	0x0(%rax)
0x07	call	__tls_get_addr@plt	0x04	movl	%fs:0,%eax
0x0c	addq	\$x@dtppoff,%rax	0x0c	addl	\$x@tpoff,%eax

Local Dynamic To Local Exec, II Load value of x into %edi.

Table 10.22: LD -> LE Code Transition, II

LD			LE		
0x00	leaq	x@tlsld(%rip),%rdi	0x00	nopl	0x0(%rax)
0x07	call	__tls_get_addr@plt	0x04	movl	%fs:0,%eax
0x0c	movl	x@dtppoff(%rax),%eax	0x0c	movl	x@tpoff(%rax),%eax

10.4 Kernel Support

Kernel should limit stack and addresses returned from system calls between `0x00000000` to `0xffffffff`.

10.5 Coding Examples

Although ILP32 binaries run in the 64-bit mode, not all 64-bit instructions are supported. This section discusses example code sequences for fundamental operations which are different from the 64-bit mode.

10.5.1 Indirect Branch

Since indirect branch via memory loads a 64-bit address at the memory location, it is not supported in ILP32. Indirect branch via register should be used instead. The 32-bit address from memory is loaded into the lower 32 bits of a register, which will automatically zero-extend the upper 32 bits of the register. Then the indirect call can be performed via the 64-bit register.

Table 10.23: Indirect Branch

LP64		ILP32	
call	*%rax	call	*%rax
call	*func_p(%rip)	movl	func_p(%rip), %eax
		call	*%rax
call	*func_p	movl	func_p, %eax
		call	*%rax
jmp	*%rax	jmp	*%rax
jmp	*func_p(%rip)	movl	func_p(%rip), %eax
		jmp	*%rax
jmp	*func_p	movl	func_p, %eax
		jmp	*%rax

Chapter 11

Alternate Code Sequences For Security

11.1 Code Sequences without PLT

Procedure Linkage Table (PLT), see Section 5.2 for detail, is used to access external functions defined in shared object and support

Lazy symbol resolution The function address is resolved only when it is called the first time at run-time.

Canonical function address The PLT entry of the external function is used as its address, aka function pointer.

The first instruction in the PLT entry is an indirect branch via the Global Offset Table (GOT), see Section 5.2 for detail, entry of the external function, which is set up in such a way that it will be updated to the address of the function body the first time when the function is called. Since the GOT entry is writable, any address may be written to it at run-time, which is a potential security risk.

11.1.1 Indirect Call via the GOT Slot

For small and medium models, different code sequence is used to avoid PLT:

Figure 11.1: Function Call without PLT (Small and Medium Models)

```
extern void func (void);  
func (void);
```

```
.globl func  
call *func@GOTPCREL(%rip)
```

The direct branch is replaced by an indirect branch via the GOT slot, which is similar to the first instruction in the PLT slot.

Figure 11.2: Function Address without PLT (Small and Medium Models)

<pre>extern void func (void); void* ptr (void) { return func; }</pre>	<pre>.globl func func: movq func@GOTPCREL(%rip), %rax ret</pre>
---	---

Instead using the PLT slot as function address, the function address is retrieved from the GOT slot.

If linker determines the function is defined locally, it converts indirect branch via the GOT slot to direct branch with a `nop` prefix and converts load via the GOT slot to load immediate or `lea`, see Section B.2 for details.

After dynamic linker resolved all symbols by updating GOT entries with symbol addresses, GOT can be made read-only and overwriting GOT becomes a hard error immediately. Since PLT is no longer used to call external function, lazy symbol resolution is disabled and a function can only be interposed during symbol resolution at startup. Tools and features which depend on lazy symbol resolution will not work properly. However, there are also a few side benefits:

No extra direct branch to PLT entry Since indirect branch is 6 byte long and direct branch is 5 byte long, when indirect branch via the GOT slot is used to call a local function, code size will be increased by one byte for each call. Since one PLT slot has 16 bytes, there will be code size increase when indirect branch via the GOT slot is used to call an external function more than 16 times.

Custom calling convention Since external function is called directly via the GOT slot, instead of invoking dynamic linker to lookup function symbol when called the first time, parameters can be passed differently from what is specified in this document.

11.1.2 Thread-Local Storage without PLT

TLS code sequences for general and local dynamic models can be updated to replace direct call to `__tls_get_addr` via the PLT entry, with indirect call to `__tls_get_addr` via the GOT slot, see Figure 11.3. Since direct `call` instruction is 4-byte long and indirect `call` instruction is 5-byte long, the extra one byte must be handled properly.

Figure 11.3: `__tls_get_addr` Call

Direct via PLT	Indirect via GOT
<code>call __tls_get_addr@PLT</code>	<code>call *__tls_get_addr@GOTPCREL(%rip)</code>

General Dynamic Model for Global Variable

For general dynamic model, one `0x66` prefix before `call` instruction is removed to make room for indirect call:

```
extern __thread int x;
```

the following alternate code sequence loads address of `x` into `%rax` without PLT:

Table 11.1: General Dynamic Model Code Sequence (LP64)

With PLT			Without PLT		
0x00	.byte	0x66	0x00	.byte	0x66
0x01	leaq	<code>x@tls_gd(%rip),%rdi</code>	0x01	leaq	<code>x@tls_gd(%rip),%rdi</code>
0x08	.word	0x6666	0x08	.byte	0x66
0x0a	rex64		0x09	rex64	
0x0b	call	<code>__tls_get_addr@PLT</code>	0x0a	call	<code>*__tls_get_addr@GOTPCREL(%rip)</code>

Table 11.2: General Dynamic Model Code Sequence (ILP32)

With PLT			Without PLT		
0x00	leaq	<code>x@tls_gd(%rip),%rdi</code>	0x00	leaq	<code>x@tls_gd(%rip),%rdi</code>
0x07	.word	0x6666	0x07	.byte	0x66
0x09	rex64		0x08	rex64	
0x0a	call	<code>__tls_get_addr@PLT</code>	0x09	call	<code>*__tls_get_addr@GOTPCREL(%rip)</code>

Static Thread-Local Variable

For local dynamic model, indirect call is used instead of direct call:

```
static __thread int x;
```

the following alternate code sequence loads the address of the TLS block of the module, which contains variable `x`, into `%rax` without PLT:

Table 11.3: Local Dynamic Model Code Sequence (LP64)

With PLT			Without PLT		
0x00	leaq	x@tlsld(%rip),%rdi	0x00	leaq	x@tlsld(%rip),%rdi
0x07	call	__tls_get_addr@PLT	0x07	call	*__tls_get_addr@GOTPCREL(%rip)

Table 11.4: Local Dynamic Model Code Sequence (ILP32)

With PLT			Without PLT		
0x00	leaq	x@tlsld(%rip),%rdi	0x00	leaq	x@tlsld(%rip),%rdi
0x07	call	__tls_get_addr@PLT	0x07	call	*__tls_get_addr@GOTPCREL(%rip)

TLS Linker Optimization

Since the code sequence with indirect call for general dynamic model has the same length as the one with direct call, linker just needs to recognize new instruction pattern to convert general dynamic access to initial exec or local exec accesses.

General Dynamic to Initial Exec To load address of `x` into `%rax`:

Table 11.5: GD -> IE Code Transition (LP64)

GD			IE		
0x00	.byte	0x66	0x00	movq	%fs:0, %rax
0x01	leaq	x@tlsgd(%rip),%rdi	0x09	addq	x@gottpoff(%rip),%rax
0x09	.byte	0x66			
0x0a	rex64				
0x0b	call	*__tls_get_addr@GOTPCREL(%rip)			

Table 11.6: GD -> IE Code Transition (ILP32)

GD			IE		
0x00	leaq	x@tls_gd(%rip),%rdi	0x00	movl	%fs:0, %eax
0x07	.byte	0x66	0x08	addq	x@gottpoff(%rip),%rax
0x08	rex64				
0x0a	call	*__tls_get_addr@GOTPCREL(%rip)			

General Dynamic to Local Exec To load address of `x` into `%rax`:

Table 11.7: GD -> LE Code Transition (LP64)

GD			LE		
0x00	.byte	0x66	0x00	movq	%fs:0, %rax
0x01	leaq	x@tls_gd(%rip),%rdi	0x09	leaq	x@tpoff(%rax),%rax
0x08	.byte	0x66			
0x09	rex64				
0x0a	call	*__tls_get_addr@GOTPCREL(%rip)			

Table 11.8: GD -> LE Code Transition (ILP32)

GD			LE		
0x00	leaq	x@tls_gd(%rip),%rdi	0x00	movl	%fs:0, %eax
0x07	.byte	0x66	0x08	leaq	x@tpoff(%rax),%rax
0x08	rex64				
0x09	call	*__tls_get_addr@GOTPCREL(%rip)			

Local Dynamic to Local Exec For local dynamic model to local exec model transition, linker generates 4 `0x66` prefixes, instead of 3, before `mov` instruction for LP64 and generate a 5-byte `nop`, instead of 4-byte, before `mov` instruction for ILP32. To load the address of the TLS block of the module, which contains variable `x`, into `%rax` without PLT:

Table 11.9: LD -> LE Code Transition (LP64)

LD			LE		
0x00	leaq	x@tlsld(%rip),%rdi	0x00	.long	0x66666666
0x07	call	*__tls_get_addr@GOTPCREL(%rip)	0x04	movq	%fs:0,%rax

Table 11.10: LD -> LE Code Transition (ILP32)

LD			LE		
0x00	leaq	x@tlsld(%rip),%rdi	0x00	nopw	0x0(%rax)
0x07	call	*__tls_get_addr@GOTPCREL(%rip)	0x05	movl	%fs:0,%eax

Chapter 12

Intel CET Extension

Intel CET (Control-flow Enforcement Technology) Extension includes:

IBT (Indirect Branch Tracking) Branch protection to defend against Jump/Call Oriented Programming.

SHSTK (Shadow Stack) Return address protection to defend against Return Oriented Programming,

12.1 Program Loading

12.1.1 Process `GNU_PROPERTY_X86_FEATURE_1_IBT`

On an IBT capable processor, the following steps should be taken:

1. When loading an executable without interpreter, enable IBT if `GNU_PROPERTY_X86_FEATURE_1_IBT` is set on the executable.
2. When loading an executable with an interpreter, enable IBT if `GNU_PROPERTY_X86_FEATURE_1_IBT` is set on the interpreter. The interpreter should disable IBT if `GNU_PROPERTY_X86_FEATURE_1_IBT` isn't set on the executable or any shared objects loaded via the `DT_NEEDED` tag.
3. After IBT is enabled, when loading a shared object ¹ without `GNU_PROPERTY_X86_FEATURE_1_IBT`:

¹The legacy code page bitmap isn't used since it can't cover the jitted code generated by legacy JIT engines.

- (a) In single-threaded applications, if legacy interwork is allowed, IBT should be disabled.
- (b) In multi-threaded applications or if legacy interwork isn't allowed, it causes an error.

12.1.2 Process `GNU_PROPERTY_X86_FEATURE_1_SHSTK`

On a SHSTK capable processor, the following steps should be taken:

1. When loading an executable without interpreter, enable SHSTK if `GNU_PROPERTY_X86_FEATURE_1_SHSTK` is set on the executable.
2. When loading an executable with an interpreter, enable SHSTK if `GNU_PROPERTY_X86_FEATURE_1_SHSTK` is set on the interpreter. The interpreter should disable SHSTK if `GNU_PROPERTY_X86_FEATURE_1_SHSTK` isn't set on the executable or any shared objects loaded via the `DT_NEEDED` tag.
3. After SHSTK is enabled, when loading a shared object without `GNU_PROPERTY_X86_FEATURE_1_SHSTK`:
 - (a) In single-threaded applications, if legacy interwork is allowed, SHSTK should be disabled.
 - (b) In multi-threaded applications or if legacy interwork isn't allowed, it causes an error.

12.2 Dynamic Linking

To support IBT, linker should generate a IBT-enabled PLT (Procedure Linkage Table) (see figure 12.1) together with a second PLT (see figure 12.2).

Figure 12.1: The IBT-enabled PLT (small and medium models)

```
.PLT0: pushq    GOT+8(%rip)           # GOT[1]
      jmp     *GOT+16(%rip)        # GOT[2]
      nopl    0x0(%rax)
.PLT1: endbr64                      # 16 bytes from .PLT0
      pushq   $index1
      jmp     .PLT0
      xchg    %ax,%ax
.PLT2: endbr64                      # 16 bytes from .PLT1
      pushq   $index2
      jmp     .PLT0
      xchg    %ax,%ax
.PLT3: ...
```

Figure 12.2: The Second IBT-enabled PLT (small and medium models)

```
.SPLT1: endbr64
      jmp     *name1@GOTPCREL(%rip)
      nopw    0x0(%rax,%rax,1)
.SPLT2: endbr64
      jmp     *name2@GOTPCREL(%rip) # 16 bytes from .SPLT1
      nopw    0x0(%rax,%rax,1)
.SPLT3: ...                          # 16 bytes from .SPLT2
```

When the IBT-enabled procedure linkage table is used, the initial value of the global offset table entry for the external function is the address of the corresponding entry of the second procedure linkage table.

Appendix A

Linux Conventions

This chapter describes some details that are only relevant to GNU/Linux systems and the Linux kernel.

A.1 Execution of 32-bit Programs

The AMD64 processors are able to execute 64-bit AMD64 and also 32-bit ia32 programs. Libraries conforming to the Intel386 ABI will live in the normal places like `/lib`, `/usr/lib` and `/usr/bin`. Libraries following the AMD64, will use `lib64` subdirectories for the libraries, e.g `/lib64` and `/usr/lib64`. Programs conforming to Intel386 ABI and to the AMD64 ABI will share directories like `/usr/bin`. In particular, there will be no `/bin64` directory.

A.2 AMD64 Linux Kernel Conventions

The section is informative only.

A.2.1 Calling Conventions

The Linux AMD64 kernel uses internally the same calling conventions as user-level applications (see section 3.2.3 for details). User-level applications that like to call system calls should use the functions from the C library. The interface between the C library and the Linux kernel is the same as for the user-level applications with the following differences:

1. User-level applications use as integer registers for passing the sequence `%rdi`, `%rsi`, `%rdx`, `%rcx`, `%r8` and `%r9`. The kernel interface uses `%rdi`, `%rsi`, `%rdx`, `%r10`, `%r8` and `%r9`.
2. A system-call is done via the `syscall` instruction. The kernel clobbers registers `%rcx` and `%r11` but preserves all other registers except `%rax`.
3. The number of the syscall has to be passed in register `%rax`.
4. System-calls are limited to six arguments, no argument is passed directly on the stack.
5. Returning from the `syscall`, register `%rax` contains the result of the system-call. A value in the range between -4095 and -1 indicates an error, it is `-errno`.
6. Only values of class INTEGER or class MEMORY are passed to the kernel.

A.2.2 Stack Layout

The Linux kernel may align the end of the input argument area to a 8, instead of 16, byte boundary. It does not honor the red zone (see section 3.2.2) and therefore this area is not allowed to be used by kernel code. Kernel code should be compiled by GCC with the option `-mno-red-zone`.

A.2.3 Miscellaneous Remarks

Linux Kernel code is not allowed to change the x87 and SSE units. If those are changed by kernel code, they have to be restored properly before sleeping or leaving the kernel. On preemptive kernels also more precautions may be needed.

Appendix B

Linker Optimization

This chapter describes optimizations which may be performed by linker.

B.1 Combine GOTPLT and GOT Slots

In the small and medium models, when there are both PLT and GOT references to the same function symbol, normally linker creates a GOTPLT slot for PLT entry and a GOT slot for GOT reference. A run-time JUMP_SLOT relocation is created to update the GOT-PLT slot and a run-time GLOB_DAT relocation is created to update the GOT slot. Both JUMP_SLOT and GLOB_DAT relocations apply the same symbol value to GOTPLT and GOT slots, respectively, at run-time.

As an optimization, linker may combine GOTPLT and GOT slots into a single GOT slot and remove the run-time JUMP_SLOT relocation. It replaces the regular PLT entry:

Figure B.1: Procedure Linkage Table Entry Via GOTPLT Slot

```
.PLT:  jmp      [GOTPLT slot]
       pushq   relocation index
       jmp     .PLT0
```

with an GOT PLT entry with an indirect jump via the GOT slot:

Figure B.2: Procedure Linkage Table Entry Via GOT Slot

```
.PLT: jmp      [GOT slot]
      nop
```

and resolves the PLT reference to the GOT PLT entry. Indirect `jmp` is an 5-byte instruction. `nop` can be encoded as a 3-byte instruction or a 11-byte instruction for 8-byte or 16-byte PLT slot. A separate PLT with 8-byte slots may be used for this optimization.

This optimization isn't applicable to the `STT_GNU_IFUNC` symbols since their GOT-PLT slots are resolved to the selected implementation and their GOT slots are resolved to their PLT entries.

This optimization must be avoided if pointer equality is needed since the symbol value won't be cleared in this case and the dynamic linker won't update the GOT slot. Otherwise, the resulting binary will get into an infinite loop at run-time.

B.2 Optimize GOTPCRELX Relocations

The AMD64 instruction encoding supports converting certain instructions on memory operand with `R_X86_64_GOTPCRELX`, `R_X86_64_REX_GOTPCRELX` or `R_X86_64_CODE_4_GOTPCRELX` if the first byte of the instruction at the relocation offset - 4 is 0xd5, relocations against symbol, `foo`, into a different form on immediate operand if `foo` is defined locally and the relocation addend is -4.

Convert call and jmp Convert memory operand of `call` and `jmp` into immediate operand.

Table B.1: Call and Jmp Conversion

Memory Operand	Immediate Operand
<code>call *foo@GOTPCREL(%rip)</code>	<code>nop call foo</code>
<code>call *foo@GOTPCREL(%rip)</code>	<code>call foo nop</code>
<code>jmp *foo@GOTPCREL(%rip)</code>	<code>jmp foo nop</code>

Convert mov Convert memory operand of `mov` into immediate operand. When position-independent code is disabled and `foo` is defined locally in the lower 32-bit address

space, memory operand in `mov` can be converted into immediate operand. Otherwise, `mov` must be changed to `lea`.

Table B.2: Mov Conversion

Memory Operand	Immediate Operand
<code>mov foo@GOTPCREL(%rip), %reg</code>	<code>mov \$foo, %reg</code>
<code>mov foo@GOTPCREL(%rip), %reg</code>	<code>lea foo(%rip), %reg</code>

Convert Test and Binop Convert memory operand of `test` and `binop` into immediate operand, where `binop` is one of `adc`, `add`, `and`, `cmp`, `or`, `sbb`, `sub`, `xor` instructions, when position-independent code is disabled.

Table B.3: Test and Binop Conversion

Memory Operand	Immediate Operand
<code>test %reg, foo@GOTPCREL(%rip)</code>	<code>test \$foo, %reg</code>
<code>binop foo@GOTPCREL(%rip), %reg</code>	<code>binop \$foo, %reg</code>

Index

- `.cfi_adjust_cfa_offset`, 100
- `.cfi_def_cfa`, 100
- `.cfi_def_cfa_offset`, 100
- `.cfi_def_cfa_register`, 100
- `.cfi_endproc`, 100
- `.cfi_escape`, 100
- `.cfi_offset`, 100
- `.cfi_rel_offset`, 100
- `.cfi_startproc`, 100
- `.eh_frame`, 100
- `%rax`, 49
- `_UA_CLEANUP_PHASE`, 88
- `_UA_FORCE_UNWIND`, 87
- `_UA_SEARCH_PHASE`, 87
- `_Unwind_Context`, 86
- `_Unwind_DeleteException`, 86
- `_Unwind_Exception`, 86
- `_Unwind_ForcedUnwind`, 86, 87
- `_Unwind_GetCFA`, 86
- `_Unwind_GetGR`, 86
- `_Unwind_GetIP`, 86
- `_Unwind_GetIPInfo`, 86
- `_Unwind_GetLanguageSpecificData`, 86
- `_Unwind_GetRegionStart`, 86
- `_Unwind_RaiseException`, 86, 87
- `_Unwind_Resume`, 86
- `_Unwind_SetGR`, 86
- `_Unwind_SetIP`, 86
- `__float128`, 9

- auxiliary vector, [30](#)
- boolean, [11](#)
- byte, [9](#)
- C++, [105](#)
- Call Frame Information tables, [85](#)
- code models, [33](#)
- Convert call and jmp, [141](#)
- Convert mov, [141](#)
- Convert Test and Binop, [142](#)
- double quadword, [9](#)
- doubleword, [9](#)
- DT_FINI, [80](#)
- DT_FINI_ARRAY, [80](#)
- DT_INIT, [80](#)
- DT_INIT_ARRAY, [80](#)
- DT_JMPREL, [77](#)
- DT_PREINIT_ARRAY, [80](#)
- DWARF Debugging Information Format, [85](#), [99](#)
- eightbyte, [9](#)
- exceptions, [23](#)
- exec, [28](#)
- fegetround, [85](#)
- fourbyte, [9](#)
- General Dynamic Model, [122](#)
- General Dynamic To Initial Exec, [125](#)
- global offset table, [75](#)
- halfword, [9](#)
- Initial Exec Model, [122](#)
- Initial Exec Model, II, [122](#)
- Initial Exec To Local Exec, [126](#)
- Initial Exec To Local Exec, II, [126](#)
- Kernel code model, [34](#)

- Large code model, [34](#)
- Large position independent code model, [35](#)
- Local Dynamic Model, [123](#)
- Local Dynamic Model, II, [124](#)
- Local Dynamic to Local Exec, [127](#)
- Local Dynamic To Local Exec, II, [127](#)
- Local Exec Model, [124](#)
- Local Exec Model, II, [124](#)
- Local Exec Model, III, [125](#)
- longjmp, [86](#)

- Medium code model, [34](#)
- Medium position independent code model, [35](#)
- micro-architecture levels, [7](#)

- PIC, [35](#), [121](#)
- Procedure Linkage Table, [66](#)
- procedure linkage table, [76–78](#)
- program interpreter, [79](#)

- quadword, [9](#)

- R_X86_64_JUMP_SLOT, [77](#), [78](#)
- R_X86_64_TLSDESC, [78](#)
- red zone, [16](#), [139](#)
- register save area, [50](#)

- signal, [23](#)
- sixteenbyte, [9](#)
- size_t, [11](#)
- Small code model, [34](#)
- small code model, [121](#)
- Small position independent code model, [35](#)
- small position independent code model, [121](#)

- terminate(), [87](#)
- Thread-Local Storage, [69](#)
- twobyte, [9](#)

- Unwind Library interface, [85](#)

va_arg, [52](#)
va_list, [51](#)
va_start, [51](#)

word, [9](#)